

UNIVERSITAT POLITÈCNICA DE
CATALUNYA (UPC) - BARCELONATECH

THESIS FOR THE DOCTORAL DEGREE IN COMPUTER
ARCHITECTURE

Network Anomaly Detection with Graph Neural Networks

by:

Hamid Latif Martínez

Supervisors:

Dr. Pere Barlet Ros

Dr. Albert Cabellos Aparicio

Thesis for the Doctoral Degree in Computer Architecture

Barcelona Neural Networking Center (BNN)
Departament d'Arquitectura de Computadors (DAC)

July, 2025

Acknowledgements

This thesis would not have been possible without the tireless work of my supervisors, Professor Pere Barlet and Professor Albert Cabellos, both of whom have provided me an endless stream of ideas, comments, suggestions and corrections that have helped me keep to not lose my way during these 4 years. Their dedication, mentorship and help have been essential to support me during every step in this journey. I deeply appreciate the opportunity that they have given me to work with them. I am certain that, without all of your advice and guidance, none of this would have been possible.

I would like to extend my gratitude to Dr. Jordi Paillissé and Dr. José Suárez Varela, whose comments and insights have provided an invaluable help during a great part of my journey. I am very thankful for every meeting and every idea that you shared with me. I am more than sure that the results of this thesis would have been worse without your participation.

To the rest of the members of the Barcelona Neural Networking Center, both present and past, and to the members of the Computer Architecture Department, thank you. Professor Sergi Abadal, Albert López, Dr. Paul Almasan, Dr. Guillermo Bernárdez, Dr. Miquel Ferriol, Berta Serracanta, Carlos Güemes and Spyros Garyfallos. You have been the best companions that I could have wished for during these 4 years. I will truly miss working with you and I will remember fondly our post-work meetings.

I would also like to dedicate a few words to Dr. Pedro Casas, Juan Vaneiro, Gonzalo Sobarzo and Martín Randall. Even in the coldest of the Viennese winter, you really made me feel at home. I learned a lot during my 6 months in Vienna and enjoyed my stay thanks to you all. ¡Gracias!

This thesis has been partially funded by the project by the Spanish I+D+i project TRAINER-A (ref.PID2020-118011GB-C21), funded by MCIN/ AEI/1-0.13039/501100011033. This work is also partially funded by the Catalan Institution for Research and Advanced Studies (ICREA). This work was also supported by the CHISTERA grant CHIST-ERA-22-SPiDDS-02 corresponding to the GRAPHS4SEC project (reference n° PCI2023-145974-2) funded by the Agencia Estatal de Investigación through the PCI 2023 call.

Lastly, I would like to express my deepest gratitude to my family and friends for your support during all these years. Thank you for believing in me when even I did not do, thank you for cheering me up when I was most discouraged, thank you for always being by my side. Your support and love has allowed me to become what I have become in life, and nothing would have been possible without you.

Para acabar, me gustaría dedicarle unas palabras en mi lengua materna a mi familia más directa. A mi madre Rosario, mi hermano Jabar y mis tías Emilia y Mari, que han sido como unas segundas madres para mí. Gracias por haber hecho posible este largo camino y por haber hecho de mí la persona que soy. Nada de lo que he conseguido hubiese sido posible sin vosotros. Y

a mi novia Cynthia, gracias por tu apoyo y confianza constantes, por acompañarme en cada paso y por seguir escogiéndome cada día.

Abstract

Modern networks support critical applications with increasingly diverse, complex, and dynamic requirements. Ensuring their proper functioning is vital to maintaining the reliability and availability of these dependent services. Anomaly Detection (AD) plays a crucial role in preserving network performance by promptly identifying disruptions or degradations. In computer networks, the behavior of a node (e.g., a router) is heavily influenced by its surrounding elements. However, many traditional and Machine Learning (ML)-based methods often fail to exploit these structural correlations effectively, and in some cases ignore them entirely.

While some existing approaches attempt to incorporate topological information into the AD task, they often do so suboptimally, losing key relational information. As a result, these systems struggle to achieve the robustness and adaptability required for deployment in dynamic, production environments.

Graph Neural Networks (GNNs) have recently gained prominence for modeling tasks where spatial correlations are essential. Nevertheless, their application to AD in computer networks remains underexplored, despite their natural suitability for such structured environments. This thesis investigates new uses of GNNs for AD in networked systems, focusing on three representative scenarios: (i) Border Gateway Protocol (BGP) AD, where hijacks and misconfigurations cause structural changes; (ii) Contextual Anomaly Detection (CAD), where anomalies depend on the behavior of neighboring entities; and (iii) zero-shot spatiotemporal modeling, enabling forecasting in data-scarce environments.

In the first scenario, we leverage the raw BGP topology to design a model capable of generalizing across incidents while maintaining a low false-positive rate, an essential requirement for real-world deployment. We evaluate the model on well-known BGP incidents and extensive regular operation data to validate its robustness.

For CAD, we propose two GNN-based models that uncover hidden relationships between flows in a backbone network. We validate these models on datasets containing both real and synthetic anomalies and show how the attention mechanisms provide interpretable visualizations of the learned contextual dependencies.

Finally, we introduce a GNN-based spatiotemporal model trained on non-networking time series, capable of achieving zero-shot forecasting performance on network monitoring data. The model exploits attention mechanisms to learn and transfer spatial and temporal correlations across domains, outperforming state-of-the-art baselines. Moreover, the learned attention patterns suggest opportunities to simplify spatiotemporal representations, which is critical for scaling models to large production networks.

Collectively, the contributions of this thesis advance the use of Graph Neural Networks for Anomaly Detection in computer networks by systematically

leveraging topological information to improve predictive performance and interpretability. Furthermore, the role of attention mechanisms for providing more interpretable results is also explored, opening the door to using them as a way to both understand the reasons behind the detected anomalies and to provide more transparency to the underlying dynamics in computer networks.

Resumen

Las redes modernas admiten aplicaciones críticas con requisitos cada vez más diversos, complejos y dinámicos. Garantizar su correcto funcionamiento es vital para mantener la fiabilidad y la disponibilidad de estos servicios dependientes. La Detección de Anomalías (AD) desempeña un papel crucial en la preservación del rendimiento de la red al identificar rápidamente interrupciones o degradaciones. En las redes informáticas, el comportamiento de un nodo (por ejemplo, un enrutador) está muy influenciado por los elementos que lo rodean. Sin embargo, muchos métodos tradicionales y basados en Aprendizaje Automático (ML) a menudo no aprovechan eficazmente estas correlaciones estructurales y, en algunos casos, las ignoran por completo.

Si bien algunos enfoques existentes intentan incorporar información topológica en la tarea de AD, a menudo lo hacen de forma subóptima, perdiendo información relacional clave. Como resultado, estos sistemas tienen dificultades para lograr la robustez y adaptabilidad necesarias para su implementación en entornos dinámicos de producción.

Las Redes Neuronales Gráficas (GNN) han cobrado relevancia recientemente para tareas de modelado donde las correlaciones espaciales son esenciales. No obstante, su aplicación a la AD en redes informáticas sigue siendo poco explorada, a pesar de su idoneidad natural para estos entornos estructurados. Esta tesis investiga nuevos usos de las GNN para AD en sistemas en red, centrándose en tres escenarios representativos: (i) AD del Protocolo de Puerta de Enlace Fronteriza (BGP), donde los secuestros y las configuraciones incorrectas causan cambios estructurales; (ii) Detección de Anomalías Contextuales (CAD), donde las anomalías dependen del comportamiento de las entidades vecinas; y (iii) modelado espaciotemporal de disparo cero, que permite la predicción en entornos con escasez de datos.

En el primer escenario, aprovechamos la topología BGP sin procesar para diseñar un modelo capaz de generalizarse entre incidentes, manteniendo una baja tasa de falsos positivos, un requisito esencial para la implementación en el mundo real. Evaluamos el modelo con incidentes BGP conocidos y amplios datos de operación regular para validar su robustez.

Para CAD, proponemos dos modelos basados en GNN que revelan relaciones ocultas entre flujos en una red troncal. Validamos estos modelos con conjuntos de datos que contienen anomalías reales y sintéticas, y mostramos cómo los mecanismos de atención proporcionan visualizaciones interpretables de las dependencias contextuales aprendidas.

Finalmente, presentamos un modelo espaciotemporal basado en GNN, entrenado con series temporales sin conexión a red, capaz de lograr un rendimiento de predicción de disparo cero en datos de monitorización de redes. El modelo aprovecha los mecanismos de atención para aprender

y transferir correlaciones espaciales y temporales entre dominios, superando las líneas base más modernas. Además, los patrones de atención aprendidos sugieren oportunidades para simplificar las representaciones espaciotemporales, lo cual es crucial para escalar modelos a grandes redes de producción.

En conjunto, las contribuciones de esta tesis impulsan el uso de redes neuronales de grafos para la detección de anomalías en redes informáticas, aprovechando sistemáticamente la información topológica para mejorar el rendimiento predictivo y la interpretabilidad. Asimismo, se explora el papel de los mecanismos de atención para proporcionar resultados más interpretables, lo que facilita su uso tanto para comprender las razones de las anomalías detectadas como para aportar mayor transparencia a la dinámica subyacente en las redes informáticas.

Resum

Les xarxes modernes admeten aplicacions crítiques amb requisits cada cop més diversos, complexos i dinàmics. Assegurar el seu correcte funcionament és vital per mantenir la fiabilitat i la disponibilitat d'aquests serveis dependents. La detecció d'anomalies (AD) juga un paper crucial en la preservació del rendiment de la xarxa mitjançant la identificació ràpida d'interrupcions o degradacions. A les xarxes informàtiques, el comportament d'un node (per exemple, un encaminador) està molt influenciat pels elements que l'envolten. Tanmateix, molts mètodes tradicionals i basats en l'aprenentatge automàtic (ML) sovint no aconsegueixen explotar aquestes correlacions estructurals de manera eficaç i, en alguns casos, les ignoren completament.

Si bé alguns enfocaments existents intenten incorporar informació topològica a la tasca d'AD, sovint ho fan de manera subòptima, perdent informació relacional clau. Com a resultat, aquests sistemes tenen dificultats per aconseguir la robustesa i l'adaptabilitat necessàries per al desplegament en entorns de producció dinàmics.

Les xarxes neuronals gràfiques (GNN) han guanyat recentment protagonisme per a tasques de modelatge on les correlacions espacials són essencials. No obstant això, la seva aplicació a l'AD en xarxes informàtiques continua sent poc explorada, malgrat la seva idoneïtat natural per a aquests entorns estructurats. Aquesta tesi investiga nous usos de les GNN per a AD en sistemes en xarxa, centrant-se en tres escenaris representatius: (i) Border Gateway Protocol (BGP) AD, on els segrestos i les configuracions incorrectes causen canvis estructurals; (ii) Contextual Anomaly Detection (CAD), on les anomalies depenen del comportament de les entitats veïnes; i (iii) modelització espaciotemporal de zero-shot, que permet la previsió en entorns amb escassetat de dades.

En el primer escenari, aprofitem la topologia BGP en brut per dissenyar un model capaç de generalitzar entre incidents mantenint una baixa taxa de falsos positius, un requisit essencial per al desplegament al món real. Avaluem el model en incidents BGP coneguts i dades d'operació regular extenses per validar la seva robustesa.

Per a CAD, proposem dos models basats en GNN que descobreixen relacions ocultes entre fluxos en una xarxa troncal. Validem aquests models en conjunts de dades que contenen anomalies reals i sintètiques i mostrem com els mecanismes d'atenció proporcionen visualitzacions interpretables de les dependències contextuais apreses.

Finalment, introduïm un model espaciotemporal basat en GNN entrenat en sèries temporals no relacionades amb la xarxa, capaç d'aconseguir un rendiment de predicció de zero-shot en dades de monitorització de xarxa.

El model explota mecanismes d'atenció per aprendre i transferir correlacions espacials i temporals entre dominis, superant les línies de base més avançades. A més, els patrons d'atenció apresos suggereixen oportunitats per simplificar les representacions espaciotemporals, cosa que és crítica per escalar models a grans xarxes de producció.

Col·lectivament, les contribucions d'aquesta tesi avancen en l'ús de xarxes neuronals gràfiques per a la detecció d'anomalies en xarxes informàtiques mitjançant l'aprofitament sistemàtic de la informació topològica per millorar el rendiment predictiu i la interpretabilitat. A més, també s'explora el paper dels mecanismes d'atenció per proporcionar resultats més interpretables, obrint la porta a utilitzar-los com una manera tant d'entendre les raons darrere de les anomalies detectades com de proporcionar més transparència a la dinàmica subjacent a les xarxes informàtiques.

Contents

Abstract	v
Resumen	vii
Resum	ix
List of Figures	xv
List of Tables	xvii
List of Acronyms	xx
1 Introduction	1
1.1 Motivation and Objectives	3
1.2 Contributions	4
1.3 Outline of the Thesis	6
2 Background	9
2.1 Graph Neural Networks	9
2.1.1 Message Passing Graph Neural Networks	10
2.1.2 Attention-based GNNs	14
2.2 Anomaly Detection	15
2.2.1 Machine Learning applied to Anomaly Detection	15
2.3 The Border Gateway Protocol (BGP)	16
2.3.1 Overview of BGP	16
2.4 Contextual Anomaly Detection	17
2.5 Zero-shot Foundation Models for Forecasting	18
3 Anomaly Detection: Methodological Review	21
3.1 Origins of Anomaly Detection	21
3.2 Classical Methods	22
3.3 Machine Learning Approaches	23
4 Real-time BGP Anomaly Detection Using the Raw Topology	27
4.1 Introduction	27
4.2 Related Work	28
4.2.1 Classical Methods	28
4.2.2 Machine Learning Approaches	29
4.3 Proposed Method	31
4.3.1 Overview	31

4.3.2	Practical Uses of Our Proposed Solution	32
4.3.3	Data collection	32
4.3.4	Data processing	33
4.3.5	Neural network model	34
4.4	Evaluation	38
4.4.1	Dataset	38
4.4.2	Evaluation Metrics	39
4.4.3	Experimental Setup	39
4.4.4	Classification Accuracy	39
4.4.5	Real-time prediction capability	41
4.4.6	Benchmarking Against the State of the Art	41
	Machine Learning-based Methods	41
	Classical methods	44
4.4.7	Testing on regular intervals	44
4.5	Summary of Outcomes	46
5	Interpretable Contextual Anomaly Detection	47
5.1	Introduction	47
5.2	Related Work	48
5.2.1	Classical and Machine Learning Approaches	49
5.2.2	Graph-based Approaches	50
5.3	GAT-AD	51
5.3.1	Data representation	52
5.3.2	Contextual predictor	52
5.3.3	Anomaly detector	54
5.4	GAT-AD+	54
5.4.1	Data representation	55
5.4.2	Contextual predictor	56
5.4.3	Anomaly detector	57
5.5	Evaluation	58
5.5.1	Methodology and Datasets	58
	Baselines	58
	Datasets	59
	Training	59
	Evaluation metrics	60
5.5.2	WaDi: Anomaly detection evaluation	60
5.5.3	WaDi: Overlap analysis	61
5.5.4	Network traffic anomalies: Abilene dataset	63
5.5.5	Analysis of the contextual predictor	69
5.5.6	Sensitivity analysis: window size and temperature	70
5.5.7	Few-shot learning	71
5.6	Summary of Outcomes	72
6	Spatiotemporal Zero-shot Forecasting for Network Monitoring	73
6.1	Introduction	73
6.2	Related Work	75
6.2.1	DL Approaches to Foundation Models	75
6.2.2	Spatiotemporal Foundation Models	76

6.3	Time Series Graph Foundation Model	78
6.3.1	Spatiotemporal Graph Data Representation	78
6.3.2	Underlying spatiotemporal GNN model	79
6.4	Evaluation	81
6.4.1	Experimental Setup	81
	Datasets	81
	Baseline models	82
	Experimental settings	82
6.4.2	TSGFM vs. Baselines	84
6.4.3	Analysis of the Self-attention Mechanism	85
6.4.4	Impact of Spatial Pruning in TSGFM Performance	88
6.5	Summary of Outcomes	90
7	Conclusions and Future Work	91
7.1	Contributions	91
7.2	Future Work	92
	Appendices	97
A	Outcomes of the Thesis	97
A.1	List of Publications	97
A.2	Other Merits	97
	Bibliography	99

List of Figures

2.1	Illustration of Euclidean and non-Euclidean data structures. .	10
2.2	Initialization of the hidden states.	11
2.3	Iteration of the message passing process.	12
2.4	Visualization of the readout function in node-level and graph-level scenarios.	13
2.5	Facebook BGP prefix 129.134.30.0/24 before and during outage	16
4.1	Proposed BGP AD solution.	32
4.2	Example of the graph based on BGP messages.	34
4.3	Overview of our message passing.	34
4.4	Anomaly prediction for the four different permutations. . . .	42
4.5	Graph generation and inference times.	43
4.6	Inference of regular samples.	45
5.1	Overview of GAT-AD.	52
5.2	Overview of GAT-AD+.	55
5.3	Contextual predictor overview.	57
5.4	Anomaly #8, detected only by GAT-AD+.	62
5.5	Anomaly scores generated during anomaly #8.	63
5.6	Recall vs number of marked samples.	65
5.7	Injected Type A anomaly and scores.	66
5.8	Contexts surrounding the injected Type A anomaly.	67
5.9	Injected Type B anomaly and scores.	68
5.10	Contexts surrounding the injected Type B anomaly.	69
6.1	Overview of TSGFM.	78
6.2	TSGFM's spatiotemporal graph representation.	79
6.3	TSGFM predictions and ground truth values for two testing time series in scenario C (TELCO dataset).	83
6.4	Comparison of predicted and actual time series for three samples in the NET-B testing dataset using TSGFM.	86
6.5	Histograms of spatial attention coefficients for the testing datasets in scenarios A, B, and C using TSGFM.	87
6.6	Spatial and temporal attention coefficient distributions generated by TSGFM on the NET-B testing dataset.	89
6.7	CDF of spatial attention coefficients for TSGFM in the NET-B testing dataset.	90

List of Tables

4.1	Description of terms used.	35
4.2	BGP incidents.	38
4.3	Dataset permutations.	40
4.4	Performance metrics.	40
4.5	Comparison of testing performance metrics.	41
4.6	Comparison of testing performance metrics with the ML model from [39].	44
4.7	Comparison of testing performance metrics with a classical technique.	44
5.1	Classification performance by GAT-AD+ and the baseline models.	61
5.2	Classification performance by GAT-AD+ and the baseline models with the new detection criterion.	61
5.3	Detected WaDi anomalies by GAT-AD+ and GDN.	62
5.4	Recall per type at 5% detection threshold.	65
5.5	Classification performance metrics and execution time on the WaDi dataset.	70
5.6	Performance metrics for different window sizes with and without temporal data.	71
5.7	Performance metrics of models with limited training data. . .	71
6.1	Overview of the five publicly available datasets employed to evaluate TSGFM.	80
6.2	Dataset permutations used for training and testing.	82
6.3	Zero-shot forecasting performance on all five dataset permutations.	83
6.4	Dataset permutations used for training and testing (network scenarios).	84
6.5	Testing performance of forecasting models on network dataset permutations.	85
6.6	Testing performance of TSGFM under different spatial pruning levels (NET-B scenario).	89

List of Acronyms

- AD** Anomaly Detection. xv, 1–6, 8, 9, 15, 17, 18, 21–25, 28–32, 38, 44, 45, 47–51, 53, 58–60, 63, 70–72, 75, 91, 94
- AS** Autonomous System. 2–4, 7, 17, 28–30, 33–37, 91
- ASTGCN** Attention-based Spatiotemporal Graph Convolutional Networks. 77
- BGP** Border Gateway Protocol. xv, xvii, 2–4, 6–9, 16–18, 27–34, 38–41, 43, 46, 91
- CAD** Contextual Anomaly Detection. 3–6, 8, 9, 18, 47, 51
- CNN** Convolutional Neural Network. 9, 49
- DDoS** Distributed Denial of Service. 64
- DFT** Discrete Fourier Transform. 77
- DL** Deep Learning. xii, 1, 2, 6, 9, 17, 19, 24, 30, 31, 47, 56, 73, 75
- EWMA** Exponential Weighted Mean Average. 49
- FFT** Fast Fourier Transform. 29
- GAN** Generative Adversarial Network. 24
- GAT** Graph Attention Network. 7, 18, 47, 48, 53, 54, 56, 70, 72, 82, 91, 92
- GCN** Graph Convolutional Network. 3
- GFM** Graph Foundation Model. 19, 77
- GNN** Graph Neural Network. xi, 2–11, 13, 14, 17–19, 25, 27, 30, 31, 46, 47, 49, 50, 52, 53, 56, 58, 69, 70, 73–75, 77, 78, 82, 90, 91
- GRU** Gated Recurrent Unit. 35–37, 76
- ID** Incident Detection. 94
- LSTM** Long Short-Term Memory. 24, 30, 49, 75, 76
- MAE** Mean Absolute Error. 59, 60, 83, 84

- MdRQA** Multidimensional Recurrence Quantification Analysis. 29
- ML** Machine Learning. xvii, 1, 6, 7, 9, 15, 17, 23, 25, 27–30, 38, 40, 41, 44, 46, 49, 50
- MLP** Multilayer Perceptron. 1, 24, 34, 53, 76, 79, 82, 90
- MPNN** Message Passing Neural Network. 10, 13, 14, 69, 79
- MRE** Mean Relative Error. 70
- MTS** Multivariate Time Series. 6, 7, 9, 18, 19, 25, 73, 74, 76, 78, 81, 82, 84, 92
- NN** Neural Network. 9, 15, 24, 53
- OD** Origin-Destination. 7, 18, 47–49, 51–59, 63–68, 70, 72, 92
- PCA** Principal Component Analysis. 22, 29, 49
- RIB** Routing Information Base. 33
- RMSE** Residual Mean Square Error. 84, 90
- RNN** Recurrent Neural Network. 49, 74–76, 82, 90
- RPKI** Resource Public Key Infrastructure. 16
- RQA** Recurrence Quantification Analysis. 29
- STGCN** Spatio-Temporal Graph Convolutional Network. 77
- SVM** Support Vector Machine. 1, 23, 24, 30, 41, 43, 49
- TM** Traffic Matrix. 49, 80
- URLLC** Ultra-Reliable Low Latency Communications. 17
- VAE** Variational Autoencoder. 1, 24, 49

Chapter 1

Introduction

Modern networks are essential to support a wide array of critical applications with increasingly complex and dynamic requirements. From vehicular communications, remote healthcare to public infrastructure and financial systems, these applications demand high levels of reliability and availability. Thus, ensuring the robustness of computer network systems is essential to avoid disruptions and maintain secure and stable operations.

Anomaly Detection (AD) plays a key role in maintaining the stability and performance of these systems. It involves identifying data patterns that deviate from expected behavior, often indicating system faults, misconfigurations, or malicious activity. Anomalies can take diverse forms, such as unexpected spikes in traffic, abnormal sensor readings, or fraudulent access attempts. In cybersecurity, AD is commonly used for intrusion detection [93], while in healthcare it helps identify irregularities in signals like electrocardiograms [151]. Across domains, robust AD is essential for resilience and operational systems.

Traditional AD methods are typically based on statistical rules or hand-crafted thresholds. Techniques such as moving averages or clustering (e.g., K-means clustering) [112, 165] can perform well in controlled environments, but often struggle with high-dimensional, noisy, or non-stationary data. These methods also tend to generalize poorly to novel anomalies.

The emergence of Machine Learning (ML) has driven significant advances in AD, enabling more flexible and data-driven models. Supervised ML methods, such as Multilayer Perceptron (MLP) and Support Vector Machine (SVM), require labeled datasets to learn patterns distinguishing normal and anomalous behavior. However, collecting and labeling such data is costly, especially in complex or dynamic environments [96]. As a result, unsupervised and semi-supervised approaches have become increasingly popular. These techniques aim to model normal behavior from unlabeled data and detect anomalies based on deviations from the learned patterns.

More recently, Deep Learning (DL) methods, including Variational Autoencoders (VAEs) and Transformers, have been applied to AD tasks [9, 160, 161]. These models excel at extracting features from high-dimensional data

and have achieved strong results in fields such as healthcare and cybersecurity. Nonetheless, most DL architectures assume that data samples are independent, which limits their ability to capture structural or relational dependencies between entities.

In many real-world systems –especially in computer networks– data points are not isolated. Instead, they are inherently interconnected through topologies that govern interactions between entities. For example, the routing infrastructure of the Internet, defined by the Border Gateway Protocol (BGP), forms a graph in which Autonomous Systems (ASes) exchange routing information over dynamically evolving paths. Anomalies in such systems often manifest through structural changes, such as hijacks, which are harder to detect by models that ignore the network topology. Similarly, in traffic monitoring, the behavior of a single node or flow is often only meaningful when considered in the context of its neighbors.

Graph Neural Networks (GNNs) have emerged as a powerful class of models for learning from graph-structured data [130, 156, 178]. Unlike traditional DL architectures, GNNs are designed to leverage both node attributes and the relational structure encoded in the graph. This makes them particularly well-suited for scenarios where context and connectivity are essential. GNNs have achieved notable success in fields such as chemistry [53], healthcare [115], computer vision [138], and recommendation systems [45]. However, their application to AD in network traffic remains relatively underexplored.

Several early studies demonstrated the potential of GNNs for AD in structured domains. For instance, [36, 105] leveraged GNNs to detect anomalous nodes in attributed graphs by identifying inconsistencies in structure or features. On the other hand, [32] modeled dependencies in sensor systems, revealing how the structural context can be used to detect anomalies in correlated nodes. Other approaches, such as [41], applied GNNs to fraud detection, where anomalies often hide in communities.

Although these methods spanned a variety of domains, their success highlighted the value of incorporating the graph topology into the detection process. By modeling entities with their relational context, GNNs uncover hidden irregularities that would have been difficult to identify with traditional models. Despite these successes, there have been relatively few works investigating GNNs for AD [155]. To address this research gap, this thesis investigates whether GNNs can provide effective solutions for AD in computer networks, where the structure and spatial correlation between nodes play a central role.

In this context, we identify three distinct types of data commonly found in network environments, and for each type, we select a representative case study:

- **Protocol data:** We focus on BGP AD, where anomalies such as hijacks and misconfigurations affect the dynamic graph of ASes, and thus can

be detected through topological changes.

- **Network traffic data:** We address Contextual Anomaly Detection (CAD), in which the anomalous status of a network entity depends on its context, defined by its relationships and interactions with neighboring nodes.
- **Network monitoring data:** We develop a foundation model for network monitoring forecasting, leveraging GNNs to enable general-purpose forecasting over monitoring data, which can serve as a basis for subsequent AD tasks.

Through these case studies, we aim to assess the value of topology-aware models for detecting anomalies in network data and to explore their potential as generalizable, robust solutions in complex and evolving environments.

The following section outlines the specific motivations that guide this research.

1.1 Motivation and Objectives

The use of GNNs for AD stems from their capability to learn complex relational patterns that go beyond individual node features. In structured domains, anomalies often arise not because the behavior of a single entity is deviating from the expected, but rather because its behavior diverges from the patterns exhibited by its neighbors or even the topology as a whole. GNNs address this by propagating information through the graph, allowing the representation of each node to reflect both its local and global contexts. This topology-aware modeling is particularly valuable in domains where anomalies may be subtle or context-dependent.

Several early studies showed that GNNs could effectively capture such correlations [36, 105]. In the context of computer network data, however, these approaches had only begun to emerge. [177] applied a Graph Convolutional Network (GCN) model to detect botnets in peer-to-peer networks. More recently, [106] introduced a GNN that incorporated edge attributes into the aggregation process to improve intrusion detection. While these approaches demonstrated promising results, many use cases in computer network environments were still left to be researched.

This thesis is motivated by the opportunity to address these gaps by exploring the use of GNNs in more diverse and realistic computer network scenarios. To do so, we consider three representative tasks in which graph-based modeling is naturally applicable and brings significant potential advantages.

In the first case study, we address AD in the BGP. BGP defines a dynamic graph of inter-AS routing relationships, where anomalies such as hijacks propagate as structural changes in the topology. Our objective is to develop a model that directly leverages the BGP topology to detect anomalies, minimizing the reliance on manually engineered features or incident-specific

heuristics. The model aims to generalize across different anomalies without requiring fine-tuning, providing robust and a low rate of false positives in real-world deployments.

In the second case study, we consider CAD, where the anomalous nature of an event depends not just on its local features but on its surrounding context. Here, our objective is to build a GNN-based model capable of learning and exposing the contextual relationships that define normal behavior, without predefined labels or correlation rules. We wish the exposed contexts to be interpretable to offer insights into the contextual structure of the data and to help network operators understand why certain nodes deviate from the expected patterns.

Lastly, the third case study addresses a broader and recurring challenge in the field: the limited availability of publicly available and labeled network traffic data. To overcome this constraint, we explore the use of foundation models trained on non-networking data to perform forecasting on networking scenarios. The end goal is not to perform the forecasting, but rather to use it as a base to design AD systems based on it. To achieve the best forecasting performance, we design a GNN-based model that learns spatiotemporal correlations from non-network time series and then generalizes to network data. By achieving accurate forecasts on network monitoring traffic, these could provide a reliable baseline for identifying anomalies as deviations from this expected behavior.

Across all three case studies, the central motivation is to exploit the correlations between nodes defined by the network topology, to reduce the dependence on manual feature engineering and enhance the model robustness, interpretability and generalization. By designing solutions that rely on the graph structure and contextual reasoning, this thesis seeks to advance the development of scalable and trustworthy AD systems for complex network environments.

1.2 Contributions

The first major contribution of this dissertation is the development of a novel approach for BGP AD based on GNNs that operates directly on the raw AS-level topology graph. We introduce a GNN-based model specifically designed to process graph-structured data, thereby eliminating the need for handcrafted features or manual feature engineering. Unlike many state-of-the-art approaches that require fine-tuning hyperparameters or retraining to handle different anomalies, our model uses a fixed architecture and a single set of hyperparameters across all evaluation scenarios. This design significantly improves the model's robustness and suitability for real-world deployments, where retraining is often impractical. We evaluate the model using four well-known BGP incidents and a hundred randomly sampled regular intervals. Results show that our approach outperforms state-of-the-art baselines in identifying known incidents while maintaining a near-zero false

positive rate in regular, non-anomalous periods. This type of long-term evaluation –rarely addressed in previous works– demonstrates the viability of our model for real-time deployment in production environments.

These results have been published in a workshop paper (*Unveiling the potential of graph neural networks for BGP anomaly detection*, Graph Neural Networking Workshop 2022) and are further expanded in a journal article currently under minor revision (*BGP anomaly detection using the raw internet topology*, Computer Networks, under minor revision, 2025). Full publication details are provided in Appendix A.1.

Building on these insights, the second contribution of this dissertation is a GNN-based framework for CAD. Contextual anomalies are defined by their dependence not only on an entity’s features but also on its surrounding environment. Yet, most existing methods fail to explicitly model this relational context. Our proposed approach addresses this gap by modeling each node’s behavior in relation to that of its neighbors, making it specifically suited for detecting context-dependent anomalies. In particular, our model is designed not as a replacement for traditional AD methods, but rather as a complementary tool that enhances their effectiveness in scenarios where contextual correlations are crucial. By forecasting a node’s behavior using only the attributes of its neighboring nodes –completely excluding its own– we amplify deviations that result from contextual inconsistency. This design choice amplifies contextual anomalies, many of which would remain unnoticed using conventional approaches. Experimental results demonstrate that our model successfully identifies a greater proportion of contextual anomalies than several baselines. Moreover, it achieves superior performance on a widely used benchmark dataset for AD that contains contextual anomalies, confirming its effectiveness in realistic CAD scenarios.

This line of research resulted in a workshop publication (*Detecting contextual network anomalies with graph neural networks*, Graph Neural Networking Workshop 2023) and a journal article (*GAT-AD: Graph Attention Networks for contextual anomaly detection in network monitoring*, Computers & Industrial Engineering, JCR Q1). See Appendix A.1 for complete references.

During the course of the thesis, we identified a significant limitation in the availability of network-related datasets. Due to privacy constraints and commercial sensitivities, organizations are often reluctant to release internal monitoring data, resulting in a scarcity of labeled high-quality datasets for training and evaluating AD models. This limitation adds a barrier to the development and validation of robust and generalizable solutions.

To address this challenge, the third contribution of this thesis lays the groundwork for unsupervised and zero-shot learning strategies using GNN-based models. These models are trained on data from a range of unrelated domains and are then evaluated on previously unseen network monitoring data. In this context, we introduce TSGFM, a time series graph-based

foundation model based on spatiotemporal GNNs for Multivariate Time Series (MTS) forecasting. TSGFM is designed to generalize across heterogeneous time series datasets by modeling both spatial dependencies among time series and their internal temporal dynamics through an attention-based graph structure. Through an extensive evaluation across five publicly available datasets from multiple domains, we show that our proposal consistently outperforms several state-of-the-art models, confirming its potential as a general-purpose foundation model for network time series analysis.

The outcomes of this work have been submitted as a workshop paper (*TS-GFM – Towards a Graph Foundation Model for Time Series Analysis in Network Monitoring*, under review, 2025). Full bibliographic details are provided in Appendix A.1

1.3 Outline of the Thesis

The remainder of this thesis is structured into six more chapters, each covering a key component of the research work:

Chapter 2 – Background

This chapter introduces the foundational concepts necessary to understand the research presented in this thesis. It begins with an overview of GNNs, describing their architecture and the message passing mechanism that enables nodes to iteratively exchange information and refine their representations. It then explores attention-based GNNs, which enhance message passing by allowing nodes to dynamically weigh information from their neighbors. Next, the chapter introduces the problem of AD, outlining its key challenges and the role of ML-based methods in addressing them. Finally, it presents the three use cases explored in this thesis: AD in BGP networks using raw topological data, CAD in traffic time series with graph-based models, and zero-shot forecasting using foundation GNNs for generalization across network environments. These topics establish the groundwork for the contributions presented in the following chapters.

Chapter 3 – Anomaly Detection: Methodological Review

This chapter provides an overview of the evolution of AD, tracing its development from early statistical techniques rooted in hypothesis testing and distribution modeling. It then explores classical approaches such as distance-based, density-based, clustering-based methods, and traces the evolution towards more complex ML models that enabled a more flexible and data-driven detection process. The chapter also covers the rise of DL techniques along with recent advances using Transformers and GNNs to detect contextual and collective anomalies in complex and structured data.

Chapter 4 – BGP Anomaly Detection

This chapter introduces a novel GNN-based architecture for detecting anomalies in BGP networks by leveraging the raw AS-level topology derived from BGP announcements. Unlike traditional approaches that rely on manual feature selection, this method feeds the complete BGP topology into the GNN, enabling it to learn and extract relevant features autonomously. The model is evaluated on four well-known datasets using leave-one-out permutations, training on three, and testing on the remaining one to study the generalization across these real-world scenarios. The results prove that the proposed model not only achieves high detection accuracy but also outperforms both classical and state-of-the-art ML-based baselines. Furthermore, the chapter validates the robustness of the proposed model in regular conditions and its suitability for real-time deployment, highlighting its potential for use in operational environments.

Chapter 5 – Contextual Anomaly Detection

This chapter introduces GAT-AD, a first iteration of a GNN-based framework designed for the detection of contextual anomalies in network traffic data, and GAT-AD+, an evolution of the first iteration. In GAT-AD+, contextual relationships between Origin-Destination (OD) flows are learned through a custom neural network model based on Graph Attention Network (GAT), enabling subtle contextual anomalies to be amplified (i.e., deviations that appear only when a flow is compared to others with similar historical behavior). Through this mechanism, a behavior that may seem normal in isolation is revealed as anomalous within its context. The two versions of the model are evaluated extensively on the Abilene network, where synthetic anomalies are injected, and on the WaDi dataset, which includes labeled incidents. In both cases, GAT-AD+ achieves superior recall and F1-scores when compared against GAT-AD and a state-of-the-art baseline. Thanks to its interpretable attention-based design, GAT-AD+ not only enables accurate detection but also reveals the underlying contextual relationships driving those decisions.

Chapter 6 – Foundation Models for Forecasting

This chapter introduces TSGFM, a novel GNN-based foundation model designed for zero-shot MTS forecasting. Foundation models pretrained on diverse and heterogeneous datasets are leveraged to address the challenges of forecasting in dynamic data and data-scarce domains such as network monitoring. TSGFM models spatial and temporal correlations through a unified attention-based graph representation, where each time series is represented as a node and connected via spatial and temporal edges in a fully connected spatiotemporal graph. Extensive evaluation is carried out across five datasets from different domains. TSGFM consistently achieves strong performance in zero-shot settings, outperforming a wide range of baselines. Further evaluations demonstrate that TSGFM successfully transfers knowledge from non-network datasets to forecasting tasks in the networking domain. Attention analyses reveal that only a small subset of spatial and temporal

edges significantly contribute to the forecasting, highlighting the potential for reducing the model complexity while keeping the obtained performance.

Chapter 7 – Conclusions and Future Work

This chapter provides a comprehensive summary of the main contributions of this thesis, which explores the use of graph-based models to address key challenges in cybersecurity and network monitoring. Across three complementary use cases (BGP AD, CAD in network traffic, and spatiotemporal forecasting using foundation models) we demonstrate how GNNs and attention mechanisms can be leveraged to uncover latent structures and improve prediction accuracy. The chapter also outlines promising directions for future research.

Chapter 2

Background

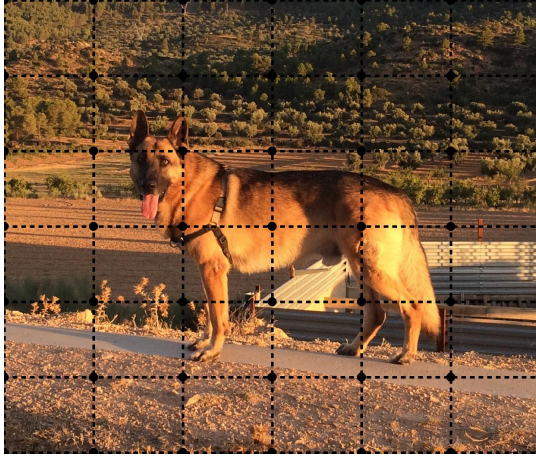
In this thesis, we adopt a GNN-based approach to tackle complex AD and forecasting tasks in network scenarios. GNN are a subclass of DL models specifically designed to operate on graph-structured data, where information is inherently relational and defined by interactions between the elements in a graph. Their efficiency in dealing with these kind of data allow them to provide near real-time predictions, making them not only accurate but also computationally practical for production environments. The aforementioned attributes make GNNs effective in network scenarios, where the underlying structure can significantly influence the behavior of individual elements.

Throughout this thesis, GNNs serve as the core modeling framework for both detecting anomalies in topological data (e.g., BGP networks) and forecasting MTS in network monitoring systems. This chapter provides a theoretical background in GNNs, as well as attention-based GNN. Then, an overview of our three use-cases (BGP AD, CAD and zero-shot foundation models) is also provided.

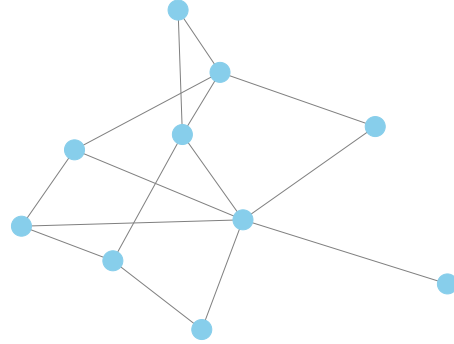
2.1 Graph Neural Networks

GNNs [130] are a specialized type of ML model explicitly designed to work with graph-based data, where entities are represented as nodes and the relationships between these entities are represented as edges connecting the nodes. One of the primary strengths of GNNs is their capability of effectively modeling graphs regardless of their size. This flexibility arises from their ability to leverage and extract meaningful knowledge directly from the underlying graph topology, capturing structural patterns that remain consistent across varying graph scales.

Unlike conventional Neural Network (NN) architectures such as Convolutional Neural Networks (CNNs), which require structured, Euclidean input data (e.g., images or grid-like datasets, as illustrated in Figure 2.1a), GNNs are inherently capable of processing non-Euclidean and irregularly-structured data, such as graphs (Figure 2.1b). CNNs exploit spatial locality by applying convolutional filters over fixed-size grids, an approach unsuitable for graphs due to their arbitrary connectivity patterns and the absence of uniform neighborhood structures. In contrast, GNNs



(A) Data organized in Euclidean space.



(B) Data represented in non-Euclidean space.

FIGURE 2.1: Illustration of Euclidean and non-Euclidean data structures.

operate directly on relational data, learning representations from both local and global patterns in the graph topology. These capabilities have led to their widespread adoption in diverse domains, including chemistry [121], recommender systems [169], computer network modeling [125, 137], and social networks [110, 98].

2.1.1 Message Passing Graph Neural Networks

Message Passing Neural Networks (MPNNs) [53] represent the most generic framework for GNNs. At the core of this framework is the intuitive idea of iteratively sharing and exchanging information among neighboring nodes in a graph, progressively refining node representations through these interactions. Such iterative refinement allows node embeddings to capture not only local node characteristics but also the broader structural context from the surrounding neighboring nodes.

Formally, the message passing paradigm can be decomposed into three fundamental phases: initialization, message passing and readout. In the initialization phase, each node i in the graph is assigned an initial hidden state h_i , which typically encodes the node attributes. These initial hidden states are represented as vectors of fixed dimensionality.

The core of the framework lies in the iterative message passing phase, during which the nodes exchange information among themselves during multiple iterations. In each iteration, every node i generates a message m_i based on its current hidden state h_i , and sends it to its neighboring nodes. Subsequently, each node aggregates the incoming messages from its neighbors using a permutation invariant operation, which ensures consistency and generalization in the processing of the data (as it guarantees that, independently of the order in which the messages are received, the result of the aggregation is the same). The aggregated messages are used to update the hidden state h_i ,

thus enriching the representation of the node i with information of its neighbors. Finally, the readout phase transforms the final node representations into predictions.

Initialization Phase In the initialization phase of a message passing GNN, each node in the graph is assigned an initial hidden state that encodes its available information prior to any interaction with its neighbors. Formally, for a given graph $G = (V, E)$, where V denotes the set of nodes and E represents the set of edges between pairs of nodes, each node $i \in V$ is associated with input features x_i . These input features, which may represent intrinsic node attributes or structural information, are embedded into a shared hidden representation space through a learnable initialization function E . This transformation yields the initial hidden state for node i , denoted as $h_i^{(0)}$:

$$h_i^{(0)} = E(x_i) \quad (2.1)$$

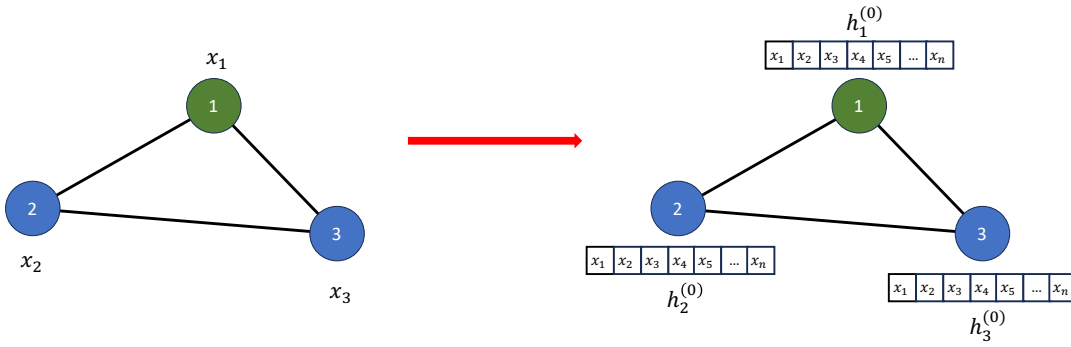


FIGURE 2.2: Initialization of the hidden states.

The purpose of this mapping is to project all node features into a common vector space of fixed dimensionality n , ensuring that nodes, regardless of their original feature complexity, start the message passing process with compatible representations. The initialization phase is represented in Figure 2.2.

Message Passing Phase Once the initial states have been assigned, the GNN begins the message passing phase. Over T iterations, the nodes in the graph exchange information with their neighbors to enrich their representations. The information sharing process can be divided into three different phases: (i) message generation, (ii) message sharing and aggregation, and (iii) representation update. Figure 2.3 illustrates these steps through an example centered on node 1 (highlighted in green).

At each iteration t , every node i generates messages to be sent to its neighbors, $j \in \mathcal{N}(i)$ based on its current state h_i , the hidden state of the neighbor h_j , and potentially any attributes associated with the edge e_{ij} , connecting the nodes i and j . The function M is used for this message generation:

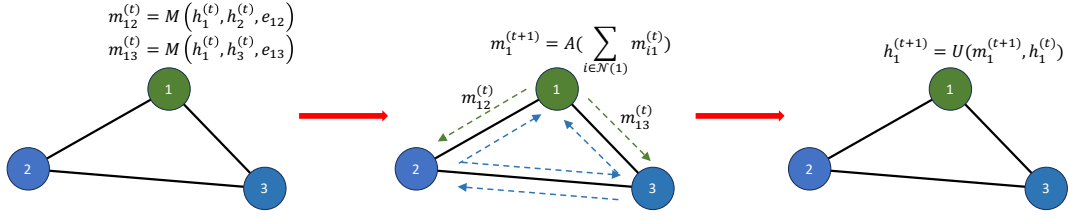


FIGURE 2.3: Iteration of the message passing process.

$$m_{ij}^{(t)} = M(h_i^{(t)}, h_j^{(t)}, e_{ij}) \quad (2.2)$$

Then, each node collects the message sent by its neighbors and aggregates them with an aggregation function A . This function combines the set of incoming messages into a single aggregated message:

$$m_i^{(t+1)} = A(m_{ij}^{(t)}, j \in \mathcal{N}(i)) \quad (2.3)$$

Importantly, this function has to be permutation invariant to the order of the incoming messages, which ensures consistent outputs regardless of the order in which messages arrive. Common choices include the summation (as in the example of Figure 2.3), mean or more sophisticated functions like trainable aggregators.

To finish the iteration t , each node updates its hidden state with the aggregated message, m_i^{t+1} . This update is performed through a learnable function U :

$$h_i^{(t+1)} = U(m_i^{(t+1)}, h_i^{(t)}) \quad (2.4)$$

This update integrates the aggregated neighborhood information and allows to iteratively refine the node's representation, accumulating characteristics from increasingly distant parts of the graph to capture both local and global structural patterns.

After the T iterations of the message passing are completed, the nodes reach convergence, and the information contained by each of them is ready to be used in the readout phase.

Readout Phase After completing the T iterations of message passing, the nodes' hidden states contain both their node features and contextual information of their multi-hop neighborhoods. In this final step, known as the readout phase, these enriched node representations are converted into final predictions suitable for specific tasks.

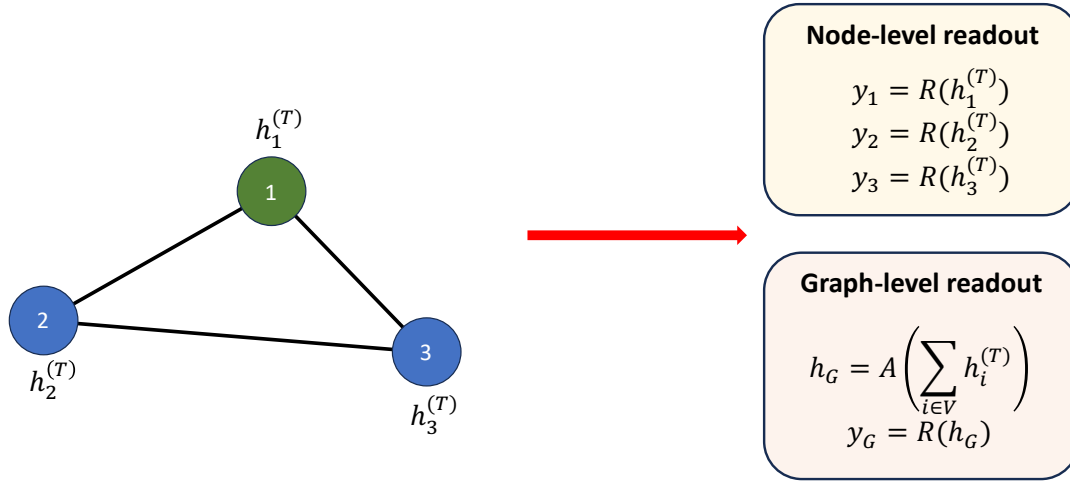


FIGURE 2.4: Visualization of the readout function in node-level and graph-level scenarios.

The design of the readout function R depends on the target application and can be either node-specific or graph-wide (Figure 2.4). For node-level tasks, such as node classification, the readout function operates individually on each node's hidden state:

$$y_i = R(h_i^{(T)}) \quad (2.5)$$

In contrast, in graph-level tasks such as graph classification, the readout function aggregates the hidden states of all nodes into a unified graph representation, which is then used for prediction. A straightforward example is given by summation aggregation:

$$y_G = R\left(\sum_{i \in G} h_i^{(T)}\right) \quad (2.6)$$

It is important to note that the aggregation method used to generate the graph-level representation should share the same permutation-invariant property required in the message passing phase. While summation aggregation is common due to its simplicity, other permutation-invariant functions such as mean, max pooling, or attention-based mechanisms can also be employed, depending on the complexity and requirements of the task.

The flexibility of the readout phase allows MPNNs (and other types of GNNs) to adapt effectively across various domains and problem settings, making them a versatile class of models for structured data [63, 87, 162].

2.1.2 Attention-based GNNs

Despite the success of standard MPNNs, their performance often depends on the chosen aggregation function [162]. Simple aggregation methods, such as summation or max pooling, treat neighbors equally or in a fixed manner, without considering their relative correlation with respect to a given node. To address this limitation, attention-based mechanisms have been introduced [150], enabling adaptive weighting of neighboring nodes according to their relevance.

Attention-based GNNs extend the message passing framework described in Section 2.1.1, by keeping the message generation and representation update steps largely unchanged, but enriching the aggregation phase with a learnable attention mechanism.

Specifically, during each iteration t , nodes generate messages to be sent to their neighbors using the same message function M defined in Equation (2.2). However, in contrast to the fixed aggregation function A in Equation (2.3), attention-based GNNs compute attention coefficients $\alpha_{ij}^{(t)}$, which determine the relative weight of each message. These coefficients are computed using a scoring function S , which evaluates the relevance of messages from a node j to a node i based on their current hidden states and, potentially, their connecting edge attributes:

$$\alpha_{ij}^{(t)} = \frac{\exp \left(S(h_i^{(t)}, h_j^{(t)}, e_{ij}) \right)}{\sum_{k \in \mathcal{N}(i)} \exp \left(S(h_i^{(t)}, h_k^{(t)}, e_{ik}) \right)} \quad (2.7)$$

Using these attention coefficients, the aggregation step becomes a weighted sum of the incoming messages, modifying Equation (2.3) to:

$$m_i^{(t+1)} = A(\alpha_{ij}^{(t)} \cdot m_{ij}^{(t)}, j \in \mathcal{N}(i)) \quad (2.8)$$

Finally, the representation update step continues to use the same function U as in Equation 2.4, applying the aggregated message to refine the node's hidden state.

By dynamically adjusting the importance of the incoming messages, attention-based GNNs offer several advantages over traditional message passing GNNs: (i) they allow to better capture complex relationships between nodes in the graph, (ii) they allow better interpretability, since the attention coefficients provide transparency regarding the decisions taken by the model, and (iii) they improve the obtained performance by filtering out irrelevant information.

Attention-based GNNs have demonstrated remarkable effectiveness across a wide range of tasks [143, 153, 35]. Their ability to dynamically

prioritize relevant node correlations makes them particularly well-suited for real-world scenarios.

2.2 Anomaly Detection

Anomalies, also known as outliers, are data samples that deviate significantly from expected patterns (or normal behavior) [24]. Their origins are diverse, ranging from malicious activities (e.g., cyberattacks or fraudulent actions) to benign causes like system misconfigurations or operational errors.

A fundamental challenge in AD lies, precisely, in defining what constitutes *normal* behavior. This distinction is often difficult to establish for several reasons: (i) normal and anomalous instances may share overlapping characteristics, making them hard to differentiate, (ii) the definition of normality is not static, as systems often evolve over time and behaviors previously considered anomalous may later become typical, and (iii) the presence of noise in the data can further blur the boundary between normal and anomalous samples, reducing the reliability of simplistic AD methods.

These factors illustrate the inherent complexity of the AD process, where the subtle and dynamic nature of anomalies requires robust, adaptable, and context-aware detection methodologies.

2.2.1 Machine Learning applied to Anomaly Detection

Even though statistical methods have been widely used for AD [123], recently, ML-based techniques have gained traction [113]. Broadly, ML-based approaches can be characterized into three different types, depending on the availability of labeled data and how they are trained: (i) supervised methods, which rely on labeled datasets containing both normal and anomalous samples, (ii) semi-supervised methods, which assume access primarily to labeled normal data, learning the behavior of normal samples and detecting deviations from it as anomalies, and (iii) unsupervised methods, which operate without any labeled data and typically rely on the assumption that the rate of anomalies in the data is low, allowing their detection as significant deviations in the intrinsic data patterns.

ML-based approaches have demonstrated strong capabilities in capturing complex, high-dimensional patterns that traditional statistical methods may fail to detect effectively. In particular, NN-based methods have gained attention due to their scalability and flexibility in learning complex feature representations directly from raw data [31].

2.3 The Border Gateway Protocol (BGP)

2.3.1 Overview of BGP

The BGP (RFC 4271) is the standard protocol to propagate routes on the Internet. Due to its ubiquity and relevance, it is often the victim of attacks and misconfigurations that cause service disruption and connectivity issues. BGP incidents are common, with a monthly average of 300 worldwide, ranging from small misconfigurations to large BGP table leaks [109, 1].

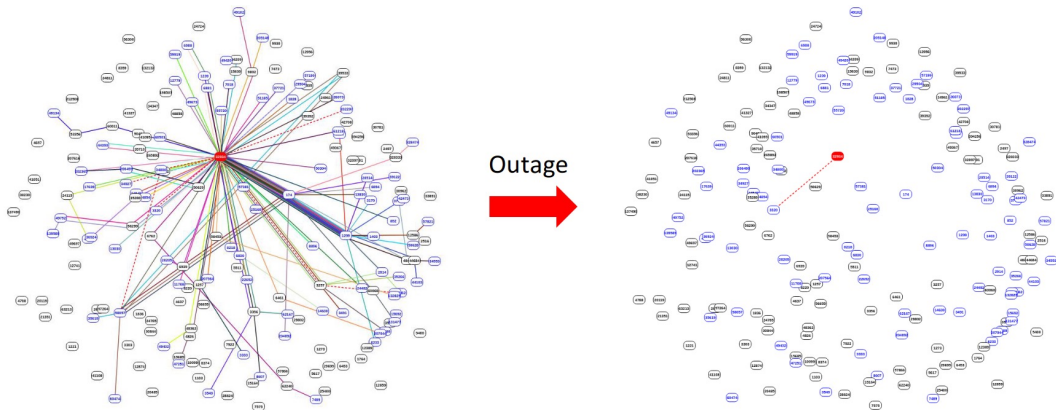


FIGURE 2.5: State of the BGP network for the prefix 129.134.30.0/24 before the Facebook outage (left) and during the outage (right).

Such accidents can have a significant economic and societal impact. For example, in 2021, a BGP misconfiguration took down all Facebook applications for several hours. Figure 2.5 shows a representation of the state of Facebook’s connections to its BGP peers before the outage (left) and during the outage (right) ¹. We can see the loss of connectivity of Facebook (red node) during the outage, as well as the impact on the structure of the BGP topology graph.

Since BGP was not designed with security in mind [111], it provides limited security safeguards against unintentional misconfigurations or targeted prefix hijacks. Several solutions are being deployed to mitigate these incidents. Two of the most relevant are the Resource Public Key Infrastructure (RPKI), a repository of digital certificates to validate BGP messages (RFC 6480 [94]), and BGPsec (a set of security extensions for BGP messages, RFC 8205 [95]), both from the IETF.

However, the deployment of the RPKI is slower than expected, with only 35% of /24 IPv4 prefixes covered by the RPKI [140], and there are serious concerns about the feasibility of deploying BGPsec due to its cryptographic overhead [79]. Even if the RPKI was fully deployed, it does not offer protection against several critical incidents, such as path hijacks, route leaks, and misconfigurations [52]. Hence, network operators have relied on a variety of

¹Source of the images: <https://stat.ripe.net/widget/bgplay>

tools to detect and mitigate the effects of these anomalies. These techniques leverage a wide range of solutions, such as different kinds of statistical analysis or exploiting BGP historical data, among others [5].

Furthermore, techniques based on ML have been recently proposed to detect BGP anomalies [28, 26, 83, 100]. Most of these solutions extract metrics from the BGP announcements (e.g., AS_PATH, number of announcements, etc.) and feed them to the ML algorithm. Usually, these algorithms require a feature selection process to reduce the dimensionality of the input data [20, 10, 7] and keep the relevant features. However, feature selection is manual and subjective [28, 86], since it is not possible to know beforehand which features will correlate with an anomaly. Hence, it can happen that a detection algorithm does not use some critical features to detect an anomaly, or that it performs well for certain anomalies but does require fine-tuning for others.

At its root, the BGP announcements generate an Internet topology of the different interconnected ASes. Interestingly, previous research [26] has not focused on exploiting the raw topology to detect anomalies. Instead they have focused on using manually curated features of the topology or the BGP messages themselves.

For instance, in Figure 2.5, we can see a representation of the state of the BGP connectivity before an outage (left) and during it (right), which highlights the relevance of the topology. As discussed earlier, previous research has used features such as the *graph centrality* as opposed as the entire graph.

Existing BGP AD detection methods often overlook the rich structural information embedded in the Internet’s AS-level topology. Instead of focusing on selected graph features, we argue that a better option is to process the entire BGP graph directly.

Recent advances in DL, especially GNNs, have enabled the modeling of complex graph-structured data. GNNs are particularly suited to this problem, as they can learn representations directly from the evolving AS topology without requiring prior feature engineering. This motivates the use of GNNs for BGP AD, a direction we pursue in Chapter 4, where we introduce a novel GNN-based approach that detects anomalies directly from the raw AS-level topology.

2.4 Contextual Anomaly Detection

Today’s networks carry very diverse workloads from applications with heterogeneous requirements. Among these, availability and reliability are becoming increasingly important in networks, especially to support critical applications such as vehicular networks or remote surgery, which require Ultra-Reliable Low Latency Communications (URLLC) [51]. In this vein, network AD is a crucial task for network operators to promptly detect potential issues in the network, identify the root cause, and fix the problem before it impacts the customers’ Quality of Experience (QoE). Network

anomalies reflect deviations from the typical network behavior, which often help identify configuration errors (e.g., BGP leaks), malicious activity (e.g., denial-of-service attacks) or equipment malfunctions that otherwise would be difficult to detect before they have a severe impact on the network [80, 85, 47].

Furthermore, the effects that these anomalies may cause in the network can vary widely. The authors of [12] propose the following network anomalies taxonomy:

- **Point anomalies:** Individual samples that are anomalous with respect to the rest of the data. For instance, a sudden increase in the traffic measurements of an OD flow — i.e., a network traffic flow that carries data from a specific origin to a given destination.
- **Contextual anomalies:** Samples that are anomalous with respect to a specific context. For instance, an OD flow that keeps stable traffic measurements while its context's OD flows measurements increase.
- **Collective anomalies:** A collection of samples that are anomalous with respect to the entire dataset. While each individual data point may appear normal in isolation, their combined pattern deviates significantly from expected norms. For example, moderate congestion levels at several network nodes might seem normal independently, but when occurring simultaneously in a specific pattern, they may indicate a larger systemic issue.

AD relies on data collected by network operators, which are often high-dimensional and noisy, adding complexity to the modeling process [3]. Identifying anomalies typically requires first modeling the expected normal behavior, a task made more difficult by the potential presence of undetected anomalies within the training data [24]. In network monitoring systems, traffic data is frequently gathered as a series of traffic matrices aggregated at the OD flow level. Since these traffic measurements are collected continuously, OD flow data is generally represented as time series, allowing to analyze the temporal evolution of traffic. Traffic anomalies are usually characterized by temporal variations that significantly diverge from the expected normal behavior [72].

In the context of CAD, GNNs are well-suited to capture the dependencies between OD flows by learning flow-specific contexts in a data-driven manner. This motivates our use of a graph-based model for detecting contextual anomalies, which we present in Chapter 5, introducing a GAT-based architecture that builds interpretable contexts from raw traffic measurements.

2.5 Zero-shot Foundation Models for Forecasting

Network monitoring is essential for ensuring the functionality and performance of computer networks [42, 146, 3]. MTS analysis plays a key role in this domain, enabling a wide range of applications such as network planning, predictive resource allocation, anomaly detection, troubleshooting, and

more. A central challenge in network monitoring is forecasting network behavior, a complex task given the dynamic and heterogeneous nature of network traffic. Accurately predicting future behavior requires models that can manage high-dimensional data, capture intricate interdependencies between time series, and effectively learn both spatial and temporal correlations.

Traditional forecasting models for MTS data often struggle to generalize across diverse network environments, requiring significant retraining and domain-specific adaptations. This makes MTS forecasting particularly challenging, as it requires models capable of capturing complex temporal patterns while maintaining computational efficiency. While traditional statistical models for time series forecasting have been widely explored, they often fall short when applied to the non-stationary, non-linear, and noisy nature of network monitoring data, resulting in suboptimal predictions. As a result, modern DL approaches have gained traction in recent years due to their ability to handle complex dependencies and generate more accurate, realistic forecasts [102]. Still, most models in the literature frequently require dataset-specific training, which is both resource-intensive and time-consuming.

Zero-shot learning offers a viable solution to generalization in DL models, allowing models to generalize to new tasks or datasets without requiring additional training or fine-tuning. Instead of being explicitly trained for each specific scenario, these models leverage knowledge learned from large-scale pretraining on diverse datasets to make inferences on previously unseen data. This capability is particularly valuable for MTS forecasting, especially in complex and data-limited domains like network monitoring, where obtaining labeled training data can be challenging. By harnessing the comprehensive pattern extrapolation capabilities of extensively pre-trained models, it circumvents the substantial time, effort, and domain-specific expertise typically demanded for crafting dedicated time-series models.

In recent years, foundation models have emerged as an effective means to design zero-shot learning systems. Foundation models provide a powerful framework for zero-shot learning by serving as generalizable representations across multiple tasks and domains [13]. Their success stems from extensive pretraining on diverse data, which enables them to adapt to new tasks under zero-shot or few-shot conditions. This makes them particularly well-suited for real-world applications in network monitoring, where retraining on domain-specific data is often infeasible due to operational constraints and data privacy concerns. In practice, foundation models are typically designed for specific data types, such as language [34], videos/images [40], or tabular data [149]. Graph Foundation Models (GFM) [104] aim to leverage extensive graph data to learn representations that are transferable to a wide range of domains, reducing the need for task-specific training.

Forecasting network behavior is particularly difficult because obtaining datasets that capture the complexity of real-world environments is often infeasible. Motivated by this, in Chapter 6 we explore how spatiotemporal GNNs can enable generalization across different domains without

requiring labeled data from the target environment. Building on the ideas of leveraging topological and contextual information introduced in Chapter 5, we introduce a foundation model approach designed to operate under zero-shot conditions, directly addressing the challenges posed by limited data availability.

Chapter 3

Anomaly Detection: Methodological Review

AD plays a critical role in maintaining the security, stability, and reliability of complex systems. As discussed in the previous chapter, the inherently dynamic and noisy nature of real-world environments presents significant challenges for accurate anomaly identification.

This chapter presents a methodological overview of AD, from its inception until the most modern proposals, to allow the reader to get a grasp of what has been done and why the current proposals are the way they are.

3.1 Origins of Anomaly Detection

AD, also referred to as outlier detection, has its origins in the statistics community. Specifically, one of the first references to the term takes place in the year 1887 [43, 24].

[38] marks a foundational contribution to AD by formalizing statistical methods for identifying outliers and clearly distinguishing two core challenges: detecting anomalous observations and developing robust statistical estimators that remain accurate in the presence of such contamination.

A canonical work was presented in 1969, when [57] introduced a hypothesis test to detect outlying observations, defined as an observation that deviates significantly from the rest of the data. These tests assume an underlying distribution (for instance, a normal distribution) and constituted AD as a statistical problem, where the objective of the task was to decide whether the sample was an anomaly or if it was a highly improbable, but possible observation.

The distribution assumptions, though, were an important issue with these models. It was observed that statistical methods were especially sensitive to deviations between the expected and true distributions of the data, which could greatly reduce their performance [77].

These initial proposals and the issues observed motivated a deeper research into newer solutions that could provide better solutions while not suffering from these.

3.2 Classical Methods

Initially, classical AD methods relied on statistical analysis. The core idea of these methods is to model a distribution that is assumed to generate regular data, and label as anomalous any sample that does not comply with this expected model. A clear example of this idea is the following: assuming that data follow a Gaussian distribution, a sample can be considered anomalous if its value exceeds a specific threshold set at a certain number of standard deviations from the mean [57].

A different statistical approach is Bayesian AD. Bayesian statistics combines prior beliefs or knowledge with new evidence from data. Instead of assuming nothing is known beforehand, Bayesian methods allow to start with an informed guess (i.e., the prior) and update it iteratively as more information is gathered. [15] presents one of the first works in this regard by introducing a Bayesian framework for AD. Rather than identifying anomalies by fitting a model, they assume that each sample can come from either a good or a bad source. Most samples are assumed to follow the regular pattern, but some might behave very differently. The two sources differ in how noisy their values are: the good data are more consistent, while the bad data are much more variable. Since the percentage of anomalous samples is unknown, the model starts with an assumed probability of anomalous samples and then, after looking at the data, updates the probability for each observation, thereby using each sample not only for classification but also for updating these two assumed distributions.

The introduction of robust statistical techniques allowed researchers to handle datasets with non-Gaussian distributions. For example, the use of the median and the median absolute deviation provided a more resistant approach to detecting outliers [64].

Dimensionality reduction techniques are widely used in AD to simplify high-dimensional data while preserving structures that differentiate normal from anomalous behavior. By reducing noise and redundancy, these methods help reveal underlying patterns more effectively. A prominent example is Principal Component Analysis (PCA), which transforms correlated variables into a set of orthogonal principal components that capture directions of greatest variance. Since anomalies often deviate from these dominant patterns, they tend to emerge in the residual subspace associated with lower-variance components [92]. Analyzing these projections allows PCA to effectively identify data points that diverge from expected norms.

3.3 Machine Learning Approaches

The development of AD has closely mirrored advances in ML, evolving from rigid statistical testing to powerful models capable of capturing complex and subtle patterns underlying in data. As datasets grew in terms of size and complexity, ML methods began to offer a more flexible alternative to classical methods.

[33] introduced a model for intrusion detection based on statistical profiling that would become the groundwork for the subsequent ML models. Even though statistical in its core, it introduced the idea of modeling behavior to detect anomalies. In the mid 1990s, [48] built on these ideas and presented a model inspired in the human immune system. Their core idea was to observe a normal sequence of Unix syscalls to learn a signature of "self". Then, any system call not observed during training was considered to be not part of that "self" group and, thus, anomalous. As it learned from normal behavior, this technique did not require any labels regarding the status of the calls, and became one of the first works in unsupervised AD, where models learn the normal behavior and identify anomalies based on deviations from it.

Later on, researchers began to shift towards distance-based and density-based methods. These approaches treat data samples in a geometric space and measure how distant each sample is from the others. One of the first examples of this is [119], which introduces the k -Nearest Neighbor approach. This approach ranks data points based on the distance to their k nearest neighbor; points with the largest distances are flagged as anomalous. The advantages of these methods are that they are both scalable and do not require assumptions regarding the data, making them appropriate for multidimensional data.

[17] advanced in this direction and introduced the idea of relative density. Rather than analyzing whether a sample is an outlier or not by distance alone, this method compared the local density of a point to that of its neighbors. The criteria to classify a point as anomalous was the density of a region: if it was sparser than its neighbors, it was deemed anomalous.

In parallel, clustering-based approaches gained traction. These methods assumed that normal data belonged to dense, well-formed clusters, while anomalies either failed to belong to any cluster or belonged to distant and small clusters. A notable example of these methods is [44], which proposes grouping data into clusters based on density thresholds and labeling non-clustered samples as anomalous. Although clustering-based methods and density-based methods often may appear similar, they differ mainly in their use of the structure: while clustering explicitly partitions the data, density generates scores locally without forming clusters.

By the 2000s, classification-based models emerged, especially in environments where labeled data was available. In supervised approaches, AD was treated as a two-class problem where models such as Decision Trees, SVMs

and MLPs were trained to distinguish between regular and anomalous samples. However, these methods require input data to be labeled, and obtaining these data is hard either because anomalies are unusual or because they are not detected when they occur.

This challenge gave rise to unsupervised training, where models were trained only on normal data and learned to model it. The clearer example of this is the one-class SVM, where a hyperplane that encloses normal data is learned, and all samples outside of it are flagged as anomalous [131]. This new methodology could be interpreted as an improvement over the original "self" idea proposed in [48].

Datasets continued to grow both in size and complexity, which caused ensemble methods such as Isolation Forests to gain popularity. Instead of modeling density or distance, these models randomly partition the data with the idea that anomalies are easier to isolate (i.e., they require fewer random splits to separate them from the rest of the data because there are few such samples and are different with respect to normal samples).

DL models became popular for AD in the 2010s. These models provided new capabilities to learn patterns from more complex data than what was previously possible. Autoencoders are a subtype of NNs that work by compressing the input data into an embedding space and then regenerating the original inputs by using the generated embeddings. By learning to compress and regenerate normal data, the intuition was that these models would perform poorly with anomalous data, which would have a different distribution than that of the normal data samples and, in turn, would yield in a high reconstruction error [127]. More advanced variants of Autoencoders like VAEs introduced probabilistic modeling by learning distributions over latent variables. This enables AD based on reconstruction probability (i.e., the likelihood of observing a data point under the model's generative distribution), which offers a better alternative to traditional reconstruction error-based scoring [9]. For sequential or time-series data, Long Short-Term Memory (LSTM) autoencoders allowed the model to capture temporal dependencies to perform AD over time patterns [108].

Generative Adversarial Networks (GANs) have also emerged as a promising approach in the AD literature. GAN-based models aim to learn the distribution of normal data by training a generator-discriminator pair: the generator learns to produce realistic data samples from latent variables generated by an encoder, whereas the discriminator learns to distinguish between real data samples and data samples produced by the generator. In this context, a sample is considered anomalous if it falls under these two cases: either the generator fails to reconstruct it, or the discriminator identifies it as unlikely to be part of the real data. The underlying idea behind this criterion is that anomalies deviate from the learned distribution of normal data samples and, therefore, cannot be correctly encoded or reproduced. With this design, GAN-based models are capable of capturing deviations in complex datasets that previous models may miss.

Researchers increasingly recognized that some anomalies could not be detected in isolation [24]. This insight led to the development of models for detecting contextual or collective anomalies (Section 2.4). A contextual anomaly is one that appears to be normal if analyzed in isolation, but is anomalous in a specific context (e.g., a certain temperature may be normal in summer but completely abnormal during winter). On the other hand, a collective anomaly refers to patterns that are only anomalous with respect to the entire dataset (e.g., a burst of failed log-in attempts). Detecting these kinds of anomalies required models capable of processing and understanding relationships across multiple data points.

More recently, GNNs and Transformers have gained popularity in AD literature due to their ability to model complex relationships in structured and sequential data. GNN-based models, such as the one proposed in [32], capture topological dependencies between entities by learning a graph structure and leveraging attention mechanisms to forecast expected behavior based on neighboring nodes. When the actual behavior deviates from these predictions, contextual anomalies can be detected. This approach is particularly effective for identifying contextual and collective anomalies that may be overlooked when analyzing nodes in isolation. The core insight is that many real-world systems exhibit interdependent behaviors, such as routers in a network influencing one another, and anomalies often emerge when these dependencies break down.

Transformers are designed to model long-term temporal dependencies in MTS using self-attention mechanisms. Unlike recurrent models that process data one step at a time, Transformers analyze entire sequences simultaneously, which allows them to model both short and long-term patterns in time series. In the context of AD, this capability allows them to detect data points that deviate from the learned temporal trends.

In summary, the evolution of ML for AD reflects a shift from rigid modeling to more flexible and data-driven options. These developments have been synthesized in several systematic surveys such as [69, 24, 113].

Chapter 4

Real-time BGP Anomaly Detection Using the Raw Topology

4.1 Introduction

In this chapter, we propose a novel approach to detect BGP anomalies by leveraging the raw topology generated by BGP announcements. Our solution uses GNNs, which can process the full BGP graph structure directly, eliminating the need for manual feature extraction. Unlike traditional ML approaches that rely on subjective feature selection, our method allows the model to learn directly from the topological and structural information embedded in BGP updates.

Our motivation stems from two key observations: First, BGP anomalies (such as outages, hijacks, and leaks) often cause significant, observable changes in the topology of the BGP topology graph. Second, GNNs are specifically designed to exploit such graph-structured data, making them ideal for real-time detection of these topological changes.

We validate our approach using real-world BGP datasets, focusing on four well-known BGP incidents [147, 27, 145, 163] that are widely referenced in the literature. Our evaluation benchmarks the performance of our model against two state-of-the-art ML-based baselines [129, 39] and a classical statistical detection method.

The main contributions of this chapter are as follows:

- We present a GNN-based solution that detects BGP anomalies directly from the raw AS-level topology, without relying on manually selected features.
- We evaluate the model using real-world BGP incident data, demonstrating that it outperforms state-of-the-art baselines in anomaly detection accuracy.
- We enrich the BGP graph representation by incorporating features from BGP messages, allowing the model to capture structural changes.

- Our method simplifies deployment and increases generalization capabilities by avoiding manual feature engineering.

The remainder of this chapter is organized as follows. Section 4.2 reviews prior work on BGP AD, outlining the limitations of classical and ML-based approaches that motivate our proposal. Section 4.3 describes the methodology, including the graph construction, model architecture, and training process. Section 4.4 presents the experimental setup, performance evaluation, and a detailed comparison against baseline methods. Finally, Section 4.5 summarizes the key findings and insights derived from our study.

4.2 Related Work

BGP is the default protocol for exchanging routing information between ASes across the Internet. Due to its lack of built-in authentication for route announcements, BGP is highly susceptible to various anomalies and attacks [19]. In the context of BGP, an anomaly typically refers to harmful changes or deviations in behavior that can significantly affect the stability and reachability of large portions of the Internet [5]. Such anomalies can manifest in different forms, including prefix hijacking, route leaks, or misconfigurations.

Detecting BGP anomalies poses considerable challenges for several reasons. First, the dynamic and decentralized nature of BGP makes legitimate routing changes and malicious activities difficult to distinguish in raw data. Second, the absence of ground truth and labeled datasets complicates the application of supervised detection techniques. Finally, BGP behavior is context-dependent due to varied and evolving routing policies among ASes.

Broadly, BGP AD methods can be categorized into classical techniques and ML-based approaches. The following sections provide a comprehensive overview of both methodologies.

4.2.1 Classical Methods

Early approaches to BGP AD primarily relied on classical techniques, including rule-based systems, heuristics, and statistical analysis of control-plane data. These early methods were based on monitoring effects on the BGP updates, such as fluctuations in the volume of announcements, origin AS changes, or AS_PATH inconsistencies, among others, to identify potential anomalies. [107] provides one of the earliest systematic taxonomies of BGP misconfigurations. Based on this, tools like the routing configuration checker [46] applied statistical analysis to router configurations, proactively detecting policy violations and latent faults before deployment, while the Prefix Hijack Alert System [89] introduced real-time monitoring of prefix origins to alert of suspicious changes.

Complementary systems like iSPY [175] empowered prefix owners to monitor their own reachability, enabling the detection of possible hijacks.

Hybrid approaches sought to overcome the limitations of pure control-plane monitoring by combining multiple data sources. For instance, [70] proposes combining BGP updates with data-plane fingerprints of suspicious prefixes, while [139] presents Argus, which correlates data-plane and control-plane data to classify anomalies into three different classes.

Other classical techniques leverage advanced statistical time series methods. [88] employed a Fast Fourier Transform (FFT) for analyzing routing instability, while [6] utilized Recurrence Quantification Analysis (RQA) for AD in dynamic, non-linear systems. Recent work has explored Multidimensional Recurrence Quantification Analysis (MdrQA), modeling inter-domain routing anomalies as deviations in collective AS behavior [132]. Furthermore, methods based on geographic data [144], PCA [76], historical BGP data analysis [84], and reachability testing [71] have enriched classical approaches.

Despite their early success, these methods have some drawbacks. The most common one is that they rely on expert-defined heuristics and hand-crafted rules, which require ongoing maintenance to remain effective in the evolving nature of the Internet [118]. The rigidity of these rules limits their capability of detecting novel anomalies. Moreover, while data-plane verification techniques like those in [175, 70] enhance accuracy, they require a substantial monitoring infrastructure (for instance, to constantly probe for reachability).

Another challenge is scalability. NetReview [62] provides logs, which allow to detect problems and pinpoint the root cause. By using a prototype, the authors prove that NetReview's resource requirements are modest and demonstrate its effectiveness against common Internet routing problems. Real-time systems such as the Prefix Hijack Alert System [89] or Pretty Good BGP [84] improve responsiveness but offer limited protection against sophisticated attacks. Furthermore, most classical methods act reactively, providing alerts after anomalies have taken place.

In summary, while classical methods have laid the groundwork for understanding and mitigating BGP anomalies, their dependence on human expertise, limited scalability, and lack of flexibility in dynamic environments restrict their effectiveness.

4.2.2 Machine Learning Approaches

As the limitations of classical methods became apparent, the interest in ML-based solutions gained traction. These promise to reduce the reliance on manual heuristics and could potentially uncover complex anomalies beyond the reach of rule-based systems. Early efforts, such as [97], represented the attributes of BGP updates as feature vectors containing attributes (such as update counts, AS_PATH length, and other indicators) and trained classifiers to recognize anomalies based on these. On the other hand, [172] used a wavelet transformation and a clustering algorithm to determine if a BGP update is anomalous.

Following these initial studies, the features and algorithms used increased in diversity. [8] provides a complete comparison of different models for BGP AD, including SVMs, Naive Bayes, and Hidden Markov Models. SVMs have become the most commonly used model for BGP AD with different input features. SVM-BGPAD uses 37 features extracted from raw BGP data, with an additional filtration step using a Fisher-Markov Selector algorithm [28]. Other approaches use fewer features, but some of them are metrics computed based on the initial features [148].

Another interesting direction involves using features derived from the BGP graph itself [129], such as centrality metrics and clustering coefficients. Some models also account for the temporal differences in the network status. For example, [26] uses an LSTM that analyzes traffic flow to learn patterns and historical features. [99] leverages Decision Trees and Fuzzy Rough Sets with common BGP features.

To tackle the lack of labeled incidents, weakly supervised techniques have emerged. [39] proposes a weakly supervised method in which their LSTM model learns from incidents labeled by other AD systems. This learned knowledge is then transferred to four well-known incidents, which are used to evaluate how well the model has learned from the weakly supervised data.

A more recent approach is BEAM [25], a model that leverages the semantics of BGP to detect anomalies in routing. BEAM models the expected behavior of each AS and detects anomalies based on deviations from this expected behavior. In the evaluation, the authors demonstrate high performance while maintaining a low false rate.

There are also examples where GNNs-based models have been used. In [117], the authors propose a framework composed of different layers, among which there is a Graph Attention Layer. Similarly, [68] demonstrated improved detection of complex structural anomalies using GNNs, but their evaluation focuses on incident-specific snapshots, limiting insights into behavior under continuous operational conditions.

Despite these advancements, ML-based methods remain constrained by several inherent challenges. The main one among them is the persistent dependence on manual feature engineering. Many models, including both traditional classifiers and numerous MLs methods, rely on pre-processed inputs extracted from BGP updates. These features require considerable domain expertise to design and maintain. Even advanced DL-based architectures, while reducing some manual effort, typically depend on carefully curated input representations or graph structures, rather than learning directly from raw network signals.

Moreover, generalization to unseen anomalies remains an unsolved challenge. Many models are primarily evaluated on historical datasets containing well-known incidents, which risks overfitting to specific patterns [97, 26]. While this often results in high performance for these familiar scenarios, their effectiveness under routine operational conditions remains less explored. In

particular, these models are rarely assessed over continuous, long-term BGP data to evaluate their performance with respect to false positives under normal routing conditions [39, 129, 67]. As a result, they may achieve high detection rates on targeted test cases but struggle to maintain performance in benign operational intervals, where excessive false alarms undermine practical usability.

Efforts to overcome these limitations include attention-based architectures [134], which attempt to focus learning on critical routing events, and weakly supervised approaches [39] that learn from partially labeled data derived from existing detection systems. GNN-based solutions [68] have demonstrated improved detection of complex structural anomalies but are predominantly evaluated on incident-specific snapshots rather than continuous operational data streams, which limits insights into their behavior under production conditions, where ensuring a low false positive rate is essential for operational viability.

In summary, while the evolution from manual feature-based classifiers to more sophisticated DL and graph-based models has significantly expanded the capabilities of BGP AD, key challenges persist. Many approaches [129, 39] remain tightly coupled to manually engineered features and rely on careful hyperparameter tuning to optimize performance on benchmark scenarios, which are typically centered around well-known incidents. However, these evaluation settings may not fully represent the complexity of production environments, where regular benign activity dominates and maintaining a low false alarm rate is crucial for developing resilient AD solutions capable of operating effectively under real-world scenarios.

4.3 Proposed Method

Building on the insights from previous work, we propose a novel approach that directly processes the raw BGP topology to detect anomalies, without relying on manually engineered features.

4.3.1 Overview

Figure 4.1 shows our proposed methodology, which can be divided into three different phases:

1. **BGP data collection:** In this phase, we make use of BGP announcements to retrieve the state of the BGP network, which are provided by BGP data collectors. These collectors provide BGP raw data containing the Routing Information Base (RIB) after processing the raw BGP updates.
2. **BGP data processing:** We process the BGP dumps obtained from the collectors to generate a raw BGP topology. This is in the form of a graph that contains three types of nodes and their corresponding edges,

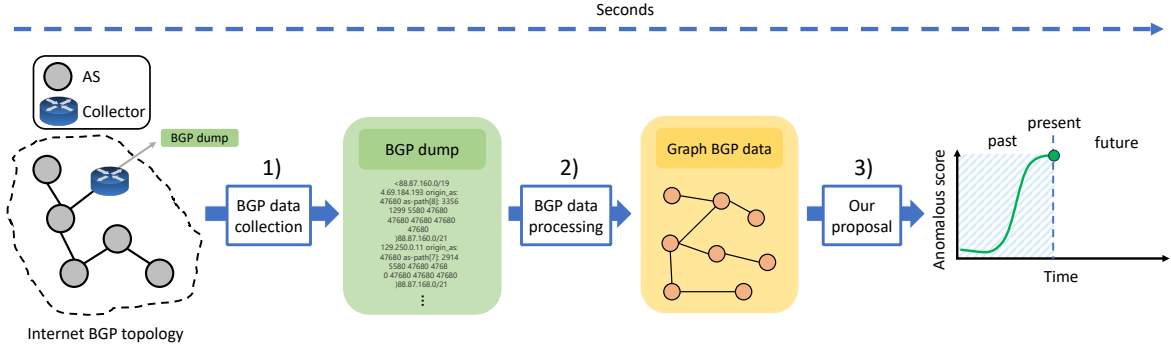


FIGURE 4.1: Proposed BGP AD solution.

which reflect the state of the BGP network at a given time. Additionally, some attributes like the state of the network (i.e., regular or anomalous) are also added.

3. **GNN modeling:** Lastly, the generated graph is fed to our GNN, which ingests the data and produces an anomalous score based on the topology and a set of attributes derived from the BGP network state. Using a GNN-based model allows us to reduce our dependency on features and harness the topology of each graph for the classification process.

4.3.2 Practical Uses of Our Proposed Solution

Our solution follows a centralized approach, relying on a central server deployed within the network's control and management infrastructure, e.g., in the Network Operation Center (NOC). To operate effectively, the server collects the full BGP table from all BGP routers at regular intervals (e.g., every 8 hours), as well as the stream of received BGP updates. While this process introduces network overhead, it is expected that the traffic load remains in the range of kilobits per second, even in large networks. Our solution is designed to process real-time control plane data, ensuring timely detection of anomalies and facilitating a rapid response. We believe that current systems can easily provide this real-time capability, since similar operations are already performed efficiently on the data plane traffic [60].

4.3.3 Data collection

In this first phase (1 in Figure 4.1), our objective is to collect all the necessary data to regenerate the state of the BGP network during a given interval. In order to do so, we need two types of data: (i) BGP dumps, which are snapshots of the full state of the network, and (ii) BGP updates, which represent specific changes in the network. These are obtained by using the library *py-bgpstream*, which allows to download BGP data from different repositories. Our idea behind this process is the following: updates only contain the differences in the state of the network, which are not enough to regenerate the full state of the network within a specific time. To overcome this, we use the

dumps, which contain the full state of the Routing Information Base (RIB) and allow to regenerate precisely the state of the network. Since dumps are available only in specific intervals of the day, we process the updates between the nearest previous dump and the start of our desired interval. By doing so, we can keep processing updates during the entire interval and be sure that our topology fully contains all the data of interest.

4.3.4 Data processing

Using the BGP dump and entries obtained from the data collection step, we generate a topology representing the state of the BGP network every 5 minutes. We start processing the nearest earlier dump (i.e., 8 hours before the start of the incident, at most) and, from there, we sequentially process BGP updates in 5-minute intervals, until 6 hours after the incident is over. For each 5-minute slot, we store a graph representing the BGP topology at that point in time.

Our resulting topology is composed of three different nodes: *Prefix*, *AS*, and *AS_PATH*. The *Prefix* nodes represent all the prefixes that are currently held by an AS and are connected to the *AS* nodes that hold them. The *AS* nodes represent all the Autonomous Systems detected during the processing of the BGP updates and are connected to the *AS_PATH* nodes that they belong to, i.e., *AS_PATH*s of which they form part. Lastly, the *AS_PATH* nodes represent all the *AS_PATH*s seen and are linked to all the *AS* nodes in a specific path. Even though this does not preserve the order of ASes, we have seen in the evaluation that it does not affect the obtained results. Figure 4.2 gives an example of our representation.

For instance, consider the following scenario: AS 13041 holds the prefix 147.83.0.0/16, with an *AS_PATH* [4637, 3356, 13041] and AS 20965 holds the prefix 84.88.0.0/16 with an *AS_PATH* [11537, 13041, 20965]. This is translated to the following nodes: two *Prefix* nodes representing the prefixes 147.83.0.0/16 and 84.88.0.0/16, five different *AS* nodes, representing each of the ASes present in this scenario, and two *AS_PATH* nodes, each representing one of the paths, and connected to the *AS* nodes forming the paths. One *Prefix* node would be connected to one *AS* node representing the AS 13041, whereas the other would be connected to another *AS* node representing the AS 20965. The node corresponding to AS 13041 would be connected to the two *AS_PATH* nodes, since AS 13041 appears in both *AS_PATH*, whereas the rest of the *AS* nodes would be connected only to one *AS_PATH* node.

Next, we assign a different set of node attributes depending on the type of node:

- *Prefix* nodes: number of community changes, origin changes, *AS_PATH* changes, number of BGP peers, and maximum and mean *AS_PATH* lengths.

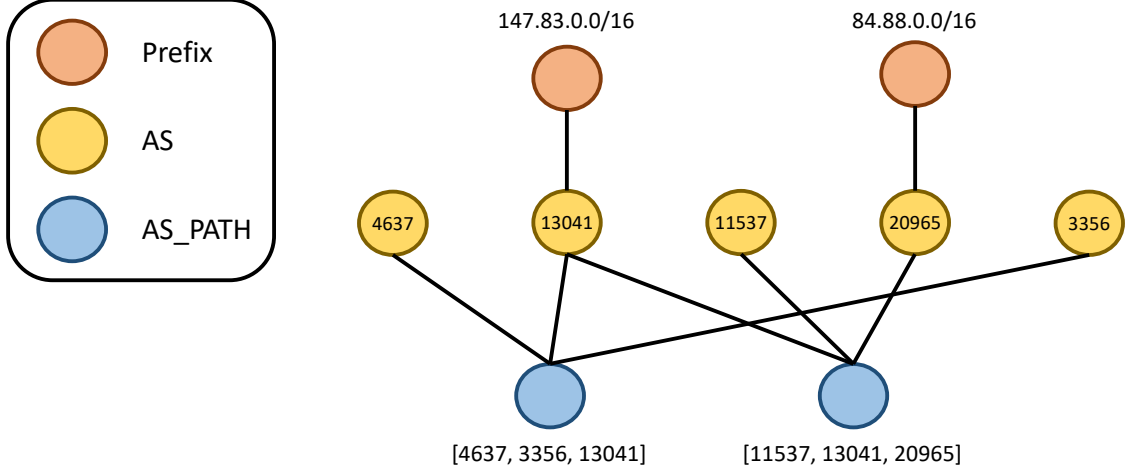


FIGURE 4.2: Example of the graph based on BGP messages.

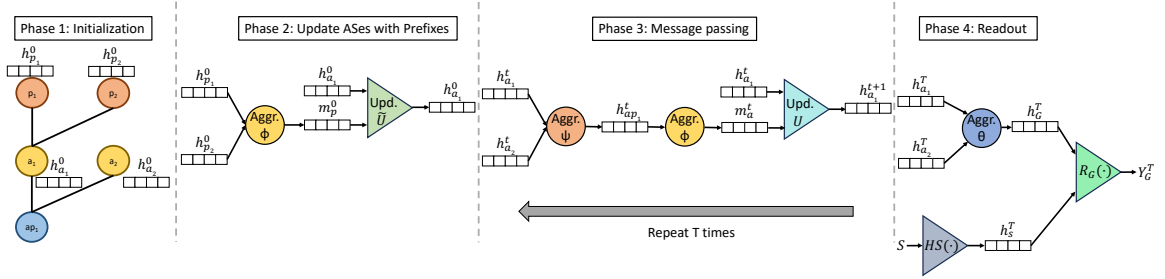


FIGURE 4.3: Overview of our message passing.

- AS nodes: the number of announcements, duplicated announcements, withdrawals, and duplicated withdrawals generated by the AS.
- AS_PATH nodes do not have any attributes.

4.3.5 Neural network model

We leverage the Message Passing Neural Network paradigm [53] for our GNN. This architecture allows propagating information among all the nodes in the graph thanks to an iterative process that generates, transmits and aggregates messages across the graph. Figure 4.3 provides an overview of the architecture of our message passing model. Table 4.1 provides an overview of the terms used in the rest of the chapter.

We assume a graph $G \in V, E$ composed of vertices V and edges E , that represents the relationships between the elements of the BGP topology. There are three vertices types: *AS*, *Prefix*, and *AS_PATH*, and each of them is represented by an n -dimensional vector, i.e., the hidden state. First, the hidden states of *AS* and *Prefix*, h_a and h_p , are initialized with their respective attributes (phase 1 in Figure 4.3). Then, the hidden states of the *Prefix* nodes are sent to the *AS* nodes, and *AS* nodes aggregate all the incoming messages, m_p (phase 2 in Figure 4.3). We aggregate the messages with *mean*, *max* and *sum* pooling operations (Equation 4.1). Then, we use a MLP to combine

Term	Description
ϕ	Aggregation function composed of the concatenation of <i>mean</i> , <i>max</i> and <i>sum</i> pooling operations.
ψ	Aggregation function composed of the concatenation of <i>sum</i> , <i>max</i> , <i>mean</i> and <i>min</i> pooling operations.
θ	Aggregation function composed of the concatenation of <i>mean</i> , <i>max</i> , <i>mean</i> , <i>min</i> , <i>std</i> and <i>median</i> pooling operations.
$\mathcal{N}(a)$	Set of neighbors of node a .
h_p	Hidden state of a node p , which represents a <i>Prefix</i> .
h_a	Hidden state of a node a , which represents an <i>AS</i> .
h_{as}	Hidden state of a node ap , which represents an <i>AS_PATH</i> .
HS_P	Function used to initialize the hidden states of <i>Prefix</i> nodes based on their attributes.
HS_A	Function used to initialize the hidden states of <i>AS</i> nodes based on their attributes.
T	Number of iterations of the message passing phase.
$\bar{d}_P$	Average number of <i>Prefix</i> neighbors per <i>AS</i> node.
$\bar{d}_A$	Average number of <i>AS_PATH</i> neighbors per <i>AS</i> node.
$\bar{d}_{AP}$	Average number of <i>AS</i> neighbors per <i>AS_PATH</i> node.

TABLE 4.1: Description of terms used.

these aggregated messages with the hidden state of the receiving AS node to generate a single hidden state, which will correspond to the new AS representation.

$$\phi = \text{cat} \left(\frac{1}{|\mathcal{N}(a)|} \sum_{p \in \mathcal{N}(a)} (h_p), \max_{p \in \mathcal{N}(a)} (h_p), \sum_{p \in \mathcal{N}(a)} (h_p) \right) \quad (4.1)$$

Next, the iterative phase of the message passing begins. This phase is repeated T times, and the hidden state of the *AS* nodes are shared among them according to their connections via *AS_PATH* nodes. Specifically, the *AS* nodes send their states to the *AS_PATH* nodes, which aggregate all the incoming messages, m_a , by using *sum*, *max*, *mean* and *min* pooling operations (Equation 4.2). Then, the *AS_PATH* nodes send the aggregated messages to their *AS* neighbors, which aggregate the messages received from the former by using *sum*, *max*, and *mean* pooling operations and use a Gated Recurrent Unit (GRU) to update their hidden state (phase 3 in Figure 4.3).

$$\psi = \text{cat} \left(\sum_{a \in \mathcal{N}(ap)} (h_a), \max_{a \in \mathcal{N}(ap)} (h_a), \frac{1}{|\mathcal{N}(ap)|} \sum_{a \in \mathcal{N}(ap)} (h_a), \min_{a \in \mathcal{N}(ap)} (h_a) \right) \quad (4.2)$$

Finally, after the T iterations, a graph-level hidden state is generated by

Algorithm 1 Internal architecture of our model

Input: A, P, AP, S ▷ AS, Prefix, AS_PATH, Previous states

- 1: **for each** $p \in P$ **do** $h_p^0 \leftarrow HS_P(p)$
- 2: **for each** $a \in A$ **do** $\tilde{h}_a^0 \leftarrow HS_A(a)$
- 3: **for** $a \in A$ **do** ▷ Message passing on ASes
- 4: $m_p^0 \leftarrow \phi(h_{p_j}^0, j \in \mathcal{N}(a))$ ▷ AS: Aggregation
- 5: $h_a^0 \leftarrow \tilde{U}(\tilde{h}_a^0, m_p^0)$ ▷ AS: Update
- 6: **for** $t = 0$ to $(T - 1)$ **do**
- 7: **for** $ap \in AP$ **do** ▷ Message passing on AS_PATHs
- 8: $h_{ap}^t \leftarrow \psi(h_{a_j}^t, j \in \mathcal{N}(ap))$ ▷ AS_PATH: Aggregation
- 9: **for** $a \in A$ **do** ▷ Message passing on ASes
- 10: $m_a^t \leftarrow \phi(h_{ap_j}^t, j \in \mathcal{N}(a))$ ▷ AS: Aggregation
- 11: $h_a^{t+1} \leftarrow U(h_a^t, m_a^t)$ ▷ AS: Update
- 12: $h_s^T \leftarrow HS_S(S)$ ▷ Previous states hidden state
- 13: $h_G^T \leftarrow \theta(h_a^T)$ ▷ Graph hidden state with AS states
- 14: $y_G^T \leftarrow R_G(h_G^T, h_s^T)$ ▷ Readout: graph state prediction

pooling all the AS hidden states with *mean*, *max*, *min*, *std*, and *median* operators (Equation 4.3, and phase 4 in Figure 4.3).

$$\theta = \text{cat} \left(\frac{1}{|A|} \sum(h_a), \max(h_a), \sum(h_a), \text{median}(h_a), \text{std}(h_a) \right) \quad (4.3)$$

To avoid excessive output fluctuations, we implement a smoothing mechanism by using the 10 most recently predicted labels. This smoothing stabilizes the predictions and reduces abrupt changes in consecutive labels. These are fed to a GRU to generate a hidden state, which is used in the readout function in combination with the graph-level hidden state to generate the output label in the range $[0, 1]$.

Algorithm 1 describes the message passing of our model. Our model receives three different objects as input: the states of the AS nodes (A), the states of the *Prefix* nodes (P), and the states of the *AS_PATH* nodes (AP), and operates on five key steps:

Initialization: The states of the *Prefix* and AS nodes are initialized (lines 1-2) by using a different non-linear function for every node type.

Message Passing Prefixes → ASes: The state of every AS node is updated using the attributes of the prefixes that it holds (lines 3-5).

Iterative Message Passing along AS_PATHs: The states of the AS nodes are shared among their neighbors, i.e., other ASes with which they share a common AS_PATH (lines 6-11). These messages are shared via the AS_PATH node: this node receives the messages of all the AS nodes that form part of the path, aggregates them using a combination of

mean, *sum*, *max* and *min* aggregation functions (ψ) and then sends this state back to all the *AS* nodes, which aggregate these messages using a combination of *sum*, *max* and *mean* aggregation functions (ϕ). Then, these aggregated messages are used to update the hidden state h_a^{t+1} .

Graph hidden state and previous labels: Once the T iterations are finished, we produce a graph-level hidden state by pooling all the *AS* node hidden states into a global graph hidden state h_G (line 13). The pooling operation (θ) is a combination of *mean*, *max*, *min*, *std* and *median* aggregation functions. We also generate a hidden state representing the previous graph hidden states by passing the last S generated labels through a GRU (line 12). As we mentioned previously, the goal of the hidden state of the previous labels is to smooth the output to prevent sudden changes.

Readout: Finally, the graph-level and the previous labels' hidden states are passed through the readout function R_G to produce the final label (line 14).

Note also that the main motivation of the *AS_PATH* node is to prevent the lack of consistency between two different *AS_PATH*s sharing a common *AS* node. In contrast, consider an architecture that only sends messages directly between *AS* nodes: this would imply that, for two *AS_PATH*s [1, 2, 3, 4, 5] and [1, 2, 3, 6, 7], states of *AS* nodes of the first and second paths would get mixed at node 3.

The computational cost of Algorithm 1 is linear in the number of nodes in the graph. Let $|A|$, $|P|$, and $|AP|$ denote the number of *AS*, *Prefix*, and *AS_PATH* nodes, respectively.

Initialization: The hidden states of *Prefix* and *AS* nodes are initialized with a cost of $\mathcal{O}(|P| + |A|)$.

Message Passing Prefixes $\rightarrow$ ASes: Each *AS* node aggregates messages from its *Prefix* neighbors and updates its state. Assuming each *AS* has an average of $\bar{d}_P$ *Prefix* neighbors, this step has a cost of $\mathcal{O}(|A| \times \bar{d}_P)$.

Iterative Message Passing along AS_PATHs: During T iterations, each *AS_PATH* node aggregates messages from its neighboring *AS* nodes using the function ψ , and each *AS* node aggregates messages from its *AS_PATH* neighbors and updates its state. This results in a total cost of $\mathcal{O}(T \times (|AP| \times \bar{d}_{AP} + |A| \times \bar{d}_A))$.

Readout: The graph-level hidden state is computed by pooling all *AS* node states with the θ pooling function, and this graph-level representation is used to generate the output label with the readout function R_G . Both operations have complexity $\mathcal{O}(|A|)$.

Assuming T is constant and the node degrees are bounded, these operations result in a final cost of $\mathcal{O}(|P| + |A| + |AP|)$.

In terms of memory complexity, Algorithm 1 is primarily determined by the storage of the hidden states, the messages during the message passing, and the parameters of the diverse layers of the model. Considering that the hidden states have a size d (Section 4.4.3), the memory required to store them is proportional to $d \times (|P| + |A| + |AP|)$. Additionally, since during the message passing each node stores the aggregated messages from its neighbors, the memory requirements to store them is proportional to $d \times (|A| + |AP|)$. Finally, the memory required for storing the model parameters is independent of the number of nodes in the graph.

4.4 Evaluation

In this section, we use a dataset composed of four well-known BGP incidents (Section 4.4.1) to study the performance of our model. We also benchmark it against two state-of-the-art ML-based proposals, as well as a classical method. Lastly, we also analyze the performance of the system in random intervals where no anomaly has been reported. With this, we aim to understand the performance of the algorithm in (assumed¹) non-anomalous scenarios. Performing well in non-anomalous environments is critical for production scenarios.

4.4.1 Dataset

Our dataset is composed of four well-known BGP incidents. Table 4.2 gives the details of the incidents. We have selected these events for comparison and reproducibility purposes, since other works also use them [129, 68]. To recreate the BGP topologies of the samples contained in each incident, we obtained the raw BGP updates from a BGP routing data collection platform, in this case, we chose RIPE RIS [122].

Incident	Victim AS	Start	Duration (mins)	Listener
TTNET	9121	24-12-2004	627	RRC05
INDOSAT	4761	02-04-2014	150	RRC04
TM	4788	12-06-2015	182	RRC04
AWS	200759	22-04-2016	115	RRC04

TABLE 4.2: BGP incidents.

RIPE RIS offers a series of collectors, which are nodes ingesting BGP routing data via peering sessions, and are widely used to collect BGP data. All the RIS collectors start with "RRC", followed by two digits. As of February of 2024, these digits start at 00 and end at 26. The collectors used in our dataset are RRC04 and RRC05 [122]. We generated the dataset according to the methodology described in Section 4.3.4.

¹We argue that, to be useful in production scenarios, an AD system should not constantly raise false alarms to prevent the operator from wasting human resources reviewing these alarms.

4.4.2 Evaluation Metrics

To evaluate the classification, we use the precision, recall, and the F1-score performance metrics, which are typically used to evaluate the performance of classification models. They are defined by the ratio of true positives (TP; correctly identified anomalous samples), true negatives (TN; correctly identified regular samples), false positives (FP; regular samples incorrectly identified as anomalous) and false negatives (FN; anomalous samples incorrectly identified as regular):

Precision: Measures the accuracy of positive predictions, representing how many of the predicted positives are actually true positives. It is defined as: $Precision = \frac{TP}{TP+FP}$

Recall: Measures the ability to detect all actual positives, indicating the proportion of true positives identified out of all actual positives. It is defined as: $Recall = \frac{TP}{TP+FN}$

F1-score: Provides a balanced evaluation of the performance by combining precision and recall. It is the harmonic mean of these two metrics and is given by: $F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$

4.4.3 Experimental Setup

Considering that the focus of our evaluation is to test the generalization capability of our model, which ensures the best fit for real-time production scenarios, we choose to use shared parameters across all dataset combinations (Table 4.3). Specifically, we use a learning rate η of 0.0005 with an *Adam* optimizer and apply a learning rate decay factor γ of 1/10 every 15 epochs. Each model is trained for a total of 30 epochs. We use $|S| = 10$, representing the last 10 previous graph labels, as described in Algorithm 1. We set the size of the hidden states to $d = 128$ and $T = 1$ (i.e., the number of message passing iterations).

For [129], we use a regularization parameter $C = 4$ and an RBF kernel with a kernel coefficient $gamma = 1$. On the other hand, for [39], we use a threshold of 1 and a weight of 0.5 for each model in the ensemble. We train each model in the ensemble with the same parameters as the authors, and then fine-tune them during 10 epochs. Lastly, for the classical method, we use a threshold of 1.126.

4.4.4 Classification Accuracy

To understand how our model performs, we designed an evaluation strategy based on permutations over the four well-known BGP incidents used in this work. We evaluate the model by selecting one incident as the test set and using the remaining three for training. This approach ensures that the model is assessed on entirely unseen incidents, providing a realistic and robust view of its generalization capabilities across different types of BGP anomalies.

Since we have four well-known incidents, the permutations result in four evaluation scenarios, named A, B, C, and D. Each scenario corresponds to a different permutation of training and testing incidents. Table 4.3 shows the specific combinations used in each case.

Name	Training	Testing
A	TM, TTNET, INDOSAT	AWS
B	AWS, TM, TTNET	INDOSAT
C	AWS, TM, INDOSAT	TTNET
D	AWS, TTNET, INDOSAT	TM

TABLE 4.3: Dataset permutations.

Table 4.4 shows the classification performance of our model using common classification metrics of precision, recall, and F1-score (Section 4.4.2). We can see that the F1-score is equal to or above 79% in all of the scenarios and, most importantly, the rest of the metrics are consistent among the different dataset combinations. We will compare it against two state-of-the-art ML-based proposals and a classical method in Section 4.4.6.

Beyond analyzing the classification performance, we can also study the evolution of the predicted labels during the incidents. Figure 4.4 shows the evolution of the predicted labels as well as the ground truth (right column) for the four dataset combinations, and during the incident of the testing dataset. In scenarios A, B and C, the model clearly predicts a regular status before the anomaly starts. Then, as the anomaly begins, this status swiftly changes to anomalous. On the other hand, in scenario D we can see two spikes shortly before the anomaly starts. In the four scenarios, the anomaly is detected during the majority of its duration, and the predicted state returns to normal near the actual end of the anomaly.

The differences between the predicted labels with respect to the ground truth can be explained by the fact that the labels are human-generated, which may imply that the labels do not properly fit the actual status of the network. The criteria by which the expert decides where the incident begins and where it ends are subjective and may be prone to errors or subtle shifts hard to be manually detected. Another aspect making manual labeling of data samples hard are the delays induced by the BGP convergence process [133].

Scenario	Training			Testing		
	Precision	Recall	F1-score	Precision	Recall	F1-score
A	0.88	0.97	0.93	1	0.65	0.79
B	0.92	0.98	0.95	1	0.87	0.93
C	0.71	0.96	0.82	0.91	1	0.95
D	0.55	1	0.71	0.83	0.92	0.87

TABLE 4.4: Performance metrics.

Finally, note also that the model predictions, which represent an anomaly score, are mostly near 0 or near 1. This anomaly score, which can be interpreted as a confidence score in the prediction, indicates that the model clearly distinguishes between regular and anomalous samples.

4.4.5 Real-time prediction capability

One of the most important features provided by our model is the capability of performing predictions in real-time. This capability is essential to enable users to gain insight into the status of the network and to react quickly in case of anomalies. To ensure this, we measured the processing time of the entire pipeline from Figure 4.1. Specifically, steps 1) and 2) involve data pre-processing (i.e., downloading raw BGP data and creating the graphs representing the BGP topology), and step 3) involves the predictions of the status labels using these raw data.

We generated data from 100 random intervals from years [2010, 2020], which resulted in 19100 graphs. As shown in Figure 4.5, in 90% of the cases, the processing steps 1) and 2) to generate a single graph representing 5 minutes of BGP data take 10 seconds or less. Regarding the inference time, it is 0.6 seconds in the worst case, which shows that the time required to process the entire pipeline to classify 5 minutes of BGP data is in the order of seconds. This makes it possible to run this algorithm in a production environment to generate alerts in near-real-time.

4.4.6 Benchmarking Against the State of the Art

Machine Learning-based Methods

Next, we compare our solution against two state-of-the-art proposals [129, 39], which are based on ML methods.

In [129], the authors manually derive 14 graph features out of the BGP topology. These features are fed into an SVM that identifies anomalies. We choose this paper [129] for comparison because we aim to understand how the performance of a solution where features are carefully selected compares to ours. In our proposal, we do not use topology features but rather use the entire raw topology as input, enabling the model to automatically learn the relevant features in the training process.

Scenario	Ours			Our version of [129]		
	Precision	Recall	F1-score	Precision	Recall	F1-score
A	1	0.65	0.79	0.24	1	0.39
B	1	0.87	0.93	0.29	0.97	0.45
C	0.91	1	0.95	0.40	0.98	0.57
D	0.83	0.92	0.87	0.97	0.94	0.96

TABLE 4.5: Comparison of testing performance metrics.

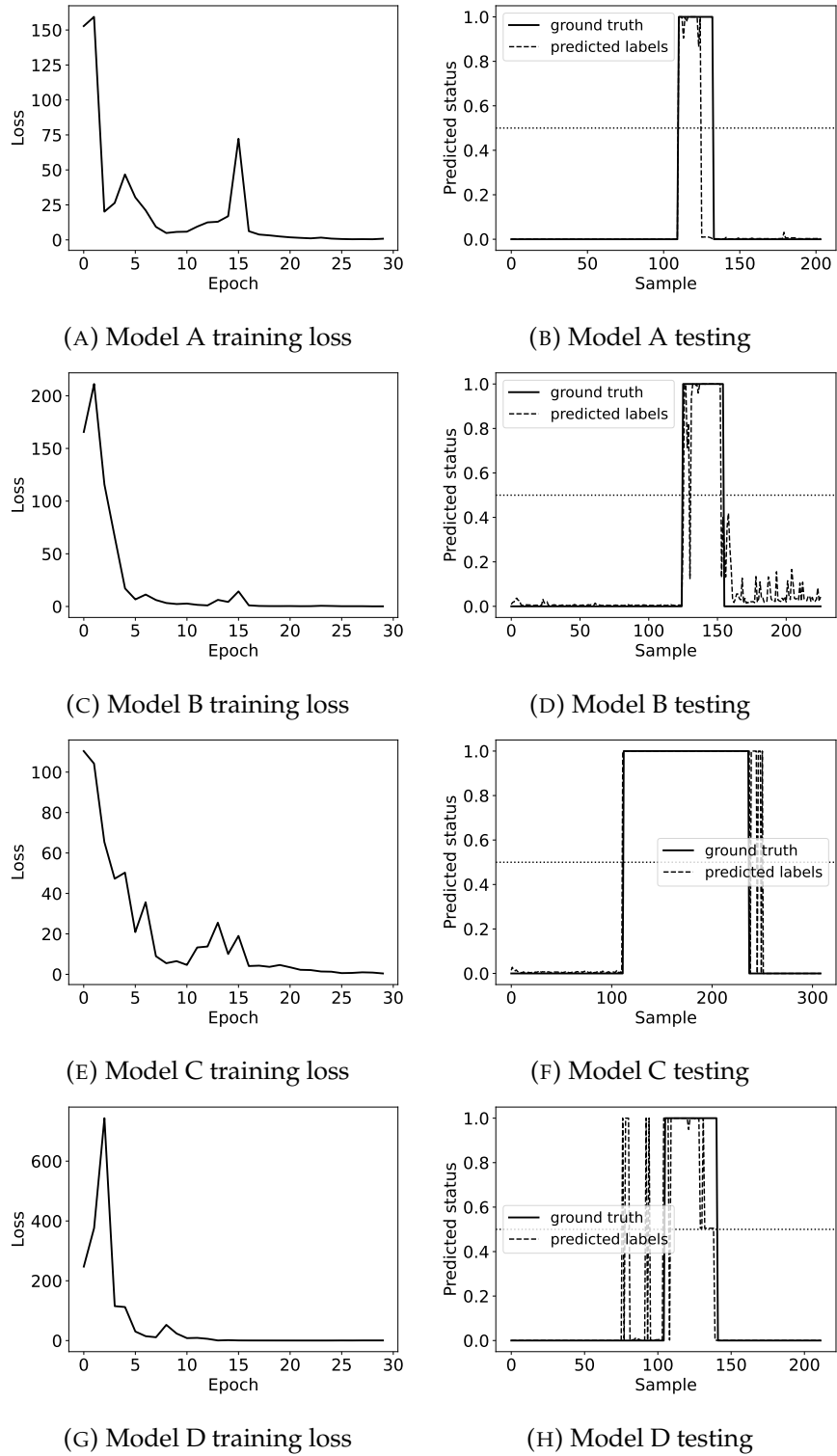


FIGURE 4.4: Anomaly prediction for the four different permutations.

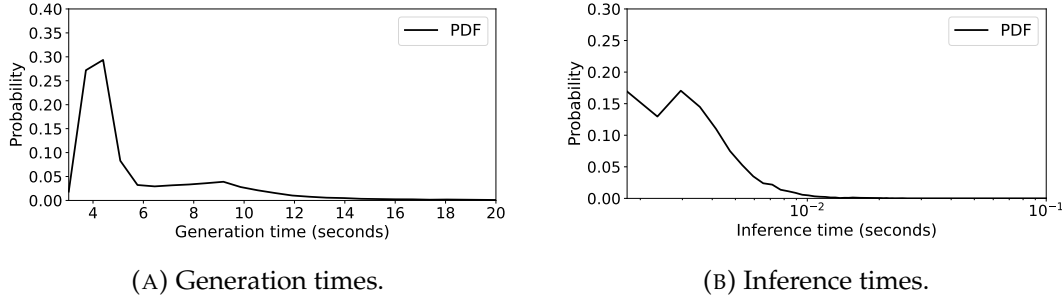


FIGURE 4.5: Graph generation and inference times.

To build the SVM model, we have followed the methodology explained in [129], that is, we processed our previously generated topology graphs to calculate the graph attributes. Since the authors did not make public the data or the code to replicate their experiments, we did our best effort to replicate their methodology using features generated from our graphs in the closest way possible to their explanation. The only difference being that we have used the same training and hyperparameters across all scenarios, which is the opposite to their approach, where the hyperparameters of the SVM and features used are optimized for every scenario.

Table 4.5 compares the results obtained by our model with the state of the art (referred to as SVM). We can see that the performance of the model is good across the board, and higher than the SVM model in all scenarios. This shows that our proposal outperforms, in the considered scenarios, the manual selection of features.

We also choose to evaluate our model against [39]. This is a state-of-the-art proposal where the authors use an ensemble deep-learning method to detect BGP anomalies. The author’s proposal can also identify the type of anomaly: Prefix Hijack, Route Leak, Breakout, Fake Route, and Defcon. The ensemble model is trained with 3085 weakly supervised events, containing anomalies of the 5 aforementioned types. Then, the ensemble is fine-tuned for each of the 4 scenarios (Table 4.3) by using three well-known incidents for training and leaving one for testing, as we do.

The authors use different hyperparameters for each scenario, effectively using different trained models. In this chapter, we have a different goal and we aim to train a model that can generalize to unseen scenarios. This means that the same model should provide good performance across all the scenarios. To benchmark our model in this scenario, we replicate their training and then execute the fine-tuning stage with common hyperparameters across all 4 scenarios. We have systematically explored the hyperparameter space and picked the ones that render the best result for [39]. Table 4.6 shows the performance metrics that we obtained with shared hyperparameters.

Scenario	Ours			[39] with shared parameters		
	Precision	Recall	F1-score	Precision	Recall	F1-score
A	1	0.65	0.79	0.86	0.80	0.83
B	1	0.87	0.93	0.19	0.92	0.32
C	0.91	1	0.95	1	0.11	0.20
D	0.83	0.92	0.87	0.79	0.92	0.85

TABLE 4.6: Comparison of testing performance metrics with the ML model from [39].

Classical methods

Recent methods for AD rely on ML, but researchers have also applied statistical methods such as [81] in other domains. To evaluate the performance of a classical approach, we process the topology data to generate graph attributes following [129] (Section 4.4.6). We then compute the z-score for each sample by subtracting the mean μ of each feature and dividing the result by the standard deviation σ of that feature (Equation 4.4). The z-score quantifies how much a sample deviates from the mean in terms of standard deviations, and we classify a sample as anomalous if its z-score exceeds a predefined threshold. To determine the optimal threshold, we evaluate a range of possible values based on the F1-score, and apply this threshold across all scenarios (Table 4.3). Unlike ML-based methods, this approach does not require a training phase and relies solely on statistical calculations. Table 4.7 shows the results obtained in each scenario. We can see that the z-score classification obtains outstanding results in precision and a high F1-score in two of the scenarios (B and D). However, the performance in the other two scenarios is much worse than our solution, highlighting the importance of building a solution that can accommodate to widely different situations.

$$z = \frac{x - \mu}{\sigma} \quad (4.4)$$

Scenario	Ours			z-score method		
	Precision	Recall	F1-score	Precision	Recall	F1-score
A	1	0.65	0.79	1	0.48	0.65
B	1	0.87	0.93	1	0.93	0.97
C	0.91	1	0.95	0	0	0
D	0.83	0.92	0.87	0.97	0.94	0.96

TABLE 4.7: Comparison of testing performance metrics with a classical technique.

4.4.7 Testing on regular intervals

Finally, we test our proposed method on regular data only, in order to verify whether our model actually detects anomalies when an actual issue is present

or, at least, when there is a high level of suspicion (i.e., a low false positive rate). We must note that, to the best of our knowledge, this test is not usually present in the literature, except for the regular periods immediately before or after the anomaly.

To test this capability, we generated 100 random intervals of 16 hours using data obtained from RIPE's RRC04, spanning from the year 2010 to the year 2020. Then, we use our model trained with the data of the four different scenarios to perform AD, i.e., we do not re-train it. We expect that the model will classify most of the samples as regular.

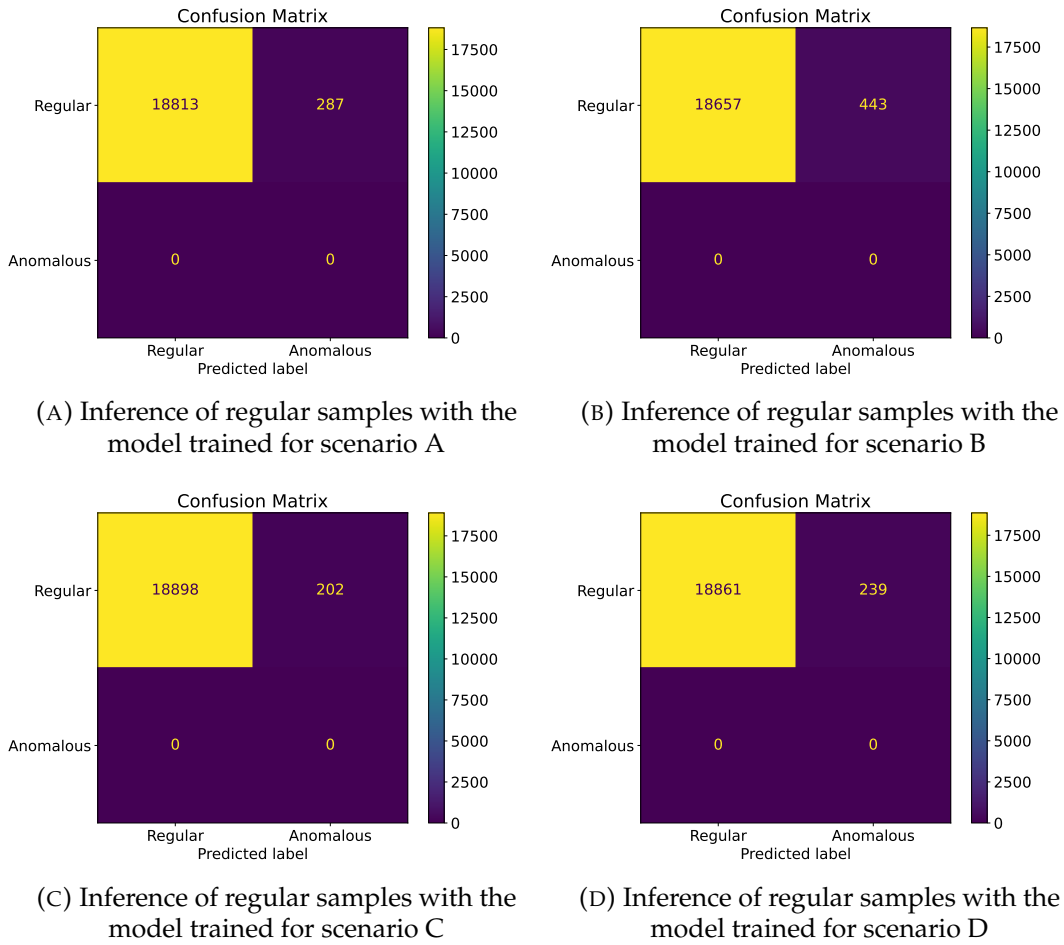


FIGURE 4.6: Inference of regular samples.

Figure 4.6 shows the four confusion matrices of the results obtained. We can see that the models trained in scenarios A, B, C, and D predict correctly as regular the 98.48%, 97.68%, 98.94% and 98.74% of all samples, respectively. As expected, our model has a low false-positive rate. Since the regular intervals are random periods of 16 hours between the 01-01-2010, and 12-31-2020, we cannot completely ignore the possibility that anomalous samples could have been generated as well.

4.5 Summary of Outcomes

In this chapter, we have presented a novel ML model to detect BGP anomalies based on the AS-level topology of the BGP network, using a GNN specifically designed to work with graph-structured data. Compared to previous classical and ML approaches, our model does not require feature selection nor hyperparameter tuning to obtain a good performance, and instead relies on the raw BGP topology and a fixed set of features. This capability makes it more suited for dealing with real-time data, which allows it to react quickly whenever an issue arises.

We have generated a dataset consisting of 4 well-known BGP incidents using data obtained from a public repository (RIPE RIS). Then, we divided the incidents into four scenarios by creating permutations, each using three incidents for training and one for testing. Our results show that our model shows a remarkable performance when compared against the two state-of-the-art models and a classical model.

Chapter 5

Interpretable Contextual Anomaly Detection

5.1 Introduction

In this chapter, we address the challenge of CAD in time series, aiming to identify anomalies that are difficult to detect using traditional network AD methods. Specifically, we focus on traffic data at the OD flow level, where a contextual anomaly is defined as a deviation in the behavior of an OD flow relative to other flows with similar past historical patterns (i.e., the *context flows*). Our objective is to develop a model capable of learning these contexts and identifying deviations in real-time traffic behavior.

Modeling flow contexts for AD poses several challenges. Contexts are dynamic, anomalies may be subtle and fall within normal operational ranges, and there are no large public datasets with labeled contextual anomalies [135]. To tackle this, we adopt an unsupervised DL approach, as it enables learning patterns without relying on labels, and provides flexibility in capturing the underlying structure of network traffic [171].

We propose two iterations of our model. GAT-AD, which introduces graph-based representations for CAD, and GAT-AD+, which is an extension that provides many improvements. GAT-AD+ is a DL-based solution that models the network as a graph where nodes represent OD flows and edges encode contextual relationships. Our model builds on GAT [150], a type of GNN that incorporates an attention mechanism to learn pairwise similarities between flows. The attention coefficients produced by GAT-AD+ reflect the probability of two flows belonging to the same context, allowing the model to perform robust contextual predictions. Deviations between predicted and observed traffic are then used to identify anomalies.

We present and evaluate both versions, showcasing the improvements of GAT-AD+ over GAT-AD in terms of performance and efficiency. For completeness, we include GAT-AD as a baseline in our experiments to quantify the advances introduced by the new design, but the chapter focuses on GAT-AD+.

To evaluate GAT-AD+, we conduct extensive experiments on two datasets:

- **Abilene** [173], a backbone network dataset with 6 months of OD flow traffic. Since it lacks labeled anomalies, we inject synthetic contextual anomalies of two types to benchmark the models.
- **WaDi** [2], a water distribution dataset widely used for time series AD, containing reliably labeled anomalies and offering a challenging testbed for contextual detection.

Our experimental results show that GAT-AD+ substantially outperforms the two baselines (GDN [32] and GAT-AD) in both datasets. On WaDi, GAT-AD+ achieves an F1-score 10.07% higher and a recall 14.1% greater than the best baseline (GDN). In Abilene, GAT-AD+ improves the recall over the best baseline by approximately 17.5%.

The main contributions of this chapter are:

- We propose GAT-AD and its second iteration, GAT-AD+, a GAT-based model for contextual AD in OD flow traffic, capable of learning contexts in an unsupervised manner without requiring labeled anomalies.
- We introduce a simplified and interpretable architecture, where attention coefficients directly quantify the relevance of flows in the context, improving model transparency and robustness.
- We demonstrate through experiments that GAT-AD+ outperforms the state-of-the-art baseline in detecting contextual anomalies, both in network traffic data (Abilene) and in general time series (WaDi).
- We perform detailed evaluations, including sensitivity analysis, overlap studies, and few-shot learning experiments, to characterize the performance and robustness of GAT-AD+.

The remainder of this chapter is structured as follows: Section 5.2 presents the related work. Section 5.3 presents GAT-AD, the initial version of our model. Section 5.4 describes a second iteration named GAT-AD+, detailing its architecture and AD approach. Section 5.5 provides the experimental evaluation and comparative analysis. Finally, Section 5.6 summarizes the key outcomes and insights of this study.

5.2 Related Work

AD in networks is a fundamental task to ensure reliability and security. Traditionally, this task has been approached by monitoring individual network elements or aggregating global metrics, often neglecting the relationships between elements that naturally exist in networked systems. However, anomalies in networks rarely occur in isolation. The nodes in networks are interconnected, and thus their behavior is correlated with their neighbors. Thus,

detecting anomalies without considering spatial context limits the detection capability, especially for complex incidents.

In this section, we explore how the concept of context has been incorporated into AD models. We focus specifically on spatial context, understood as the relationships and dependencies between neighboring nodes in a network.

We analyze two families of methods: first, classical and ML approaches that leverage spatial information without explicit graph representations, and second, graph-based approaches, including recent advances in GNNs, that integrate topological data into their predictive models.

5.2.1 Classical and Machine Learning Approaches

Network traffic AD has been widely studied in the past [3]. Classical methods such as PCA have been extensively used. For instance, in [37] the authors use PCA to analyze Traffic Matrices (TMs), which represents all the OD flows in a network, and decompose them into regular and anomalous subspaces. Then, they use the Euclidean distance to measure deviations between two consecutive samples in the regular subspace. The algorithm detects an anomaly when the distance falls outside the range of regular values. There are many known limitations to PCA, though, as explained in [16]. Additionally, other classical methods such as Exponential Weighted Mean Average (EWMA) have also been used for network AD. In [23], the authors propose a method based on EWMA that predicts time series and performs detections based on the deviation of the prediction from the ground truth.

ML models have traditionally been applied to AD by analyzing individual data points or relying on temporal trends and to deal with the high complexity and vast amount of data collected [152]. LSTMs are especially suitable for learning temporal relationships in the data and generate low-dimension representations out of time series. For instance, [126] proposes using an LSTM to model the normal behavior of time series data. The proposed model generates a latent vector representation (i.e., a compressed representation) of the input data and passes it to a SVM to classify it as either regular or anomalous. The authors train the model using only regular traffic, and after training, they label outliers as anomalous samples. In [103], the authors propose a hybrid model that combines an LSTM with a VAE to perform unsupervised AD in time series data. On the other hand, [21] proposes a method that combines CNN and a Recurrent Neural Network (RNN) to perform supervised time series AD, and validates its approach in a real industrial scenario. However, these models do not offer support for permutation invariance across the different elements in the AD use case considered in this chapter. For example, to represent the potential relationships between all OD flows and automatically extract the most relevant ones for each OD flow (i.e., its context).

Some approaches have attempted to incorporate spatial context by leveraging information from neighboring nodes to improve detection accuracy.

For instance, works such as [141, 14] propose exploiting the behavior of neighboring nodes, under the assumption that neighboring nodes typically exhibit correlated patterns and that deviations from this expected behavior can be considered anomalous. Other methods, such as [61], introduce quantitative metrics to evaluate the similarity of neighbors and distinguish between regular variations and anomalous ones.

While these approaches represent an important step forward, they still have several limitations. First, many of these methods are designed for specific environments, which complicates their transferability to other scenarios. Second, they often rely on handcrafted features and statistical comparison between neighboring nodes, which limits their capability to capture more complex relational patterns [18]. These limitations restrict their generalization capability.

As a result, despite demonstrating the value of incorporating the spatial context, these ML approaches remain relatively narrow in scope. They are insufficient for addressing the complexity and diversity found in modern network AD tasks.

5.2.2 Graph-based Approaches

Graph-based models provide a straightforward framework for incorporating the structural information of network topology into the AD process. By explicitly modeling nodes and their relationships, these approaches can capture both local structural patterns and global behavior. Early work in this area, such as [4], analyzed node neighborhoods to detect structural irregularities, demonstrating that anomalous behavior often appears as deviations from expected connectivity patterns or attribute distributions among neighbor nodes. Other examples, such as [74], propose a GNN-based framework for network modeling. The authors prove that their framework is capable of accurately predicting network performance metrics in a fraction of the time required by traditional simulators, making it suitable for real-time network optimization.

The emergence of GNNs has significantly advanced the capabilities of graph-based AD. Proposals such as [105, 124] leverage the expressive power of GNNs to aggregate information from neighboring nodes, which enables them to capture complex contextual dependencies and exploit them to improve the AD performance. By learning node embeddings that encode information from the surrounding context, these models have shown promising results in identifying anomalies.

Recent proposals rely on learning the underlying contexts automatically. One such example is [32], which explicitly models inter-sensor dependencies by learning the graph structure using trainable node embeddings. This learned graph is then used within a graph attention-based forecasting module to predict the node behavior, enabling AD as deviations from the expected patterns. While this proposal effectively learns sensor embeddings

and graph structures from training data, these representations are specific to the dataset on which it was trained. As a result, applying the model to a different system requires full retraining, since the learned embeddings do not generalize across different environments. Additionally, it relies on a hyperparameter that defines the number of incoming edges per node. Choosing an appropriate value can be challenging and can significantly affect the performance.

[75] proposes a self-supervised AD approach for attributed networks that integrates both local and global contextual information. The model captures structural and attribute-based patterns by predicting the hop counts between node pairs, and uses these predictions to detect anomalies. The underlying intuition is that anomalous nodes, due to irregular connections or inconsistent attributes, tend to be topologically distant from their local neighborhoods once weak links are removed. By learning to predict hop counts across the network, the model leverages both immediate and more distant relationships. Anomalies are identified as nodes whose average predicted hop distances to their neighbors are unexpectedly high. This method is computationally intensive due to hop count calculations and requires careful hyperparameter tuning for optimal performance.

Nevertheless, several challenges remain unresolved. One of the most critical limitations is the reliance on labeled datasets that reflect the diversity of network topologies and contextual behaviors encountered in real-world environments. While some network monitoring datasets provide anomaly labels, they often lack detailed annotations regarding the underlying contexts in which anomalies occur, as these can be complex, dynamic, and not formally defined. Moreover, CAD remains a relatively niche area. In summary, although graph-based approaches have demonstrated encouraging progress towards CAD, they continue to be constrained by issues of data scarcity, generalization, and interpretability, which underscores the need for further exploration in this domain.

5.3 GAT-AD

Figure 5.1 shows an overview of GAT-AD. It is divided into three main blocks:

- **Data representation:** Converts traffic measurement data into graph-structured representations, where graph nodes contain the OD flow traffic measurements during a specific time period t .
- **Contextual predictor:** Learns the context of a target OD flow and uses it to predict the next value using only information from the context flows at $t+1$.
- **Anomaly detector:** Applies a detection method to classify anomalous deviations in OD flows with respect to their context.

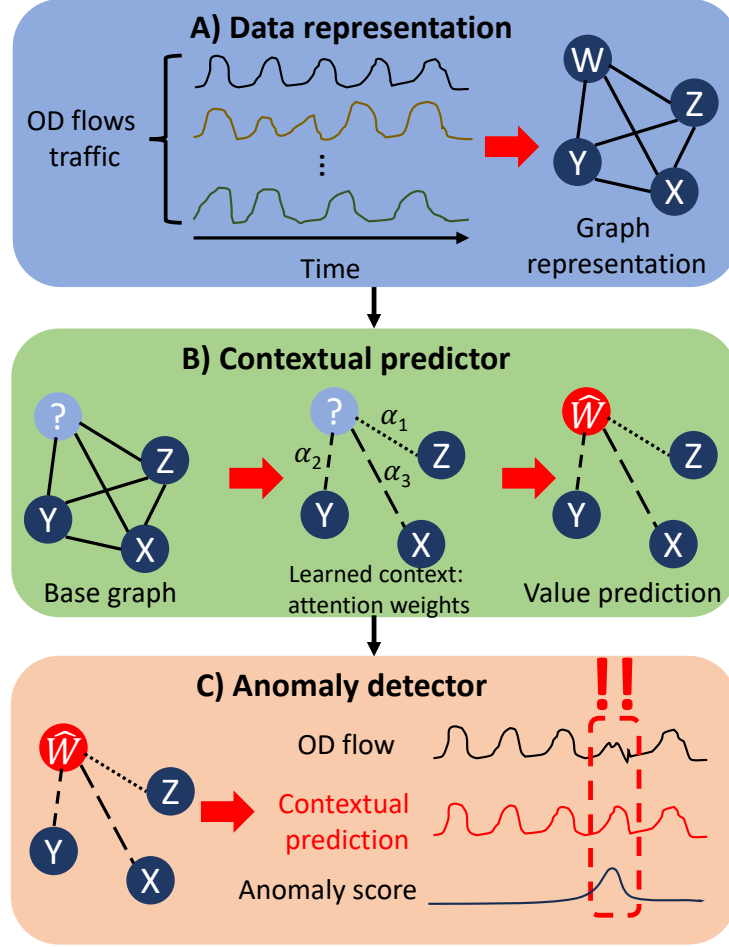


FIGURE 5.1: Overview of GAT-AD.

5.3.1 Data representation

Our method is designed to process OD flow traffic measurements as input [173], which are represented by a matrix $\mathbb{R}^{M \times T}$, where M is the number of ODs flow in the network, and T is the number of temporal samples per flow. We create a graph representation G for every time sample t (see Figure 5.1-A). We represent each OD flow as a graph node f_i , and connect flows with graph edges e_{ij} that represent their relationships (i.e., the context). Since the context of flows is initially unknown, we connect flows all-to-all; then we use a contextual predictor that determines the context of a flow.

5.3.2 Contextual predictor

A main feature of our method is that it learns the underlying flow contexts. To do so, we use a GNN model that learns to set attention weights $\alpha_{i,j}$ on the edges between flows (see Figure 5.1-B). These edges represent the relevance of the connected flows to their context. Our GNN model comprises three main inputs: (i) the OD flow traffic measurements in a given time window $[t - W, t]$, (ii) a positional encoding that uniquely identifies flows (one-hot

Algorithm 2 GAT-AD contextual predictor

Input: F, P, O

- 1: **for each** $f \in F$ **do** $h_f^0 \leftarrow \text{MLP}_F(f)$
- 2: **for each** $p \in P$ **do** $h_p^0 \leftarrow \text{MLP}_P(p)$
- 3: **for each** $o \in O$ **do** $h_o^0 \leftarrow \text{MLP}_O(o)$
- 4: **for** $p_i \in P, p_j \in \mathcal{N}(i), o_i \in O, o_j \in \mathcal{N}(i)$ **do**
- 5: $\alpha_{ij} \leftarrow \text{Attention}([h_{p_i}, h_{o_i}], [h_{p_j}, h_{o_j}])$
- 6: **for** $k = 0$ to $(K - 1)$ **do**
- 7: **for** $f \in F$ **do**
- 8: $m_f^{k+1} \leftarrow \sum_{f_j \in \mathcal{N}(i)} h_{f_j}^k \times \alpha_{ij}$
- 9: $h_f^{k+1} \leftarrow U(h_f^k, m_f^{k+1})$
- 10: $f^{t+1} \leftarrow R_f(h_f^K)$

encoding), and (iii) a flow label that identifies the OD flow where the AD will be performed (one-hot encoding).

Three main ideas motivate the design of our contextual predictor:

- **Positional encoding:** We need to identify uniquely flows to then define in the model which ones are related between them, and what is the relevance to their context.
- **Target flow label:** We need the model to identify the flow where the contextual prediction is made. Then, it predicts the next value in the flow ($t+1$) by considering only measurements from its context flows at $t+1$.
- **Attention weights:** Each flow is related to the rest of the flows of its context, but different flows may have different degrees of relation. For this reason, our model implements an attention mechanism (GAT [150]) that quantifies the level of relation between flows.

Our GNN-based model (Algorithm 2) implements a message passing process that combines the information of flow measurements F , the positional encodings P , and the target flow label O to predict the traffic of the target flow $\hat{f}$ at $t+1$. First, the hidden states of flow measurements h_f^0 , positional encodings h_p^0 , and target flow label h_o^0 are initialized (lines 1-3) by using a MLP. Then, the hidden states of positional encodings h_p^0 , and the target flow label h_o^0 are passed through the *Attention* layer to generate attention coefficients α_{ij} for each flow pair (lines 4-5). Here, $\mathcal{N}(i)$ is a function that returns the set of neighboring flows for flow i . Afterward, a message passing process is executed for K iterations (lines 6-9). First, the model generates flow-level aggregate messages m_f by combining the neighbors' flow measurement hidden states h_f with the corresponding attention coefficients α_{ij} (line 8). Then, flow hidden states h_f are updated along with their aggregated messages m_f by using the U function (line 9), which is implemented by a NN. After the message passing process, the final hidden state of the target flow h_f^K is passed through the Readout function R_f , which produces a traffic estimate for the next time step $\hat{f}^{t+1}$ (line 10).

5.3.3 Anomaly detector

Algorithm 3 GAT-AD anomaly scoring

Input: $F, \hat{F}$

- 1: **for** $f \in F, \hat{f} \in \hat{F}$ **do**
- 2: $Err_{pred} \leftarrow |f_i^{t+1} - \hat{f}_i^{t+1}|$
- 3: $MAE_{tr} \leftarrow MAE_{tr}^f$
- 4: $score_f \leftarrow \frac{Err_{pred}}{MAE_{tr} \times \delta}$
- 5: **Return** $[score_0, \dots, score_M]$

Lastly, the anomaly detector generates an anomaly score at the OD flow level (see Figure 5.1-C). Algorithm 3 shows the anomaly scoring method, which is based on computing differences between the traffic estimates of the contextual predictor $\hat{F}$ and the real traffic values measured at $t+1$. We use a configurable detection threshold δ to tune the volume of anomalies generated. For each flow, we compute the prediction error Err_{pred} (line 2). Then, we pick the MAE of the contextual predictor for flow f (i.e., MAE_{tr}^f), which was stored during training (Section 5.3.2). The MAE_{tr}^f values are used as normalization parameters to calibrate the detection sensitivity across OD flows. Finally, we produce flow-level anomaly scores by combining the prediction error Err_{pred} , the detection threshold δ , and the calibration value MAE_{tr}^f (line 4). All samples with an anomaly score > 1 are then considered anomalous.

5.4 GAT-AD+

Building upon GAT-AD, GAT-AD+ introduces several enhancements aimed at improving computational efficiency, robustness, and interpretability. The main improvements in GAT-AD+ include: (i) the ability to simultaneously model all nodes in the graph, enabling it to capture more complex correlations, (ii) eliminating the need for positional encoding to specify the target node, thus simplifying the model structure, and, (iii) using the raw OD flow measurements directly rather than relying on embeddings in the hidden space, significantly enhancing the interpretability of the contextual relationships identified by the model.

Figure 5.2 shows an overview of GAT-AD+. Similarly to GAT-AD, it is divided into three main blocks:

- **Data representation:** Converts traffic time series into graph-structured representations, where each node in the graph represents traffic measurements aggregated at the OD flow level. It creates a graph for every time sample t , and each node in the graph encodes the traffic measurements of an OD flow over a specific time window $[t - W, t]$.
- **Contextual predictor:** This module is implemented by a custom neural network model based on GATs, which uses a simplified architecture that is computationally more efficient and offers greater interpretability. The

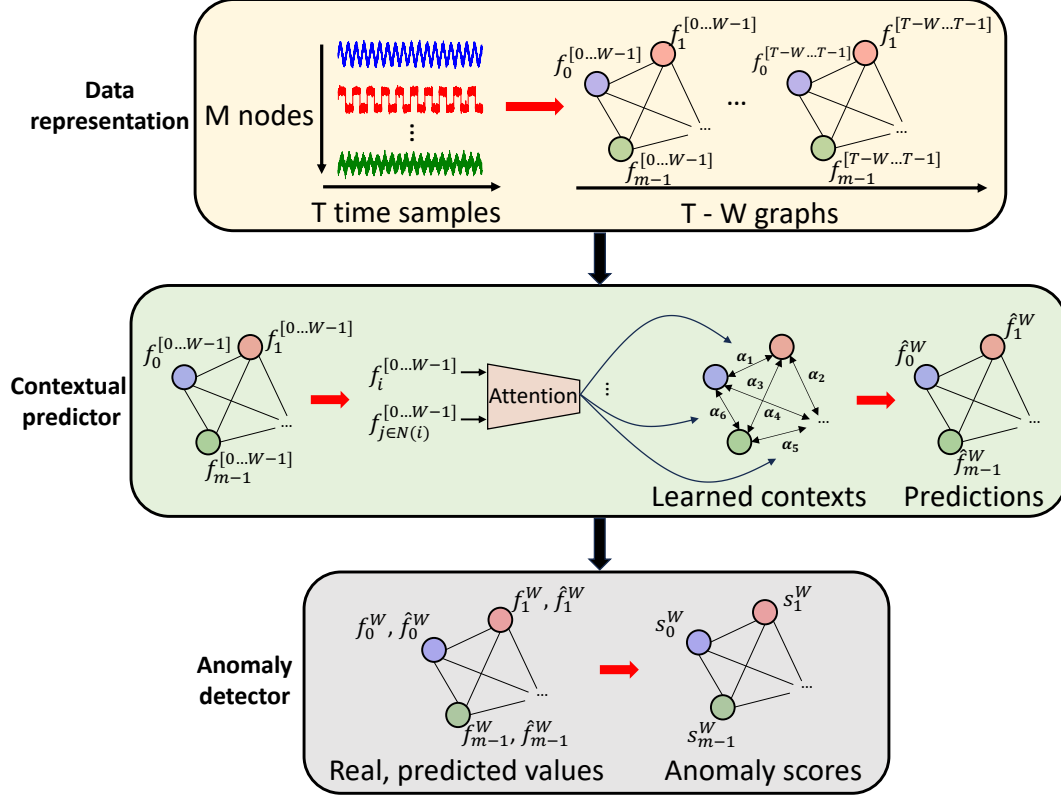


FIGURE 5.2: Overview of GAT-AD+.

model learns how to build the context of OD flows by looking at their temporal evolution over a specific time window $[t - w, t]$. It is trained in a self-supervised manner to predict the expected OD flow traffic measurements at $t + 1$.

- **Anomaly detector:** Receives as input the real and the expected traffic time series (i.e., the OD flow measurements) and compares them to generate an anomaly score, which is used to classify samples as normal or anomalous.

We further describe the design of the three previous blocks in the next subsections.

5.4.1 Data representation

GAT-AD+ constructs graphs in a similar manner to GAT-AD, using OD flow traffic measurements arranged in a matrix $R^{M \times T}$, where M is the number of OD flows and T denotes the number of temporal measurements. Then, it processes sliding temporal windows of size W , spanning $[t - W, \dots, t]$, to form a graph where each node f_i represents an OD flow aggregated over the window. Unlike GAT-AD, GAT-AD+ does not require positional encodings to uniquely identify nodes, nor does it use one-hot encodings to indicate a target flow, since it predicts traffic for all flows simultaneously. As before, all node pairs (f_i, f_j) are initially connected with directed edges e_{ij} , enabling

the model to learn asymmetric, direction-aware relationships through a contextual predictor that dynamically refines edge weights to capture nuanced inter-flow dependencies.

5.4.2 Contextual predictor

For the new contextual predictor, we design a custom model based on GATs that produces pairwise attention coefficients, which in our case can be interpreted as the probability of two OD flows to be in the same context. Contrary to GAT-AD’s contextual predictor, this version uses the raw OD flow values instead of converting them to hidden states, which allows to generate more interpretable contexts. Furthermore, by predicting all of the OD flows at the same time, the positional encoding and one-hot encodings can be removed, which makes the model simpler and more efficient computationally. A relevant aspect of our approach is that contexts are not pre-filtered in advance using a heuristic, as it occurs in GDN [32]. Instead, the model defines directly attention coefficients over the entire set of OD flows, allowing it to learn the optimal similarity metric according to the specific problem to be addressed.

The design of GAT-AD+ is motivated by the following:

- **Robust contextual prediction:** We design our model to emphasize the use of contextual information to predict the OD flow measurements. We achieve this by learning the contexts of each OD flow through the attention module, which generates the α attention coefficients. We employ these coefficients to weight the information that each OD flow f_i receives from its neighbors $f_j \in \mathcal{N}(i)$, where $\mathcal{N}(i)$ contains all the OD flows f_j where there exists an edge $e_{i,j}$.
- **Interpretability:** We use the actual neighbor OD flows measurements to predict the target OD flow’s measurements, rather than transforming the measurements into latent representations, as in GATs and other popular GNN models [156]. This makes it possible to directly interpret attention coefficients as weights in a linear combination, which allows to analyze the context generated by the model to make the predictions (e.g., what are the most relevant flows in the context captured by the model).

DL models, such as GNNs, enable the modeling of complex relationships underlying the data, but at the cost of interpretability, as their decision-making processes are often opaque [73]. On the other hand, we designed GAT-AD+ to make it as simple as possible. It receives as input OD flow measurements over a time window $[t - w, t]$, represented in a single graph, and calculates the α coefficients representing the contexts in time $t + 1$. To generate the α coefficients, the attention module receives all pairs of OD flows $(f_i, f_j), \forall f_j \in \mathcal{N}(i)$ (Figure 5.3). Our attention module includes a configurable parameter of temperature τ that defines the sharpness of the attention distribution [66]. The attention module correlates every pair of OD flows to determine the strength of their relationship, which allows to generate contexts from these correlations. Then, we use these learned

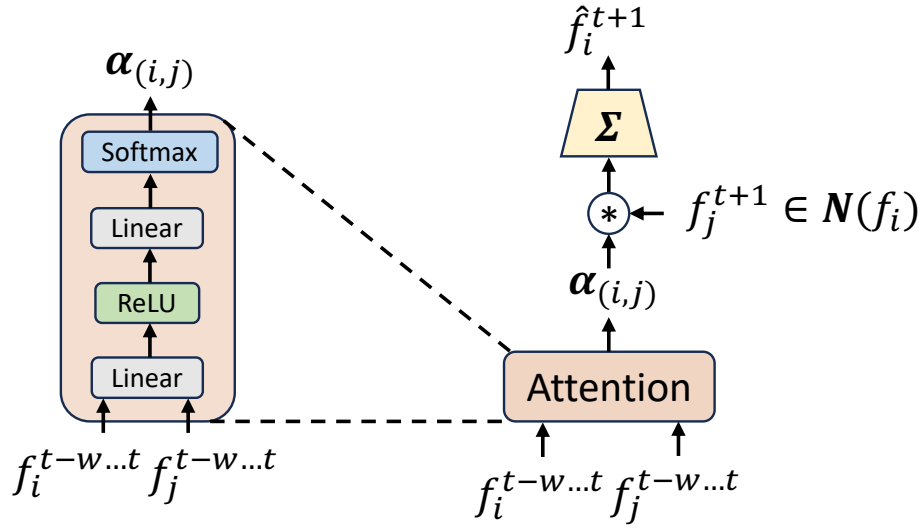


FIGURE 5.3: Contextual predictor overview.

contexts to predict the OD flow's next measurements as $\hat{f}_i = \sum_{j \in \mathcal{N}(i)} (\alpha_{i,j} \cdot f_j$ (Algorithm 4).

Algorithm 4 GAT-AD+ contextual predictor

Input: F

- 1: **for** $f_i \in F, f_j \in \mathcal{N}(i)$ **do**
 - 2: $\alpha_{i,j} \leftarrow \text{Attention}([f_i^{t-W \dots t-1}, f_j^{t-W \dots t-1}])$
 - 3: **for** $f_i \in N$ **do**
 - 4: $\hat{f}_i^t = \sum_{j \in \mathcal{N}(i)} (\alpha_{i,j} \cdot f_j^t)$
-

5.4.3 Anomaly detector

Finally, the anomaly detector is responsible for generating the anomaly scores. Algorithm 5 describes the method for generating the anomaly scores from both the ground truth OD flow measurements $F^{M \times T}$ and the predicted OD flow measurements $\hat{F}^{M \times T}$. Similarly to how the score is calculated in GAT-AD, the algorithm starts by calculating the absolute error (err) between the two values and then uses the interquantile range iqr of the errors (i.e., the difference between quartiles Q3 and Q1) to normalize them, as well as an ϵ value, as in [32]. Finally, we apply a smoothing to the scores by calculating the moving average over a window of size δ .

The variable $score^{M \times T}$ represents the anomaly score at an OD flow level, that we can convert to a global graph-level score $score^{1 \times T}$ by adding up the top K OD flow scores. Then, every sample exceeding a fixed threshold $thresh$ is considered anomalous.

Algorithm 5 GAT-AD+ anomaly scoring

Input: $F^{M \times T}, \hat{F}^{M \times T}$

- 1: $err \leftarrow |F - \hat{F}|$
- 2: $iqr \leftarrow Q3(err) - Q1(err)$
- 3: $score^{M \times T} \leftarrow \frac{err - \text{median}(err)}{|iqr| + \epsilon}$
- 4: $score^{1 \times T} \leftarrow \sum_{m \in M} \text{topK}(score_m)$

5.5 Evaluation

In this section, we evaluate GAT-AD+ and compare it against state-of-the-art baselines to answer the following questions:

- How does GAT-AD+ compare to state of the art solutions for AD? In Section 5.5.2, we compare GAT-AD+ with two state-of-the-art solutions based on graph representations and apply them to a widely used dataset for AD in time series that is reliably labeled (WaDi [2]).
- How do the detections overlap in comparison with the best baseline solution? What anomalies does each proposal detect? (Section 5.5.3).
- How does GAT-AD+ perform for the task of AD in network traffic? We take as a reference a well-known dataset from the Abilene network [173], which includes traffic matrices with real-world traffic aggregated at the OD flow level. Since this dataset does not contain labeled anomalies, we inject synthetically generated anomalies to compare GAT-AD+ with the other baselines (Section 5.5.4).
- How does the custom design of the contextual predictor of GAT-AD+ affect the overall performance? In Section 5.5.5, we compare our model with other baselines that use standard GNN architectures (Message-Passing Neural Networks, and Graph Attention Networks).
- How do hyperparameters affect the performance of GAT-AD+? In Section 5.5.6, we perform a sensitivity analysis looking at two parameters of our solution (window size and temperature).
- Few-shot learning: how does the performance of GAT-AD+ degrade when we reduce the size of the training dataset? In Section 5.5.7 we train GAT-AD+ with only 10% of the training dataset and compare the results in terms of AD precision, recall, and F1-score.

5.5.1 Methodology and Datasets

Baselines

To benchmark our solution, we select a state-of-the-art proposal for AD in time series and GAT-AD, which we consider the closest to GAT-AD+ given that they are also based on graph representations and GNNs.

1. GDN [32]: This method obtained state-of-the-art results in the WaDi dataset. GDN uses a GAT-based model that includes information of the past values of graph nodes to predict their future values.
2. GAT-AD: The initial proposal on which GAT-AD+ is built. Contrary to GAT-AD+, it trains individual models to make predictions on different graph nodes and uses positional encodings to identify the different nodes.

Datasets

Even though the main scope of GAT-AD+ is AD in network traffic, we could not find any publicly available dataset with annotated anomalies in flow-level traffic time series. Hence, we use two different datasets for our evaluation: (i) Internet2's Abilene, where we inject synthetically-generated anomalies, and (ii) WaDi, which is widely used in AD in time series data.

1. Abilene [173] is a backbone network with 12 nodes, 15 links, and 132 OD flows. The measurements are collected at the OD flow granularity and sampled in 5-minute intervals, over 6 months. In total, the dataset contains 48,096 temporal samples per OD flow and no labels regarding the presence or not of anomalies in the data.
2. WaDi [2], provided by *iTrust, Centre for Research in Cyber Security, Singapore University of Technology and Design*, represents a water distribution system. This dataset contains 2 weeks of normal operation data, which is used as training, and several controlled attacks in the test data, whose labels are provided. Although water distribution systems are out of our core interest, WaDi provides reliably annotated anomalies in a scenario similar to the network-wide AD use case for which GAT-AD+ has been designed. Specifically, WaDi contains monitoring data from sensors – represented as time series – that are distributed in the water distribution system, hence forming a network of sensors. Accordingly, we use this dataset to make a fair comparison with the state-of-the-art baselines (Section 5.5.1) using reliably annotated anomalies.

We preprocess and use each dataset differently. In Abilene, we observed that the traffic measurements follow a heavy-tailed distribution, hence, we adopt a logarithm transform. We use the first 40% of the data as training samples, the following 10% as validation samples, and the remaining 50% as testing. On the other hand, in the WaDi dataset, we apply a preprocessing similar to [2]. We perform a min-max normalization. To reduce the training time, we downsample 1 sample every 10 seconds with the median value. Since the systems take 5 to 6 hours to reach a stable phase when they are turned on, we remove the first 2,160 samples [54].

Training

After a grid search optimization process, we train GAT-AD+ with a dimension space of 128, a Mean Absolute Error (MAE) loss function, a learning rate of 0.0001 in combination of an Adam optimizer, a window size W of

100 samples, a temperature τ of 0.1 for the attention module in both Abilene and WaDi and a score smoothing δ of size 5. To train GDN, we use the same attributes as those defined in [32] for WaDi and a window size of 10 for Abilene, while keeping the rest of the attributes unchanged. Lastly, to train GAT-AD, we use a learning rate of 0.001, an Adam optimizer, a hidden space of size 128, and a MAE loss function.

Evaluation metrics

Precision, recall and the F1-score are metrics typically used for evaluating the performance of classification models in machine learning, and are defined by the ratio of true positives (TP; the number of correctly identified anomalous samples), true negatives (TN; the number of correctly identified regular samples), false positives (FP; the number of regular samples incorrectly identified as anomalous) and false negatives (FN; the number of anomalous samples incorrectly identified as regular samples):

- **Precision:** Defined as the ratio of true positives to all predicted positives, $Precision = \frac{TP}{TP+FP}$, indicating the accuracy of the positive predictions.
- **Recall:** Defined as the ratio of true positives to all actual positives, $Recall = \frac{TP}{TP+FN}$, measuring the ability to detect all actual positives.
- **F1-score:** The harmonic mean of precision and recall, given by $F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$, providing a balanced measure of the performance by considering both false positives and false negatives.

While the accuracy metric is often relevant in classification problems, it can be misleading in cases of imbalanced datasets like ours, where anomalous samples are much less frequent than normal ones (Sections 5.5.2 and 5.5.4). In such datasets, accuracy may suggest high performance even if a model only performs well on the majority class (normal samples in our case) while missing many anomalous samples, which are the main focus of our evaluation. For this reason, we choose not to include the accuracy metric.

5.5.2 WaDi: Anomaly detection evaluation

As mentioned earlier, the lack of labeled datasets with anomalies in network traffic forces us to use a dataset from an unrelated area. In this study, we choose to use the WaDi dataset, which is well-suited for time series AD due to its lack of anomalies in the training data and the availability of labels for injected anomalies in testing. We begin our evaluation by studying how GAT-AD+ compares against two state-of-the-art solutions in AD.

Table 5.1 shows the classification results on the WaDi dataset of the three proposals. We perform the detection of each sample belonging to an anomaly independently of the rest (i.e., without applying point adjustment). At the time of this writing, GDN represents one of the state-of-the-art solutions for the WaDi dataset, containing 5.74% anomalous samples in total. As we can observe, GAT-AD+ significantly outperforms GDN in terms of F1-score

Model	Precision	Recall	F1
GAT-AD+	81.87%	45.33%	58.36%
GDN	82.13%	35.90%	49.96%
GAT-AD	98.71%	15.52%	26.82%

TABLE 5.1: Classification performance by GAT-AD+ and the baseline models.

(+8.4%), specifically it exhibits a considerably larger recall (+9.43%) at the cost of a slightly reduced precision (-0.26%). Moreover, it shows a clear advantage over GAT-AD, which shows a limited recall in the WaDi dataset (15.52%).

Alternatively, we use a different criterion. As explained in [78], considering the detections at a per-sample level may have some limitations. For instance, incorrectly classifying as anomalous the closest regular sample before an anomalous interval is penalized in the same way as doing so in a far-away sample. For this reason, and since anomalies tend to take place in contiguous anomalous intervals [11], we apply point adjustment, which considers an entire anomaly as detected if at least one of its samples is correctly detected, as it is current practice in the literature [11, 136, 160].

Model	Precision	Recall	F1
GAT-AD+	86.69%	65.42%	74.57%
GDN	86.79%	51.32%	64.50%
GAT-AD	98.74%	15.92%	27.42%

TABLE 5.2: Classification performance by GAT-AD+ and the baseline models with the new detection criterion.

Table 5.2 shows the results obtained by GAT-AD+ and the baseline methods when we use the new criterion: if at least one sample of the anomalous interval is detected, the entire interval is deemed detected. The results show that GAT-AD+ outperforms the best baseline method (GDN) both in terms of recall (+14.1%) and F1-score (+10.07%).

5.5.3 WaDi: Overlap analysis

After comparing the different baselines, we aim to analyze the overlap of the anomalies detected between them. Specifically, we compare GAT-AD+ with the best-performing baseline method from Section 5.5.2, which is GDN. We aim to understand which anomalies are detected only by GAT-AD+ and which ones are detected by both solutions. To do so, we perform both a quantitative and a qualitative analysis.

First, we categorize the 13 anomalies in WaDi based on their detection status by each solution. As Table 5.3 shows, GDN detects 7 anomalies out of 13, while GAT-AD+ detects all these anomalies plus two more that remained undetected by GDN. To further understand the overlap, we analyze individually the time samples labeled as anomalous by each method and find that

GAT-AD+ has an overlap of 52.75% with respect to the samples marked as anomalous by GDN. By applying point adjustment and considering one entire anomaly as detected if at least one of its samples is detected, the overlap of the two models increases up to 70.16%.

Anomaly ID	GAT-AD+	GDN
1	No	No
2	No	No
3	Yes	Yes
4	Yes	Yes
5	No	No
6	Yes	Yes
7	Yes	Yes
8	Yes	No
9	Yes	Yes
10	Yes	Yes
11	Yes	Yes
12	Yes	No
13	No	No

TABLE 5.3: Detected WaDi anomalies by GAT-AD+ and GDN.

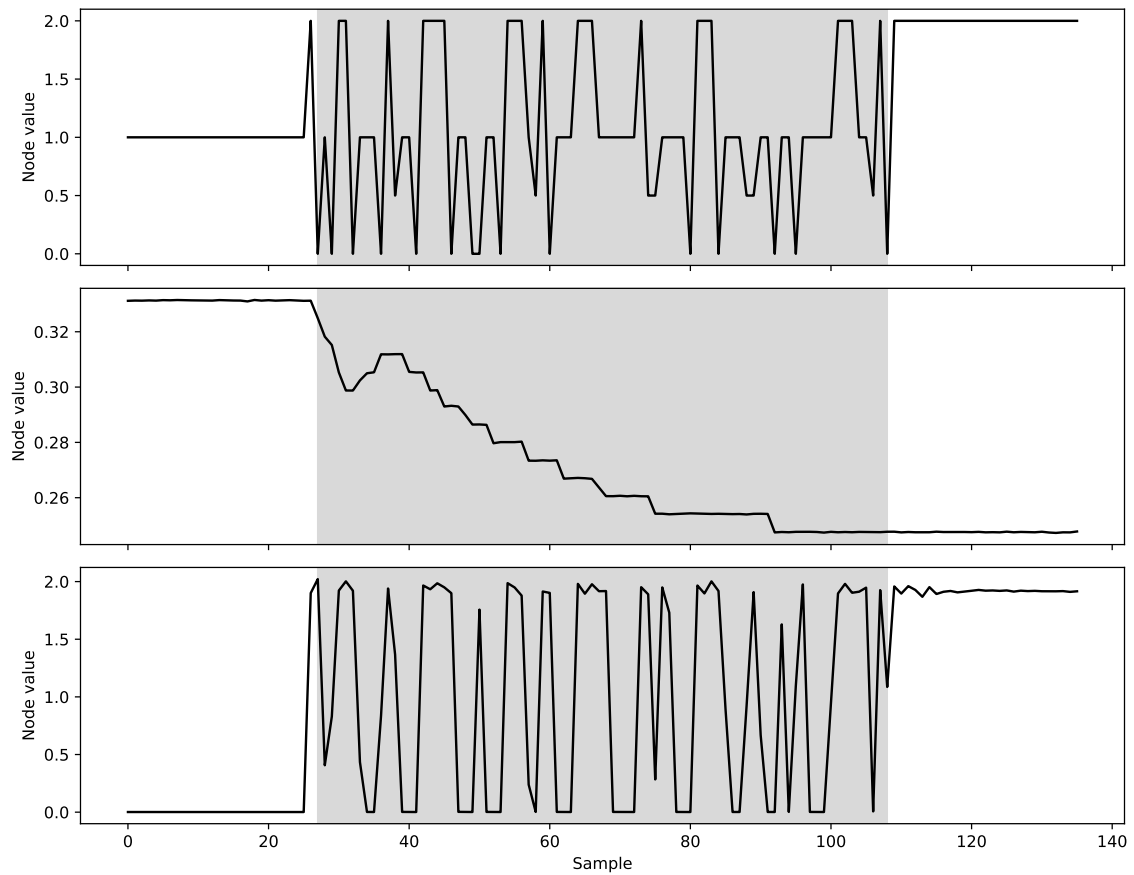


FIGURE 5.4: Anomaly #8, detected only by GAT-AD+.

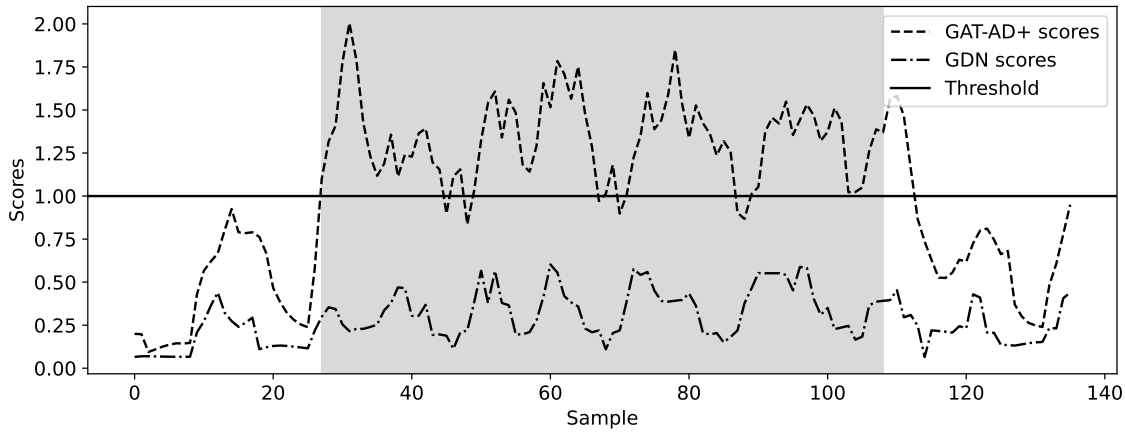


FIGURE 5.5: Anomaly scores generated during anomaly #8.

Figure 5.4 shows the changes in the nodes' time series experienced during the anomaly #8, which is one of those anomalies detected only by GAT-AD+. Figure 5.5 shows the anomaly scores generated by GAT-AD+ and GDN during this incident. Here we see that our method properly detects this attack during almost all its samples, while GDN's score remains always below the detection threshold.

In anomaly #8, a node in the system is damaged to cause an effect in other elements, which implies that a model harnessing contextual information may be more prone to detect this incident. Accordingly, the exploitation of contextual information by GAT-AD+ causes its anomaly score to spike during this incident.

5.5.4 Network traffic anomalies: Abilene dataset

In the previous experiments, we have used WaDi to evaluate the performance of GAT-AD+ and the baselines due to the lack of publicly labeled datasets for network traffic anomalies [3]. In this section, we use Abilene, a dataset with real-world traffic from a backbone network, where traffic measurements are aggregated at the OD flow level. Despite this dataset may contain traffic anomalies, it does not include any label with annotated incidents, thus we inject synthetic anomalies of two types and only take into account detections performed over these, ignoring the rest (since we cannot know whether they are true positives or not). The two types of anomalies are: (i) Type A, which involves a change in the behavior of an OD flow while its context remains unchanged, and (ii) Type B, which involves a change in the behavior of the context of an OD flow while it remains unchanged, thus making the target OD flow look inconsistent with respect with its context.

We begin by dividing the entire test dataset, composed of 24048 samples (Section 5.5.1), into 80 chunks of equal size. Each chunk consists of 300 time samples, which is three times the window size of 100 samples that the tested AD solutions use (see Section 5.5.1). This ensures enough buffer space

between injected anomalies, preventing the effects of one from influencing the next.

Then, we inject a Type A anomaly every four chunks starting from the second one (i.e., chunks 2, 6, 10, etc.) and a Type B anomaly also every four chunks, but starting from the fourth one (i.e., chunks 4, 8, 12, etc.), thus ensuring that at least one unmodified segment separates two modified ones. For every chunk, we select a random time interval of size $s \in [20, 80]$, and select a random magnitude $m \in [3, 6]$. We aim to generate a smooth and gradual increase in the original OD flow measurements that peaks at a maximum and then slowly decreases back. To do so, we generate a Gaussian-shaped function μ centered in the subinterval, which has minimum values of $m - 1$ and increases until it reaches its maximum value of $m + 1$ right in the middle of the time interval. Then, we apply the following procedure:

- **Type A anomalies:** We select a random OD flow f_i among the 132 that are present in Abilene. We increase its traffic according to the previous function μ during the chosen interval: $f_i^{[t...t+s]} = f_i^{[t...t+s]} \times \mu$. These effects may happen, for instance, due to a routing misconfiguration that forwards a huge amount of traffic to a specific network node.
- **Type B anomalies:** We select a random OD flow f_i among the 132 present in Abilene. We calculate the distances $d = d_1, d_2, \dots, d_n$ between f_i and every OD flow $f_j, j \neq i$ by using fast dynamic time warping [128]. Then, we define the context C_{f_i} of an OD flow as: $C_{f_i} = \{f_j \mid d_j < \frac{\mu_d}{2}\}$, where $\mu_d = \frac{1}{n} \sum_{k=1}^n d_k$. Then, we increase the traffic of all the OD flows in the context as $f_j^{[t...t+s]} = f_j^{[t...t+s]} \times \mu, \forall f_j \in C_i$. These effects may take place, for example, due to a Distributed Denial of Service (DDoS) affecting multiple network OD flows [91].

We train the contextual predictor of GAT-AD+ on the training dataset of Abilene (40% of the dataset); then, we inject 20 anomalies of each of the two types into the test dataset. The injected anomalies represent the 1.9% of all testing samples, even though since the dataset is unlabeled, we do not know if there are unlabeled anomalies in the data. In this case, since we detect anomalous OD flows in the network, we detect anomalies using node-level scores instead of graph-level scores, which implies not performing the last step in Algorithm 5.

Figure 5.6 shows the detection of the injected anomalies in Abilene. The vertical axis describes the recall of these anomalies, whereas the horizontal axis describes the threshold that defines how many samples are considered anomalous. Note that in this case, we cannot measure the precision of the methods due to the potential inclusion of traffic anomalies in the original data of Abilene, so we cannot differentiate whether detections outside the injected anomalies are correct (i.e., true positives) or not (i.e., false positives). The results show a gradual improvement in GAT-AD+ as the detection threshold is lower, especially up to the point where it marks 2% of samples, where it achieves a recall of $\approx 70\%$ over the injected anomalies, which is

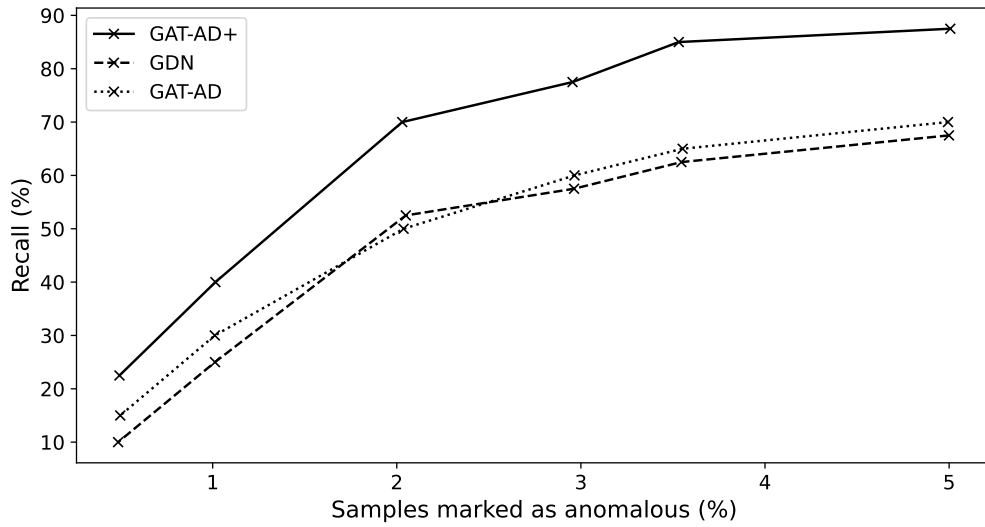


FIGURE 5.6: Recall vs number of marked samples.

$\approx 17.5\%$ higher than the best baseline. This figure gives an overview of each proposal's detection capabilities and allows to compare their effectiveness in identifying the injected anomalies.

Model	Type A	Type B
GAT-AD+	85 %	90 %
GDN	70 %	65 %
GAT-AD	90 %	50 %

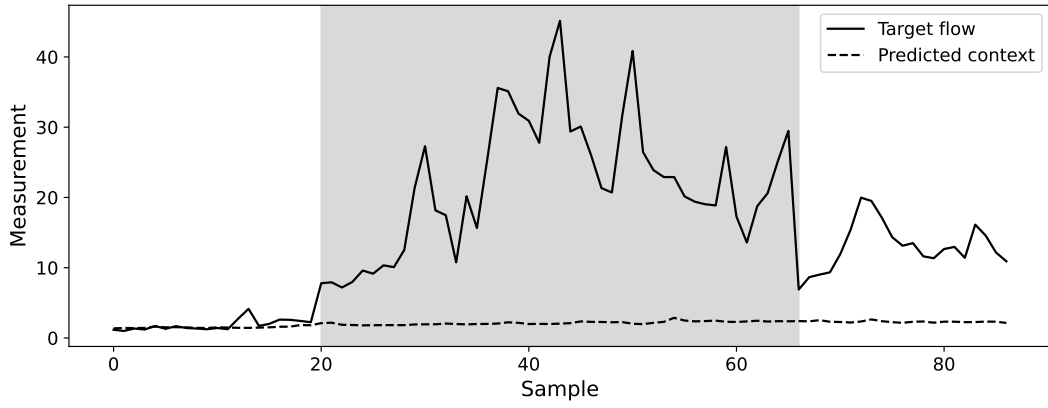
TABLE 5.4: Recall per type at 5% detection threshold.

Table 5.4 shows the recall per anomaly type on the 5% detection threshold (i.e., 5% of all samples marked as anomalous). The results show that GAT-AD+ has a high recall of both types of contextual anomalies and highlight its effectiveness in detecting these kinds of anomalies.

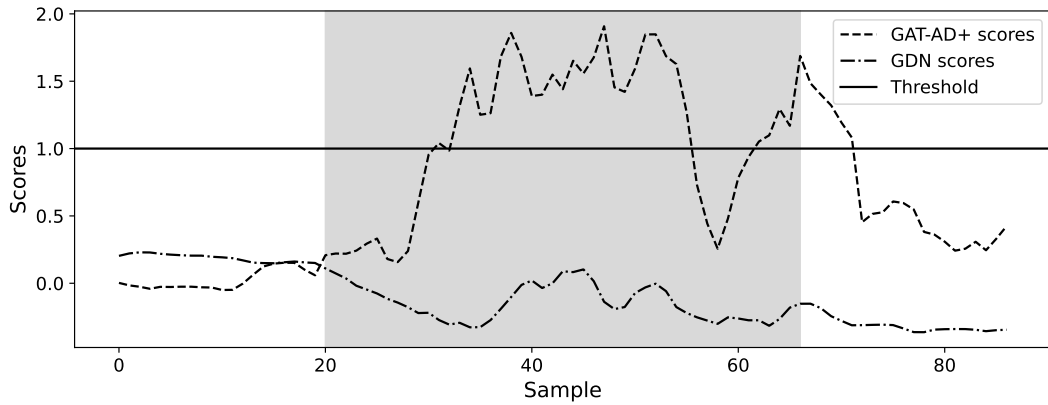
Next, we analyze a specific anomaly of each type. We injected the anomalies previously described into Abilene with a two-fold objective in mind:

- To verify that GAT-AD+'s predicted contexts adjust as expected when we modify the original contexts during anomalous periods.
- To induce a change in predictions, which in turn leads to a detection, since these predictions depend on contextual information.

To provide insights about this, we show the anomalies, the associated anomaly scores generated by GAT-AD+ and GDN, and the attention coefficients generated by GAT-AD+, which represent the context of the target OD flow.



(A) Target OD flow and predicted context measurements.



(B) Predicted scores.

FIGURE 5.7: Injected Type A anomaly and scores.

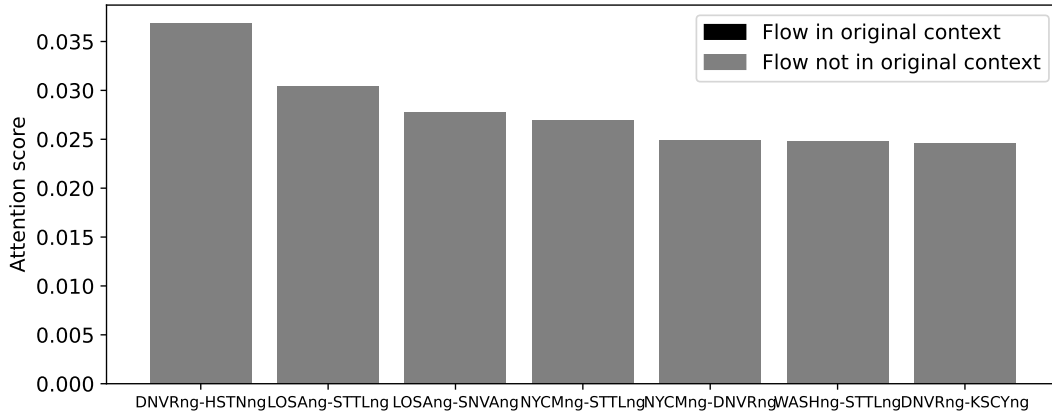
Figure 5.7a shows the target OD flow measurement and its predicted context measurement. The horizontal axis shows each time sample, whereas the vertical axis shows the OD flow and predicted context traffic measurements. We show the frozen initial context to better visualize how the modified target OD flow deviates from its original one. Figure 5.7b shows the scores generated by GAT-AD+ and by GDN on one of the Type A contextual anomalies that we injected. The horizontal axis describes the time interval samples, whereas the vertical axis describes the normalized score, where each sample with a score greater than 1 is considered anomalous. As the anomalous zone starts (in gray) and we amplify the target OD flow measurement, it deviates from the predicted initial context behavior (Figure 5.7a). This causes the anomaly score generated by GAT-AD+ to slowly increase (Figure 5.7b), something that happens due to the sliding window smoothing of the score (Section 5.4.3). This score starts to decrease as GAT-AD+ finds a new context that is more similar to the modified target OD flow.

Likewise, Figure 5.8 shows the attention coefficients generated by GAT-AD+ before and during the injected Type A anomaly. The horizontal axis of each subfigure shows the neighbor OD flows with the highest attention coefficients, while the vertical axis shows the values of the attention coefficient

that correlates the target OD flow with each neighbor. The color of the bars indicates whether the OD flow belongs to the target OD flow’s original context (in black) or if it is not (in gray), which are defined as explained before. Figure 5.8a shows the context before the anomaly starts, and is the context that we use to plot the context measurement in Figure 5.7a. As the target OD flow measurement is modified, GAT-AD+ modifies its context to better reflect its new behavior, as shown in Figure 5.8b. Furthermore, the new context does not include OD flows that were present in the initial context, since their behavior deviates from the modified target OD flow. This change in the predicted context improves the target OD flow prediction and is the reason behind the decrease in the anomaly score generated by GAT-AD+ before the anomaly ends (Figure 5.7b).



(A) Context before the injected Type A anomaly.

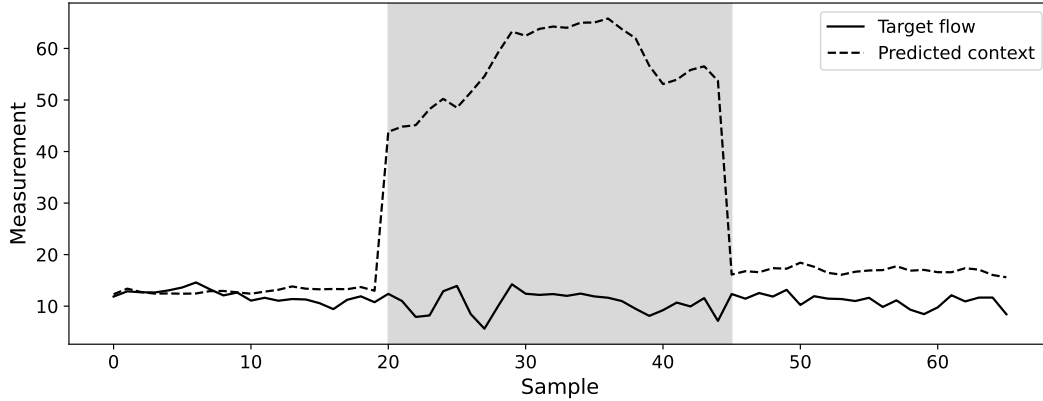


(B) Context during the injected Type A anomaly.

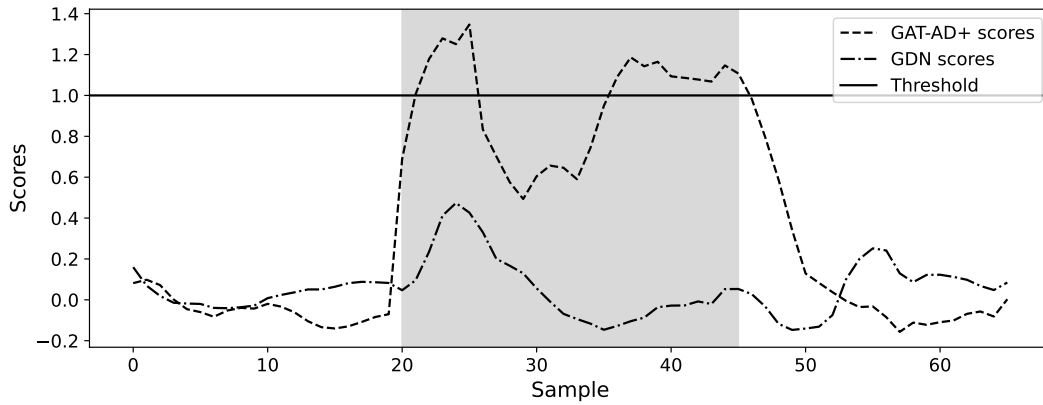
FIGURE 5.8: Contexts surrounding the injected Type A anomaly.

Now, we analyze a Type B injected anomaly. As in Figures 5.7a and 5.7b, respectively, Figure 5.9a shows the target OD flow and context measurements

and Figure 5.9b shows the anomaly scores generated by GAT-AD+ and GDN.



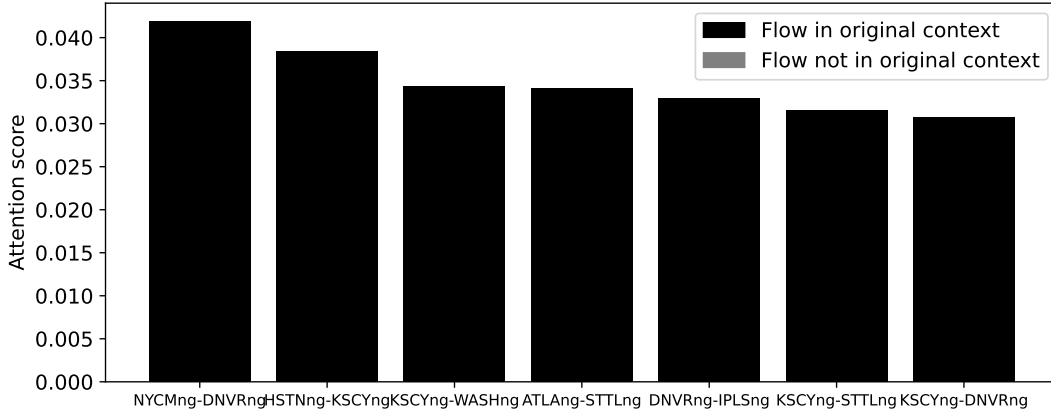
(A) Target OD flow and predicted context measurements.



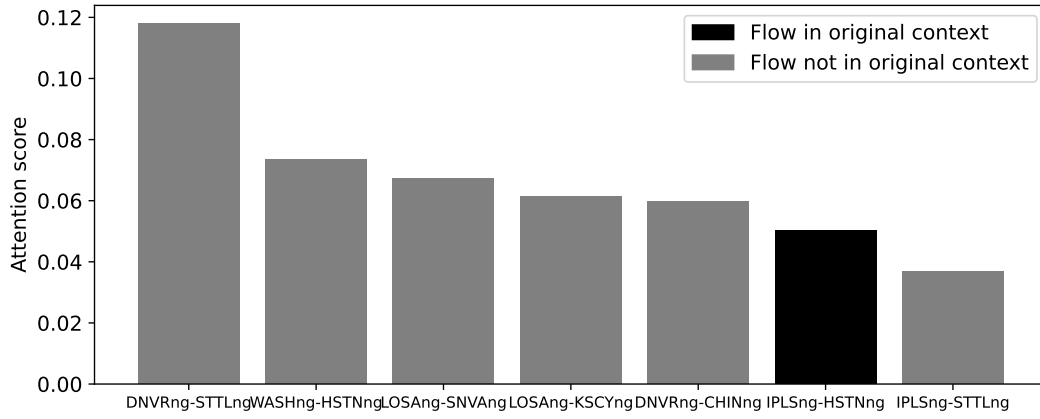
(B) Predicted scores.

FIGURE 5.9: Injected Type B anomaly and scores.

As explained previously, in Type B anomalies we modify the measurement of the OD flows in the context, as is reflected in Figure 5.9a. As before, the predicted context shown is the one identified before the anomaly starts, rather than the updated context detected by GAT-AD+ once the anomaly begins. Once this deviation starts (sample 20 in Figure 5.9a), the score generated by GAT-AD+ increases (Figure 5.9b). It then starts to decrease as our model finds a new context more similar to the target OD flow, but the score starts to increase again as the context measurement reaches a new peak (samples 30 to 40). Then, as the anomalous region ends and the initial context returns to its original behavior, the score decreases. This anomaly reflects the behavior that we expect, where a sudden change in a context behavior generates a peak in GAT-AD+'s anomaly score, which slowly decreases as the model adjusts to a new and more similar context.



(A) Context before the injected Type B anomaly.



(B) Context during the injected Type B anomaly.

FIGURE 5.10: Contexts surrounding the injected Type B anomaly.

5.5.5 Analysis of the contextual predictor

The design of the contextual predictor behind GAT-AD+ is motivated by the possibility of making the model more robust and interpretable (see Section 5.4.2). In this section, we compare the performance of the proposed model with two well-known GNN architectures that were widely used in the past for different tasks:

1. MPNN [53]: MPNNs are among the most expressive GNN models to date. These models can be described in three main steps: (i) an initialization function, in which the node (and optionally edge) features are transformed into fixed-size vectors (i.e., the hidden state), (ii) a message passing phase, in which the node's hidden states are exchanged with their neighbors during T iterations and, after each iteration, each node updates its hidden state based on the messages received from its neighbors by using a permutation invariant function, and (iii) a readout function, which takes as input the converged hidden states (i.e., the hidden states after

the T iterations of the message phase) and converts them to the desired output, either at a node-level or a graph-level. We use $T = 1$ and a *sum* aggregation function in this model.

2. GAT [150]: This model incorporates an attention mechanism to better model the relationships between nodes. Instead of weighting each received message uniformly, GATs allow to assign a different weight to each received message, allowing to learn the underlying relationship between every node pair in a graph.

Compared to the previous models, our contextual predictor (Section 5.4.2) takes directly the initial node features (i.e., the OD flow measurements) instead of generating hidden state representations from these. Then, it applies attention coefficients as a linear combination on OD flow measurements. Conversely, the other GNN models (MPNN and GAT) project node features into non-linear latent representations that cannot be directly interpreted.

To validate our design choices, we evaluate the performance of three solutions based on each model: MPNN, GAT, and GAT-AD+, on the WaDi dataset. We measure three main features of these models: (i) their performance in AD (precision, recall, and F1-score), (ii) the accuracy of their predictions over the OD flow time series, measured as the Mean Relative Error (MRE) of the predictions, and (iii) their training time over the test dataset. Table 5.5 shows the results. Overall, we observe that GAT-AD+ outperforms the two other models in all metrics. Moreover, although GAT obtains almost the same F1-score as GAT-AD+, the time necessary to train GAT-AD+ during the same number of epochs is $\approx 20\%$ faster. Also, in terms of time series predictions using the metric Mean Relative Error (MRE), GAT exhibits the worst performance, which hints that this model is less robust than the two others.

Beyond the previous quantitative analysis, it is worth noting that GAT-AD+'s contextual predictor is interpretable by design, due to its direct application of attention coefficients over the OD flow measurements, without creating latent representations of nodes' states. This enables us to interpret what flows were most relevant for the model to make the predictions at each time step (i.e., interpret the flows' context), as we did in Figures 5.8 and 5.10.

Model	Precision	Recall	F1	MRE	Training time
MPNN	51.70%	26.17%	34.75%	20.58%	53m 31s
GAT	81.78%	43.71%	56.97%	122.19%	1h 18m 30s
GAT-AD+	81.87%	45.33%	58.36%	18.80%	1h 2m 32s

TABLE 5.5: Classification performance metrics and execution time on the WaDi dataset.

5.5.6 Sensitivity analysis: window size and temperature

AD solutions often comprise a wide set of configurable parameters that have some impact on their final performance. To understand the sensitivity of our

solution to its configurable parameters, we select two core features: (i) the window size W , and (ii) the temperature τ of the attention mechanism. We evaluate these parameters via grid search, by testing different combinations on the WaDi dataset.

Window size	Temperature	Precision	Recall	F1
100	1	76.88%	43.51%	55.57%
100	0.1	81.87%	45.33%	58.36%
200	1	64.27%	40.87%	49.97%
200	0.1	64.65%	42.29%	51.13%
400	1	73.52%	32.66%	45.22%
400	0.1	49.57%	41.28%	45.05%
800	1	93.41%	30.58%	46.07%
800	0.1	69.25%	42.93%	53.00%

TABLE 5.6: Performance metrics for different window sizes with and without temporal data.

Table 5.6 shows the results. In our case, we focus on maximizing the F1-score, where the combination of window size $W=100$ and temperature $\tau=0.1$ yields the best results. Thus, we used these values for all the experiments in the previous subsections. We observed that the behavior of these parameters was consistent across both the WaDi and the Abilene datasets.

5.5.7 Few-shot learning

In this section, we aim to analyze the robustness and generalization capabilities of GAT-AD+ when only a small set of training samples is available (i.e., few-shot learning). For this purpose, we perform AD using the same methodology as we previously used with the WaDi dataset, but we train the models using only the first 10% of the training dataset.

Model	Precision	Recall	F1
GAT-AD+	73.95%	39.45%	51.46%
GDN	84.79%	33.37%	47.89%
GAT-AD	99.21%	12.68%	22.48%

TABLE 5.7: Performance metrics of models with limited training data.

Table 5.7 shows the results obtained after training all models with the reduced dataset. Since this table shows the results at a sample-level (i.e., we do not consider that one detected sample implies that the entire anomalous interval is detected), the results should be compared against Table 5.1. The results show that GAT-AD+ still outperforms the best baseline (GDN) in recall (+6.08%) and F1-score (+3.57%). Compared to the results with the full training samples, GAT-AD+ has worsened its performance in all metrics when

using fewer training samples: the precision, the recall and the F1-score decrease -7.92%, -5.88% and -6.9%, respectively. Nonetheless, the results show that GAT-AD+ is still able of keeping a proper performance when using only 10% of all available samples for training.

5.6 Summary of Outcomes

This chapter presents GAT-AD, an initial version of our model, and GAT-AD+, a novel solution based on GATs for network traffic AD. We compared GAT-AD+ against GAT-AD and a state-of-the-art graph-based method and evaluated their performance on two datasets: Abilene, which contains real-world traffic data, and WaDi, a water distribution dataset. Our results demonstrate that GAT-AD+ substantially improves detection rates on both datasets. In Abilene, our proposal identifies more injected anomalies and obtains a recall of $\approx 70\%$, outperforming the best baseline by $\approx 17.5\%$, largely due to its ability to capture contextual anomalies by analyzing temporal patterns across OD flows. Additionally, on the WaDi dataset — a widely used benchmark for time series AD — GAT-AD+ detects up to 14.1% more anomalies and achieves a F1-score 10.07% higher than the best baseline. The results on both datasets highlight that GAT-AD+ outperforms the two baselines by effectively harnessing contextual information, which is achieved thanks to the attention module, which constructs contexts through a linear combination of neighbor OD flow measurements, instead of relying on non-linear latent spaces. As future work, we consider testing our model in transfer learning scenarios to study how the attention module can create contexts based on time series learned from different datasets.

Chapter 6

Spatiotemporal Zero-shot Forecasting for Network Monitoring

6.1 Introduction

In this chapter, we tackle the challenge of generalization in MTS forecasting for network monitoring applications. Traditional DL models often require extensive retraining on domain-specific data, making them impractical for dynamic and heterogeneous network environments. Our goal is to develop a model that can generalize across different domains without requiring fine-tuning or additional training, enabling accurate forecasting under zero-shot conditions.

Zero-shot learning has emerged as a promising approach to enhance model generalization by leveraging pretraining over large, diverse datasets [13]. Inspired by this paradigm, we propose the Time Series Graph Foundation Model (TSGFM), a spatiotemporal GNN specifically designed for zero-shot MTS forecasting. TSGFM models each time series as a node and captures complex spatial and temporal dependencies through an attention-based graph structure, connecting nodes both across time and space. This design allows TSGFM to learn generalizable representations that can be transferred to unseen network monitoring scenarios, where obtaining labeled training data is often challenging.

Unlike traditional MTS forecasting models that rely heavily on retraining, TSGFM is pretrained on diverse time series from multiple domains (including energy, finance, and telecommunications) and deployed directly on new, unseen network data without further fine-tuning. This approach enables efficient and scalable forecasting in operational environments, overcoming the traditional barriers of dataset-specific adaptation.

To assess TSGFM performance, we conduct an extensive evaluation using five publicly available datasets spanning various domains, including two datasets from the energy sector [176, 65], a stock-market finance dataset [90], and two datasets from both fixed-line [174] and mobile networks [49]. These

datasets feature a broad variety of time series exhibiting different patterns and interdependencies, making them ideal for assessing TSGFM’s capability to generalize and adapt across domains. We compare TSGFM with a broad selection of state-of-the-art deep-learning models used as baselines, including more traditional neural architectures as well as advanced graph-based architectures. Specifically, our evaluation includes: (i) a RNN-based model composed of a RNN followed by a fully-connected layer, which makes it ideal for capturing temporal correlations, (ii) FreTS [168], (iii) a basic GNN model, implemented with a RNN that encodes the window values of each sliding window and then shares it by using a full-mesh graph, (iv) FourierGNN [167], (v) GAT-AD, which leverages contextual information to perform the forecasting – we overview FreTS and FourierGNN in Section 6.2.2. We perform zero-shot testing and cross-validation, training models on a collection of datasets and testing them on an unseen dataset. TSGFM achieves superior performance in six out of seven zero-shot testing scenarios, outperforming all competing models by at least 18% in zero-shot network monitoring, and maintaining a simple and interpretable graph representation. Furthermore, our analysis of the spatial attention coefficients (Section 6.4.3) reveals that TSGFM leverages its attention mechanism to selectively focus on a reduced set of meaningful edges. This suggests that a full-mesh spatial representation is not needed to obtain high forecasting performance, particularly in network monitoring scenarios, where we show that spatial attention is highly concentrated in a few edges.

The main contributions of this chapter are:

- We propose TSGFM, a novel spatio-temporal GNN architecture designed for zero-shot MTS forecasting, which learns transferable representations without requiring retraining on target domains.
- We design an interpretable spatiotemporal graph structure that explicitly models both spatial and temporal dependencies through attention mechanisms.
- We conduct extensive zero-shot evaluations across five datasets from different domains, demonstrating that TSGFM outperforms strong baselines in both network monitoring and other MTS forecasting tasks.
- We provide a detailed analysis of the attention patterns learned by TSGFM, highlighting its ability to focus on the most relevant spatial relationships, especially in network scenarios.

The remainder of this chapter is structured as follows: Section 6.3 details the architecture and methodology of TSGFM. Section 6.4 presents the experimental evaluation, performance analysis, and insights into model behavior. Finally, Section 6.5 concludes the chapter and outlines directions for future work.

6.2 Related Work

As discussed in Section 5.5.4, one of the main obstacles in AD lies in the scarcity of labeled network monitoring datasets that faithfully represent the diversity and complexity of real-world network environments. A promising direction to mitigate this issue is the use of foundation models (i.e., large, pre-trained models trained on broad data sources) which can be updated to specific tasks with minimal supervision.

In this section, we provide a comprehensive overview of the state of the art in foundation models applied to network data forecasting. The review is organized into two main parts: (i) foundation models for computer network forecasting, and (ii) spatiotemporal foundation models, with a particular focus on GNNs and their applicability to network data.

Forecasting tasks in computer networks typically involve predicting metrics such as the traffic volume. Early approaches used classical statistical models which, while effective for linear trends, often struggle with the non-linear behavior of network traffic. More recently, DL models have shown superior performance by capturing these complex temporal dynamics.

In parallel, foundation models for time-series forecasting have emerged as a powerful approach. These models are trained on massive and diverse datasets, enabling them to capture a wide range of temporal behaviors and generalize to unseen scenarios without task-specific retraining, which makes them particularly attractive in contexts where labeled data is scarce or where training from scratch would be computationally prohibitive. There is a recent surge in papers targeting the conception of foundation models for time-series data [101], capable of generating accurate predictions for diverse datasets not seen during training. The underlying concept of these models is to rely on highly expressive, large-scale architectures which are trained on millions or billions of time-series data points, coming from very diverse domains and having high heterogeneity in terms of temporal behaviors and characteristics. TimeGPT-1 [50], PromptCast [164], LLM-Time [58], TimesFM [29], Lag-Llama [120], Time-LLM [82], and FAE [55] are all examples of novel foundation models for time-series forecasting, which target a zero-shot learning application. The main limitation of foundation models for time-series forecasting, particularly in the context of network monitoring, lies in their underlying hardware requirements. Due to their extensive pretraining and large-scale architectures, these models often demand significant computational resources, including high-memory GPUs and large-scale distributed systems, making them impractical for deployment in resource-constrained environments.

6.2.1 DL Approaches to Foundation Models

Among DL approaches, RNNs (and, particularly, LSTMs) have gained prominence due to their ability to model long-term temporal dependencies. These models have been successfully used to forecast cellular base station

traffic, outperforming traditional techniques [154]. In another example, the authors of [114] propose using RNNs to forecast weather temperatures and validate their approach on a dataset containing hourly weather data from five cities in China. RNNs have some limitations such as vanishing and exploding gradients [116]. To address these issues, more advanced architectures such as GRU have been introduced. For example, [142] proposes using a GRU-based architecture to perform MTS forecasting with missing values. However, RNN-based methods such as LSTM face limitations when modeling extremely long sequences due to vanishing gradients. Moreover, they often neglect spatial dependencies, which are crucial in network environments where traffic patterns are influenced by the interactions between the different network elements.

Interestingly, there are some proposals for time series forecasting based on MLP, which are simpler models not specifically designed to work with sequential data. In [168], the authors argue that methods based on MLPs, which are simpler and lower in complexity, can be effectively applied in combination with data transformed to the frequency domain to perform time series forecasting. In their evaluation, they show that an architecture based on MLPs is indeed capable of outperforming a set of state-of-the-art baselines in both short-term and long-term forecasting.

Recently, transformer-based models have been explored for network traffic forecasting due to their strong ability to capture long-range temporal dependencies [176]. While these models are well-suited for handling long input sequences and complex temporal patterns, they lack dedicated mechanisms to model spatial correlations between network elements, which limits their effectiveness in scenarios where the spatial structure plays an important role in the behavior of the elements.

Foundation models trained on broad, generic time-series data have been proposed as a way to overcome the limitations of task-specific models and reduce the need of labeled data. For instance, [30] is pre-trained on more than 100 billion time points and achieves strong zero-shot performance across diverse forecasting tasks. However, this proposal is designed for univariate time series and does not incorporate spatial relationships between variables. This may cause the model to miss important information when modeling network data.

6.2.2 Spatiotemporal Foundation Models

Spatiotemporal models aim to capture both the spatial and temporal correlations underlying in the data. In computer networks, each node (e.g., a router) forms a topology, and forecasting tasks benefit from understanding how the behavior of one node affects its neighbors.

[56] presents a transformer-based model able of modeling spatiotemporal

correlations by representing each variable at each timestep as a separate input token. This allows the model to learn dynamic spatial and temporal correlations without relying on predefined graphs. Even though this proposal achieves promising results, it introduces a key computational drawback: representing the input as dense spatiotemporal sequences causes the sequence length to grow, which leads to a quadratic growth in memory usage. As a result, scaling to high-dimensional or long sliding window sizes becomes challenging.

GNNs have emerged as a primary tool to encode such non-Euclidean spatial structures. One of the earliest models in this area is the Spatio-Temporal Graph Convolutional Network (STGCN) [170], which replaces recurrent structures with convolutional layers to enable faster training. However, the model relies on a predefined static graph structure, which limits its flexibility in scenarios where spatial relationships evolve over time. Other architectures, such as [59], propose Attention-based Spatiotemporal Graph Convolutional Networks (ASTGCN) that enhance forecasting performance by dynamically learning which spatial and temporal dependencies are most relevant.

Following the early success of [170, 59], several models were proposed to further address the challenges of scalability and dynamic graph learning. Graph Wavenet [158] introduced an adaptive adjacency matrix learned through node embeddings, enabling the model to capture hidden spatial dependencies that are not explicitly defined by a static topology. On the other hand, [157] further enhanced this idea by jointly learning both the graph structure and the forecasting model.

[22] introduced one of the first attempts to incorporate Discrete Fourier Transforms (DFTs) into GNNs. They do so to model spatial and temporal correlations jointly in the spectral domain. Later on, [166] proposed modeling both spatial and temporal dependencies with a novel data structure, the hypervariate graph, where each timestamp-variable is treated as a node in a full-mesh graph (in an idea that resembles [56]). This representation allows the model to capture fine-grained spatiotemporal correlations. By using an operator that performs efficient graph convolutions in the Fourier space, this proposal achieves a lower computational cost while maintaining the expressiveness of the hypergraph representation. However, even though the Fourier transformations reduce the computational cost of the operations, the graph still grows rapidly in size with the size of the window or the number of nodes.

The rapid advance of foundation models has recently extended into the graph domain, giving rise to GFMs, which are large-scale, pre-trained models designed to generalize across diverse graph-structured tasks [104]. These models aim to capture complex patterns in the data through extensive pre-training. Within this broader landscape, a research direction that is growing in interest focuses on spatiotemporal foundation models, which must jointly model spatial and temporal correlations jointly. Spatiotemporal GFMs hold

particular promise for domains like traffic forecasting and network monitoring, where relational structure and time-dependent behaviors are deeply intertwined. However, the field remains in its early stages, with challenges yet unresolved such as the lack of large-scale and diverse pre-training datasets or the difficulty of unifying the spatial and temporal information.

6.3 Time Series Graph Foundation Model

TSGFM is a GNN-based model that incorporates spatial and temporal information from correlated time series to predict future values of each time series. To address the challenge of spatiotemporal time series forecasting, we propose a foundation model designed to capture complex dependencies across multiple datasets. TSGFM leverages graph-based representations to encode both spatial and temporal correlations between time series. Figure 6.1 shows an overview of TSGFM’s architecture and workflow for MTS forecasting. Sliding windowing for context, node embeddings for time-series, spatial and temporal attention, spatiotemporal merge, and readout stages are described next.

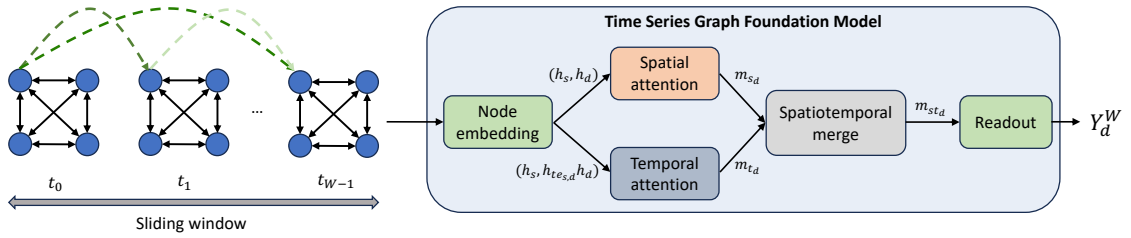


FIGURE 6.1: Overview of TSGFM.

6.3.1 Spatiotemporal Graph Data Representation

We represent each time series as a node within a graph, enabling the model to exploit the inherent relationships between time series from the same dataset. We construct fully-connected graphs for each dataset, and create spatial edges between each node pair from the same dataset to facilitate the propagation of spatial information between time series in the same dataset. When handling multiple datasets, we generate N fully-connected graphs for each time interval t . It is important to note that no spatial edges are created between two pairs of nodes of different datasets, ensuring that the spatial relationships are maintained independently within each dataset. Each node representing the time series n in the graph contains the value of that temporal series at instant t .

To capture temporal correlations, we introduce temporal edges that connect each node n at timestep t to itself at future timesteps t' , where $t' = t + \Delta t$ and $\Delta t \in \{1, 2, \dots\}$, forming edges of the form $(n^t, n^{t'})$, allowing the model to learn the evolution of each time series over time. Inspired by [159], we add

a feature Δt to every temporal edge, representing the relative temporal distance between the two nodes. An overview of this representation is shown in Figure 6.2.

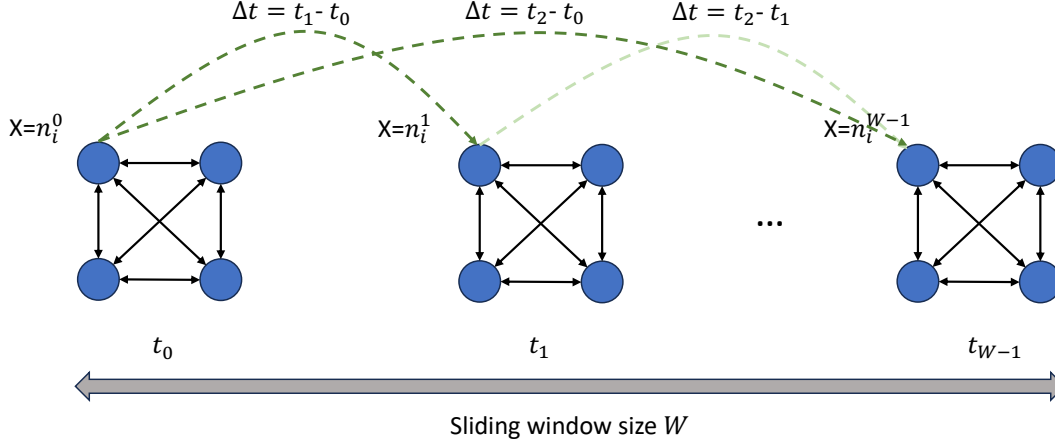


FIGURE 6.2: TSGFM's spatiotemporal graph representation.

We implement a sliding window of size W in the temporal dimension, defining the context of the model. For each timestep t , we generate N independent graphs – one per dataset – where each node contains the value of a specific time series at that time. As a result, the sliding window produces a total of $N \times W$ graphs, capturing both the spatial structure at each time step and the temporal progression across the windows. Nodes from different datasets remain also isolated in the temporal dimension: no spatial edges are created between them. Each graph remains fully connected within its dataset, allowing spatial information to propagate internally, while temporal edges link nodes to themselves in the future. This representation enables TSGFM to jointly model spatial correlations among time series in each dataset and temporal dependencies within each time series.

6.3.2 Underlying spatiotemporal GNN model

We implement a MPNN to harness the proposed data representation. We design a message passing composed of two phases executed in parallel: (i) a temporal message passing, and (ii) a spatial message passing. Let a node represent a time series at a specific timestep within the sliding window. We begin by projecting the node features into a hidden representation by using a MLP, resulting in an initial hidden state h_n for each node n . This is followed by T message passing iterations, where both spatial and temporal information is propagated. Each iteration consists of the two aforementioned phases.

In the temporal message passing phase, we generate an embedding for the temporal edges, denoted as h_{te} , using the Δt attribute, which represents the relative temporal distance between the source and destination nodes. For each temporal edge $te_{s,d}$ connecting the source node s to a destination node d , we assign an attention coefficient $\alpha_{te_{s,d}}$, computed as:

Dataset	Description
Spain	Electrical consumption, generation, pricing, and weather data from Spain
ETTh1	Operational data from two electricity substations, including transformer oil levels and temperature readings
Exchange	Daily exchange rates of eight foreign currencies from 1990 to 2016
TELCO	Twelve different time series with a five-minute granularity, representing mobile network measurements
Abilene	TM data collected from the Abilene backbone network

TABLE 6.1: Overview of the five publicly available datasets employed to evaluate TSGFM.

$$\alpha_{te_{s,d}} = \text{MLP}(h_s, h_{te_{s,d}}, h_d) \quad (6.1)$$

where h_s and h_d are the embeddings of the source and destination nodes, respectively, and $h_{te_{s,d}}$ is the temporal edge embedding derived from the relative time difference Δt between s and d . The attention coefficients $\alpha_{te_{s,d}}$ are then used to weight the incoming messages received by each node during the aggregation. Specifically, for each destination node d , we aggregate the incoming messages from its incoming temporal neighbors, $\mathcal{N}_t(d)$, by performing a weighted sum of the source node embedding:

$$m_{t_d} = \sum (s \in \mathcal{N}_t(d) \alpha_{te_{s,d}} \cdot h_s) \quad (6.2)$$

In parallel, we model spatial dependencies by assigning attention coefficients to spatial edges between nodes representing different time series at the same timestep. For each spatial edge $se_{(s,d)}$ we compute:

$$\alpha_{se_{s,d}} = \text{MLP}(h_s, h_d) \quad (6.3)$$

Where h_s and h_d are the embeddings of the source and destination nodes, respectively. As in the previous phase, the attention coefficients $\alpha_{se_{s,d}}$ are used to weight the incoming messages received by each node:

$$m_{s_d} = \sum (s \in \mathcal{N}_t(d) \alpha_{te_{s,d}} \cdot h_s) \quad (6.4)$$

Rather than updating the node embedding separately with the spatial and temporal messages, m_{s_d} and m_{t_d} , respectively, we first merge these messages using a *merge* function:

$$m_{st_d} = \text{merge}(m_{t_d}, m_{s_d}) \quad (6.5)$$

This merged message m_{st_d} integrates both temporal and spatial information, capturing the interactions across both dimensions. We then update the node embedding using an *update* function:

$$h'_d = \text{update}(h_d, m_{st_d}) \quad (6.6)$$

After T iterations, we use the hidden states of the nodes from the final timestep in the sliding window (i.e., $t = W - 1$) to produce the forecasting output for each time series at the next timestep using the *readout* function:

$$Y_d^W = \text{readout}(h'_d) \quad (6.7)$$

In essence, the readout layer takes the aggregated spatiotemporal information captured in the final hidden states within the sliding window and transforms it into the predicted values for the subsequent time point, completing the forecasting process for each time series.

6.4 Evaluation

In this section, we assess and benchmark the performance of TSGFM as a foundation model for MTS forecasting across different domains, further studying its internal workings through the analysis of its attention mechanisms.

6.4.1 Experimental Setup

Datasets

We evaluate TSGFM using five publicly available datasets spanning various domains, including two datasets from the energy sector [176, 65], a stock-market finance dataset [90], and two datasets from both fixed-line [174] and mobile networks [49]. Table 6.1 shows the datasets that we used in our evaluation, and provides a brief description of each of them. To evaluate the zero-shot performance and robustness of TSGFM to adapt to new domains, we generate all possible permutations of the datasets using a 4-to-1 split, where four datasets are used for training and one for testing in each permutation. Table 6.2 describes the five permutations. The training datasets are further split into 90% for training and 10% for validation. We apply min-max to normalize the data.

Permutation	Training	Testing
A	Exchange, Abilene, TELCO, ETTh1	Spain
B	TELCO, Exchange, Spain, ETTh1	Abilene
C	Exchange, Spain, Abilene, ETTh1	TELCO
D	Spain, Abilene, TELCO, ETTh1	Exchange
E	Exchange, Spain, Abilene, TELCO	ETTh1

TABLE 6.2: Dataset permutations used for training and testing.

Baseline models

We conduct a comprehensive comparison of TSGFM with a range of representative and state-of-the-art methods to evaluate its performance in MTS forecasting. The baseline models include simpler and more complex architectures to study how different complexities and design decisions influence the final results.

First, we compare against basic spatiotemporal architectures, such as a simple spatiotemporal GNN. This model initializes the hidden state of each node using a RNN, which encodes a sliding window of length W , and updates the node states iteratively through message passing with their neighbors.

We also evaluate against GAT-AD, a GAT-based model that incorporates dynamic attention mechanisms to capture spatiotemporal dependencies. Furthermore, we include FourierGNN [167], a GNN-based architecture that combines frequency domain transformations to efficiently capture spatial and temporal relationships.

For non-graph baselines, we include RNNs, which are well-suited for processing sequential data. These models maintain a hidden state that captures temporal correlations across timesteps, making them ideal for modeling temporal data. In our case, RNNs process sliding window sequences of size W , where each sequence consists of W consecutive samples of each time series. Furthermore, we compare with FreTS [168], which leverages frequency-domain transformations and simple MLP architectures, demonstrating that lower complexity models can also be effective for MTS forecasting.

The selection of baselines ensures a comprehensive evaluation of TSGFM against a variety of architectures, ranging from classical models to modern frequency-enhanced and graph-based approaches.

Experimental settings

All models use the same training, validation, and testing datasets, the same split sizes, and the same data normalization strategy. Specifically, we use z-score normalization in all datasets, and we use the mean and standard deviations from each dataset to normalize its data samples. Since we are working in a foundation scenario, where the models are supposed to be trained with

vast amounts of data, we cannot assume a training dataset distribution. For this reason, we also normalize the testing samples with the distribution of the testing data set. In all executions, we use a fixed seed $s = 42$ for the sake of reproducibility. All the experiments are conducted in Python using Pytorch 2.4 and a single NVIDIA RTX 3090 GPU. All the models are trained using the MAE loss function, a learning rate of $\eta = 0.001$, and we set $T = 1$ for TSGFM. We set up early stopping with a patience of 4 based on the training loss (i.e., the training stops after 4 epochs with no improvement).

Model \ Perm.	A		B		C		D		E		Avg.	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
RNN	1.23	1.13	1.62	1.04	0.99	1.00	1.25	1.29	0.97	0.94	1.52	1.05
FreTS	0.91	0.86	1.25	1.01	1.11	1.01	0.75	0.86	0.99	0.96	1.20	1.00
BasicSTGNN	1.32	1.08	2.16	1.12	1.02	0.99	1.75	1.57	0.98	0.95	1.96	1.11
FourierGNN	1.03	0.96	3.29	2.82	0.98	0.98	0.75	0.86	1.00	0.98	2.83	2.45
GAT-AD	0.98	0.82	1.32	1.22	1.40	1.17	4.75	5.29	2.00	2.12	1.34	1.22
TSGFM	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

TABLE 6.3: Zero-shot forecasting performance on all five dataset permutations.

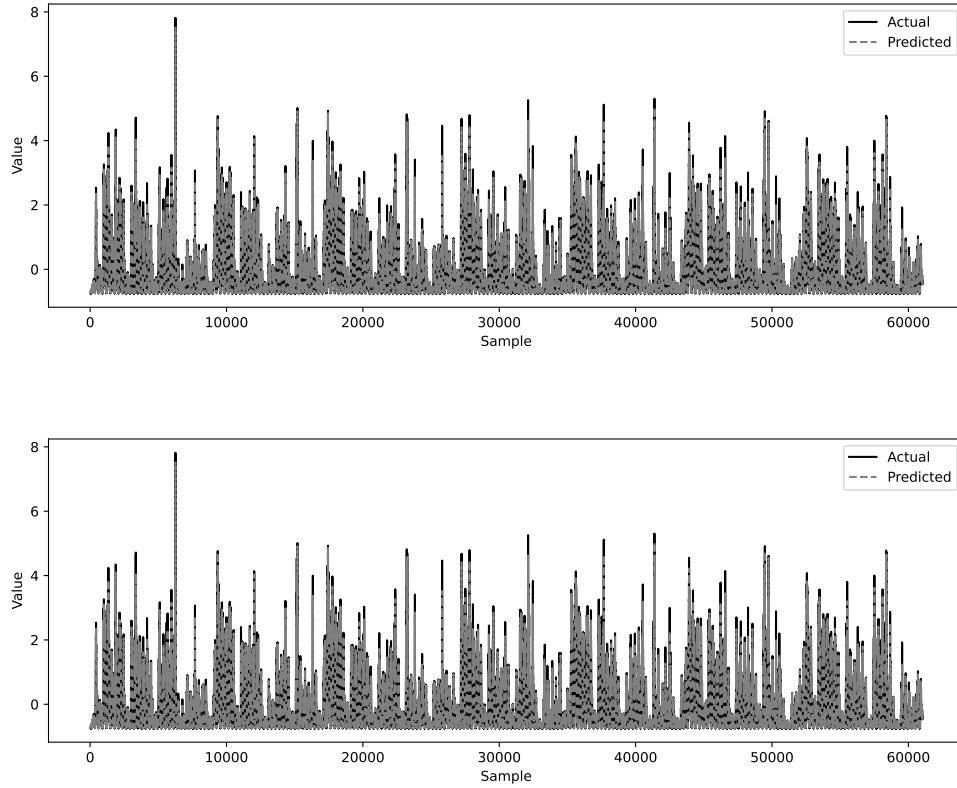


FIGURE 6.3: TSGFM predictions and ground truth values for two testing time series in scenario C (TELCO dataset).

6.4.2 TSGFM vs. Baselines

Table 6.3 shows the testing performance obtained by TSGFM and the baseline models for each permutation scenario, in terms of MAE and Residual Mean Square Error (RMSE). All metrics are normalized with respect to TSGFM, allowing for a direct comparison. The final column reports the average performance of each model, weighted by the number of samples of each dataset. TSGFM consistently outperforms all competing models when considering the overall weighted performance. Although FourierGNN achieves competitive results in several permutations, its poor performance on the Abilene networking dataset – used in permutation B and the largest dataset in the evaluation – significantly impacts its overall average performance, making it the worst-performing model in aggregate. Across permutations A, C, and E, TSGFM and FourierGNN demonstrate nearly identical performance, indicating that our model is equally effective at capturing spatiotemporal dependencies. However, in permutation B, TSGFM outperforms FourierGNN, highlighting its robustness on larger and more complex datasets. While FourierGNN slightly outperforms TSGFM in permutation D, the overall results suggest that TSGFM offers a more consistent and reliable performance, achieving a good forecasting accuracy without the added complexity of a hyper-graph representation. More generally, a model’s effectiveness should be evaluated based on its consistency across diverse setups, rather than just its peak performance in select scenarios. While FourierGNN performs well in some cases, its severe underperformance on the Abilene dataset highlights its lack of robustness. A model that excels in certain conditions but fails drastically in others cannot be considered reliable, as its effectiveness is highly dependent on the specific dataset or setup. For completeness, Figure 6.3 provides a visual comparison of the predicted and actual values for two time series from the TELCO dataset – mobile networks’ MTS data – in scenario C, as generated by TSGFM. Predictions highlight TSGFM’s ability to capture temporal patterns, showcasing how closely the model forecasting aligns with real-world network behavior with different temporal patterns in this case.

Permutation	Training	Testing
NET-A	Exchange, Spain, ETTh1	TELCO
NET-B	Exchange, Spain, ETTh1	Abilene

TABLE 6.4: Dataset permutations used for training and testing (network scenarios).

To further investigate the feasibility of applying foundation models trained on non-network data to the networking domain, we introduce two new evaluation permutations: NET-A and NET-B, as detailed in Table 6.4. These permutations are adapted from B and C (cf. Table 6.2), but are specifically designed to evaluate the extent to which learned temporal and structural patterns from non-network datasets can be effectively transferred to network-specific scenarios. The goal is to assess how well TSGFM generalizes to network monitoring tasks without direct exposure to network

Model \ Perm.	NET-A		NET-B		Avg.	
	MAE	RMSE	MAE	RMSE	MAE	RMSE
RNN	1.135	1.027	1.686	1.029	1.629	1.029
FreTS	1.096	1.003	1.185	1.016	1.176	1.015
BasicSTGNN	1.474	1.103	3.307	1.293	3.117	1.273
FourierGNN	1.673	1.860	4.832	5.013	4.505	4.687
GAT-AD	1.654	1.243	2.353	1.286	2.281	1.282
TSGFM	1.000	1.000	1.000	1.000	1.000	1.000

TABLE 6.5: Testing performance of forecasting models on network dataset permutations.

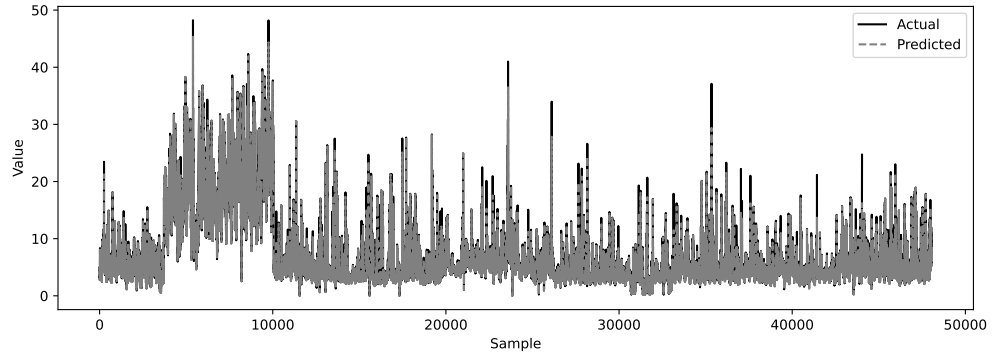
traffic data during pretraining, putting the emphasis on the general lack of labeled data in the networking domain.

Table 6.5 shows the performance obtained in the testing datasets of the two new permutations. We observe how TSGFM outperforms the rest of the baselines in the two scenarios, which hints at its superior capability of extracting correlations and features from diverse sources of data and apply the learned knowledge in a networking scenario.

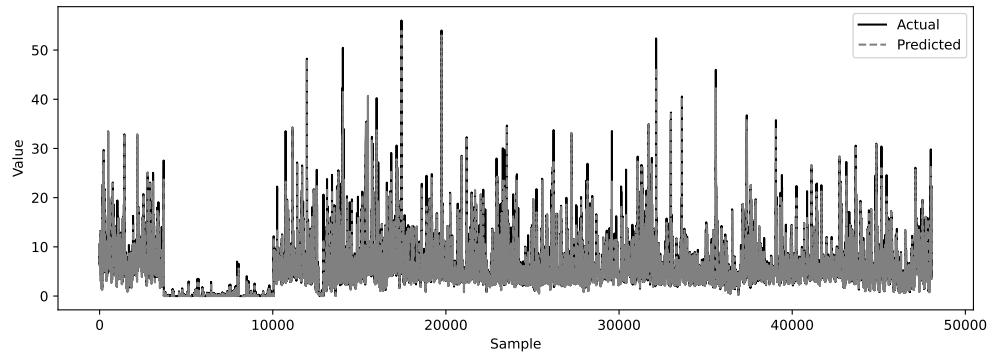
Table 6.5 reports the performance results on the testing datasets for the NET-A and NET-B permutations. Results demonstrate that TSGFM consistently outperforms all baseline models across both scenarios, by at least 18% when compared to FreTS, but largely outperforming all the other baselines. TSGFM demonstrates its superior ability to capture meaningful correlations and extract relevant features from diverse data sources. This strong performance suggests that TSGFM effectively transfers learned knowledge from non-network data to network-specific forecasting tasks, reinforcing its potential as a robust and adaptable foundation model for network monitoring applications. For the sake of completeness, Figure 6.4 shows three time series from the testing dataset in NET-B, as well as the predictions generated by TSGFM.

6.4.3 Analysis of the Self-attention Mechanism

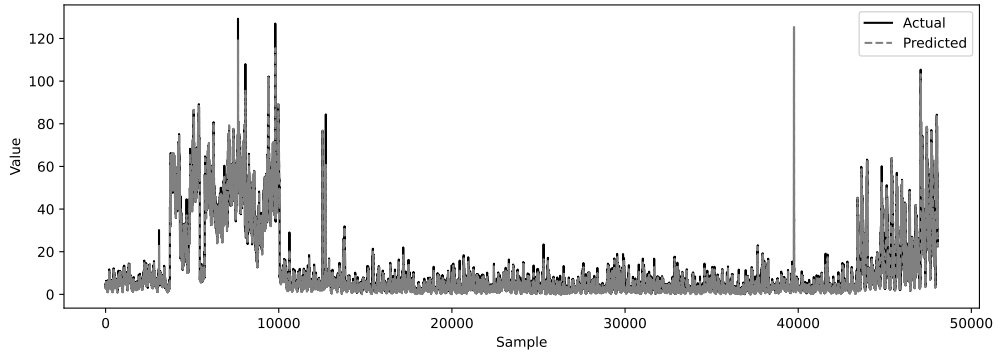
The analysis of the attention coefficients generated by the spatial attention mechanisms in TSGFM provides valuable insight on the relevance of the full-mesh spatial relationships. Figures 6.5a, 6.5b and 6.5c show histograms of the spatial *alphas* (cf. Section 6.3.2) obtained in testing. In permutations A and C, the distribution of spatial attention coefficients exhibits a very strong bimodal characteristic, where a significant number of values are clustered near 0 and another set is concentrated around 1. This distribution suggests that the model identifies a subset of key neighbors that strongly incorporate spatial information while effectively ignoring the rest. This suggests that the



(A) Time series 6.



(B) Time series 37.

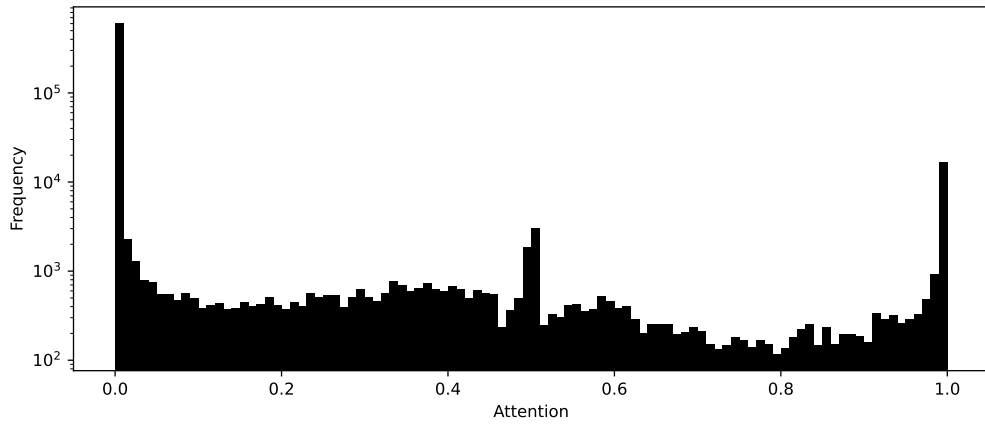


(C) Time series 84.

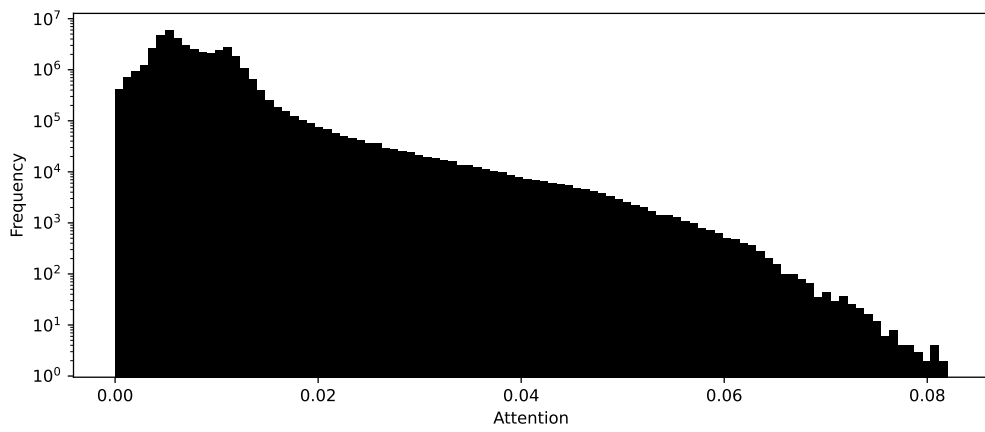
FIGURE 6.4: Comparison of predicted and actual time series for three samples in the NET-B testing dataset using TSGFM.

fully connected spatial structure could be effectively pruned in these permutations to keep only the most relevant neighbors. On the other hand, in the testing dataset of permutation B, the distribution of the attention coefficients resembles a right-skewed normal distribution. Here there are no clear dominant nodes in the spatial dimension, but rather a spread of moderately important neighbors. This indicates that, in this scenario, spatial information is more evenly distributed.

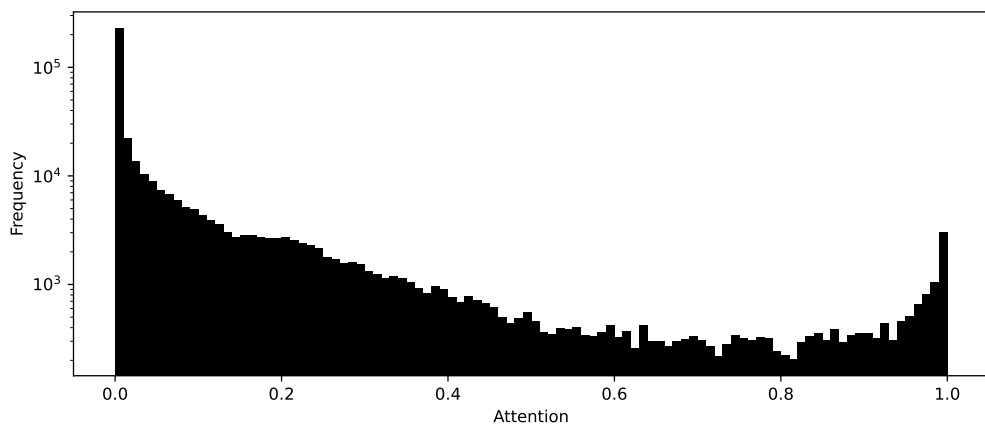
The wide range of spatial attention distributions across different datasets highlights an important insight: a full-mesh spatial attention mechanism is



(A) Scenario A



(B) Scenario B



(C) Scenario C

FIGURE 6.5: Histograms of spatial attention coefficients for the testing datasets in scenarios A, B, and C using TSGFM.

not always necessary. In many datasets, a more structured and selective adjacency pattern may be sufficient, and even preferable, as it would reduce

complexity and the computational resources necessary without degrading performance.

We also analyze the spatial and temporal attention coefficients in the network permutations, as reported in Table 6.4. Figure 6.6a shows the spatial attention alphas generated by TSGFM on the NET-B testing dataset, whereas Figure 6.6b shows the temporal attention alphas on the same dataset. The spatial attention is inclined towards lower values, with few coefficients near the maximum value, a little over 0.16. This suggests that only a small subset of spatial neighbors significantly contribute to updating the representation of a node, with the majority having a negligible influence. Taking into account that the testing dataset in NET-B (Abilene) has the highest number of time series – and thus, the highest number of spatial edges per node – these results highlight the possibility of avoiding a full-mesh spatial representation and choosing a more compact one with lower dimensionality.

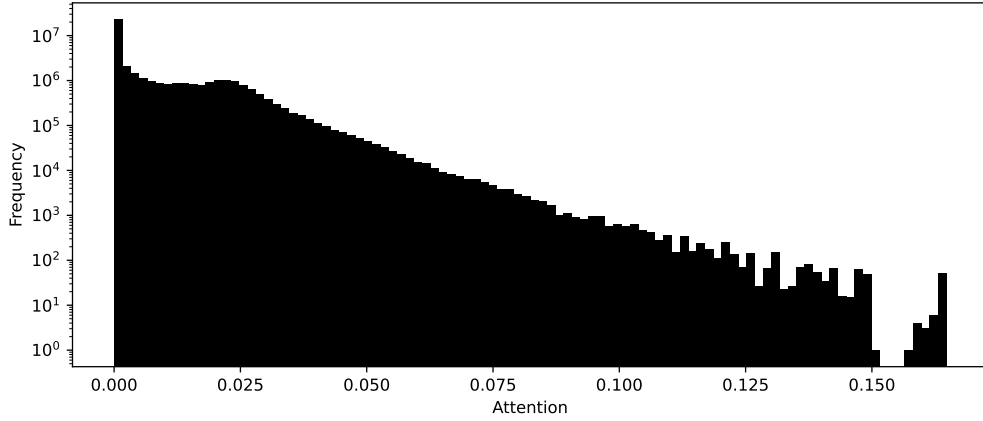
Similarly, the temporal attention histogram in Figure 6.6b shows a distribution where a huge part of the temporal edges are weighted with a low attention, indicating that they could be removed with potentially limited impact on the model performance.

6.4.4 Impact of Spatial Pruning in TSGFM Performance

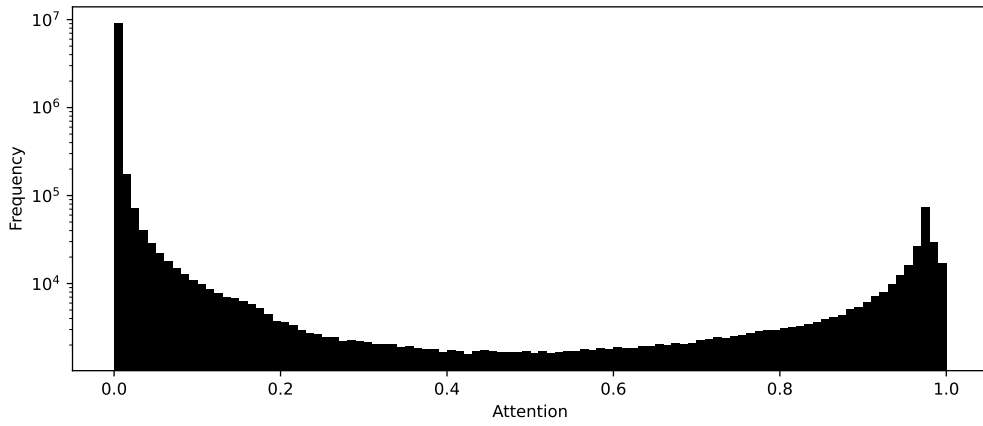
Based on the analysis in Section 6.4.3, where we observed that a significant number of edges could potentially be pruned – especially in the spatial dimension – in this section, we explore a spatial edge pruning strategy aimed at reducing the computational overhead, while preserving accuracy. Specifically, we compute pairwise Euclidean distances between the hidden states of source and destination nodes in each spatial graph, and retain only the top K nearest neighbors for message passing, where K represents a fraction of the total number of spatial edges.

To evaluate the effectiveness of this edge pruning strategy, we tested TSGFM under varying pruning levels with $K \in [0.2, 0.4, 0.6, 0.8]$. Table 6.6 summarizes the results for the NET-B scenario. The findings reveal that a substantial portion of the spatial edges –up to 40%– can be removed without a noticeable degradation in forecasting performance, highlighting the redundancy in densely connected spatial graphs. Beyond that limit, pruning starts impacting performance, suggesting that relevant information is being removed from the model.

Figure 6.7 shows the cumulative distribution function (CDF) of the spatial attention coefficients in the NET-B scenario for TSGFM under different pruning levels. The curves shift progressively to the right as the pruning becomes more aggressive (i.e., as K decreases). This skew indicates that fewer spatial edges receive near-zero attention values, meaning that the model has fewer irrelevant spatial edges it can choose to ignore. Despite this, Table 6.6 shows that the model maintains competitive forecasting performance even under high pruning levels such as $K = 0.2$. This suggests that a large number of



(A) Histogram of spatial attention coefficients.



(B) Histogram of temporal attention coefficients.

FIGURE 6.6: Spatial and temporal attention coefficient distributions generated by TSGFM on the NET-B testing dataset.

Model	MAE	RMSE
TSGFM ($K = 0.2$)	1.108	0.996
TSGFM ($K = 0.4$)	1.044	1.000
TSGFM ($K = 0.6$)	0.922	0.990
TSGFM ($K = 0.8$)	0.991	0.997
TSGFM (no pruning)	1.000	1.000

TABLE 6.6: Testing performance of TSGFM under different spatial pruning levels (NET-B scenario).

spatial edges in the fully connected graph are not essential for accurate forecasting, as long as the model has a mechanism that can effectively prioritize the most important spatial adjacencies among the reduced set of available ones. Notably, the pruning strategy used is fully independent of the attention mechanism itself—based solely on pairwise Euclidean distances between node embeddings—ensuring that the evaluation is unbiased and does not artificially favor the attention patterns learned by the model.

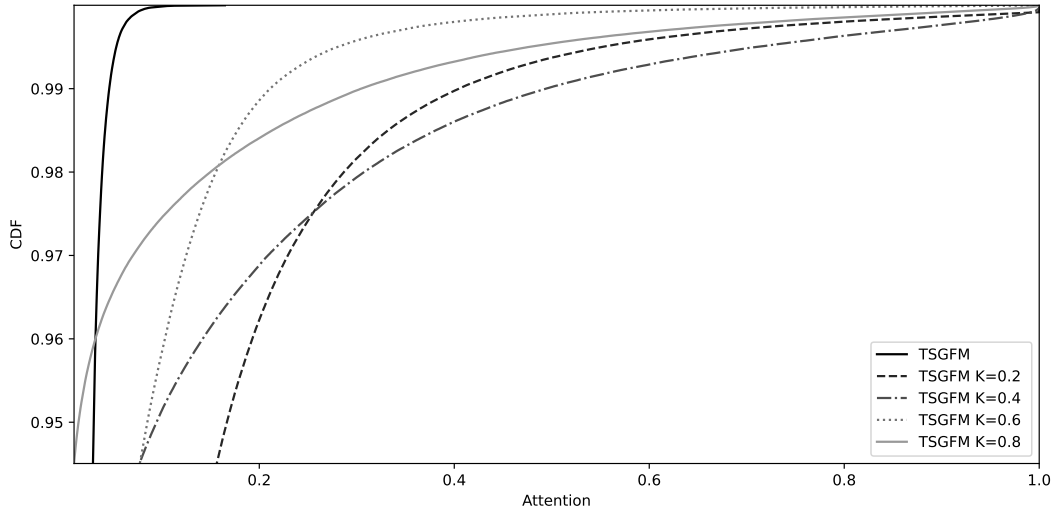


FIGURE 6.7: CDF of spatial attention coefficients for TSGFM in the NET-B testing dataset.

6.5 Summary of Outcomes

In this chapter, we have presented TSGFM, a GNN-based foundation model for zero-shot multivariate time series forecasting. We evaluated its performance using five different datasets spanning domains such as finance, energy, and computer networks, and compared it against five baseline models, including approaches based on GNNs, RNNs and MLPs. These baselines were selected to provide a broad perspective on forecasting methods. Evaluation results, summarized in Table 6.3, demonstrate that TSGFM consistently matches or outperforms all baselines across the evaluated permutations. The baselines include a diverse set of architectures capturing different modeling strategies for multivariate time series forecasting. Among them, FourierGNN shows competitive performance in several scenarios due to its expressive hypergraph-based representation. However, it struggles with the Abilene dataset (permutation B), which is the largest in the benchmark. By averaging the results obtained across all permutations, we see that TSGFM outperforms all the baselines but FreTS, which obtains the same performance in terms of RMSE, but is outperformed in absolute values. To further explore TSGFM’s generalization capability in network monitoring scenarios, we introduced two new evaluation setups: NET-A and NET-B. TSGFM significantly outperforms all baselines in both scenarios, reinforcing its suitability as an effective foundation model for time series forecasting in network monitoring scenarios.

Chapter 7

Conclusions and Future Work

7.1 Contributions

In this thesis, we have explored advancements in cybersecurity through three distinct case studies: (i) the development of a novel methodology that leverages the raw topology of the BGP network for anomaly detection, (ii) the use of a GAT-based model to unsupervisedly uncover and exploit latent contextual patterns in network monitoring data, enabling the detection of anomalies that might otherwise go unnoticed, and (iii) the application of GNNs to capture and utilize spatiotemporal correlations across data from multiple domains for forecasting purposes in the network monitoring domain. A common thread across these contributions is the emphasis on systematically incorporating the underlying topology of various data sources to enhance the prediction accuracy. While prior approaches have considered topological information to some extent, our work takes a more explicit and integrated approach, leveraging this structure more deeply and consistently to uncover patterns that were often underexploited.

The following paragraphs summarize the specific contributions presented in each chapter of this thesis.

The first main contribution is the development of a novel AD model that directly leverages the raw topology of the BGP network. Rather than relying on manually engineered features or incident-specific heuristics, our model leverages the inherent structure of the AS-level Internet graph to detect anomalous behavior. This approach significantly limits the need for extensive feature selection and hyperparameter tuning.

A key strength of our proposal lies in its ability to generalize across a wide range of anomalies. The model is trained using a fixed set of hyperparameters, regardless of the specific incident being analyzed. This is in contrast with most existing solutions, which often require fine-tuning to optimize performance for each individual anomaly. The evaluation demonstrates that by relying solely on topological information, without the need of heuristic features, our model is capable of detecting both known and previously unseen BGP anomalies. Furthermore, the evaluation using regular intervals

validates that this is obtained with a negligible level of false alarms, which make its deployment in real-world scenarios possible.

The second main contributions focus on enabling the unsupervised discovery and interpretation of contextual relationships within network traffic data. To achieve this, we proposed GAT-AD and GAT-AD+. GAT-AD+ is a GAT-based solution that builds on the knowledge obtained from GAT-AD and efficiently learns and explicitly reveals underlying contexts in network traffic data without relying on predefined heuristics or labels. By directly applying interpretable attention weights over raw traffic measurements, our model makes visible previously opaque contextual correlations, enabling network operators to gain deeper insights into the interdependencies of OD flows.

This enhanced interpretability allows network operators to understand not only *what* anomalies may be occurring, but also *why* and *how* they deviate from normal behavior relative to their respective contexts. By uncovering and leveraging these otherwise hidden correlations, GAT-AD+ efficiently identifies subtle, context-driven anomalies that traditional solutions may not be able to capture.

Lastly, the third main contribution is an exploratory proposal that introduces a graph-based foundation model for MTS forecasting in the context of network monitoring. Through the design and evaluation of TSGFM, we provide initial evidence that a single, unified model can effectively capture spatial and temporal correlations across time series from diverse domains without requiring task-specific training. A key insight from our analysis is that forecasting performance can be preserved even when simplifying the underlying graph representation. Specifically, our study of attention coefficients reveals that a large portion of spatial edges contribute minimally to the model's output, suggesting that a reduced and more meaningful set of connections is sufficient. This simplification is critical for production environments, where the dimensionality of the network can be orders of magnitude larger. By minimizing the unnecessary complexity in the topology representation, we pave the way for more scalable and deployable models, which is an essential step towards bringing spatiotemporal solutions to operational network monitoring systems.

7.2 Future Work

The contextual discovery model presented in Chapter 5 has provided valuable insights into learning the latent correlations between nodes in a network. However, its main limitation lies in its reliance on a full-mesh topology, where the attention mechanism evaluates all possible node pairs and assigns a correlation score to each. While this enables fine-grained context discovery, it also introduces significant computational overhead. Moreover, such an exhaustive topology may be unnecessary in real-world scenarios,

where many node pairs are inherently uncorrelated, for example, nodes located in distant regions of a global network. This inefficiency is further illustrated in Chapter 6, where an analysis of attention coefficients revealed that a large proportion of edges had minimal influence on the model’s predictions, suggesting they could be removed without loss of performance.

To address this, we propose the development of models that begin with a more compact, lower-dimensional "base" topology instead of a full mesh, and subsequently refine it through contextual learning. In this setting, the model would focus attention on a selective subset of edges, allowing it to learn meaningful relationships while avoiding unnecessary computation. A similar direction is explored in [32], where graph topology is learned through node embeddings. However, that approach has two key limitations: (i) the node degree is a fixed hyperparameter, requiring careful tuning, and (ii) it imposes uniform context sizes across nodes, despite some nodes potentially requiring fewer connections than others. We argue that allowing the context size to vary across nodes—and designing the topology accordingly—could lead to more accurate and efficient representations. A regularization mechanism could also be introduced to discourage overly large contexts, encouraging sparsity and interpretability in the learned topology.

Similarly, the work in Chapter 6 demonstrates that a spatiotemporal forecasting model does not necessarily need to explicitly model graph structure through predefined edges. By leveraging powerful sequence-based foundation models, it is possible to implicitly capture complex spatial and temporal dependencies purely through attention mechanisms. This challenges the conventional reliance on fixed or even learned graph structures and opens up a new direction for research.

While the current evaluation provides valuable insights into TSGFM’s performance, we acknowledge that a broader set of experiments—including more datasets and additional baseline models—is necessary to fully assess its generalization capabilities and practical deployment potential. This results in Chapter 6 represents an initial exploration aimed at analyzing how TSGFM performs across diverse scenarios, and it has already yielded important takeaways regarding its strengths, limitations, and avenues for improvement. Expanding the evaluation remains a key priority for future work, which we plan to pursue as extensively as computational resources allow.

An important direction we aim to explore is the way to apply attention in graph-based models. Specifically, rather than starting from a full-mesh topology and computing attention coefficients for all the edges, we aim to jointly learn both the attention weights and the underlying graph structure. Specifically, we wish to explore a module that dynamically constructs the graph topology by identifying and retaining only the most relevant edges (i.e., those with meaningful attention values) while pruning connections with near-zero attention. This approach would not only enhance interpretability by revealing salient dependencies but also offer significant computational benefits by reducing the number of edges over which attention must be computed.

Another interesting direction for future work is to investigate how the methodologies developed in this thesis can be extended from AD to Incident Detection (ID), which goes beyond identifying outliers and focuses on events with operational or security impact. While the current models are designed to identify deviations from the expected behavior, cybersecurity environments require identifying between benign anomalies and actual security incidents that may require intervention. To bridge this gap, future research could leverage labeled incident data or additional sources of domain knowledge. These adaptations could potentially enable the models to classify and prioritize anomalies based on their relevance to cybersecurity threats.

Appendices

Appendix A

Outcomes of the Thesis

A.1 List of Publications

Workshop Papers:

- Latif-Martínez, H., Paillissé, J., Yang, J., Cabellos-Aparicio, A., & Barlet-Ros, P. (2022, December). *Unveiling the potential of graph neural networks for BGP anomaly detection*. In *Proceedings of the 1st International Workshop on Graph Neural Networking* (pp. 7–12).
- Latif-Martínez, H., Suárez-Varela, J., Cabellos-Aparicio, A., & Barlet-Ros, P. (2023, December). *Detecting contextual network anomalies with graph neural networks*. In *Proceedings of the 2nd on Graph Neural Networking Workshop 2023* (pp. 25–30).
- **(under review)** Latif-Martínez, H., Casas, P., Suárez-Varela, J., Cabellos-Aparicio, A., & Barlet-Ros, P. (2025). *TSGFM – Towards a Graph Foundation Model for Time Series Analysis in Network Monitoring*.

Journal Papers:

- Latif-Martínez, H., Suárez-Varela, J., Cabellos-Aparicio, A., & Barlet-Ros, P. (2025). *GAT-AD: Graph Attention Networks for contextual anomaly detection in network monitoring*. *Computers & Industrial Engineering (JCR Q1)*, 200, 110830.
- **(minor revision)** Latif-Martínez, H., Paillissé, J., Barlet-Ros, P., & Cabellos-Aparicio, A. (2025). *BGP Anomaly Detection using the Raw Internet Topology*. *Computer Networks (JCR Q1)*

A.2 Other Merits

- Accepted Poster at the TMA 2022 PhD School. "*Unveiling the Potential of Graph Neural Networks for Network-Wide Anomaly Detection*"
- Accepted Poster at IFIP 2023 PhD Symposium on Next-Generation Networks. "*Unveiling the Potential of Graph Neural Networks for BGP Anomaly Detection*"

- Accepted Poster at TMA 2025. TSGFM - Towards a Graph Foundation Model for Time Series Analysis in Network Monitoring.
- 6-month research stay at the *Austrian Institute of Technology* under the supervision of Dr. Pedro Casas. The main research focused on foundation models for forecasting on network monitoring data.

Bibliography

- [1] Alejandro Acosta. *BGP Stream: An Analysis of One Year of BGP Incidents*. <https://blog.lacnic.net/en/routing/bgp-stream-an-analysis-of-one-year-of-bgp-incidents>. Accessed: 2024-04-16.
- [2] Chuadhry Mujeeb Ahmed, Venkata Reddy Palleti, and Aditya P Mathur. "WADI: a water distribution testbed for research in the design of secure cyber physical systems". In: *Proceedings of the 3rd international workshop on cyber-physical systems for smart water networks*. 2017, pp. 25–28.
- [3] Mohiuddin Ahmed, Abdun Naser Mahmood, and Jiankun Hu. "A survey of network anomaly detection techniques". In: *Journal of Network and Computer Applications* 60 (2016), pp. 19–31.
- [4] Leman Akoglu, Mary McGlohon, and Christos Faloutsos. "Oddball: Spotting anomalies in weighted graphs". In: *Advances in Knowledge Discovery and Data Mining: 14th Pacific-Asia Conference, PAKDD 2010, Hyderabad, India, June 21-24, 2010. Proceedings. Part II* 14. Springer. 2010, pp. 410–421.
- [5] Bahaa Al-Musawi, Philip Branch, and Grenville Armitage. "BGP anomaly detection techniques: A survey". In: *IEEE Communications Surveys & Tutorials* 19.1 (2016), pp. 377–396.
- [6] Bahaa Al-Musawi, Philip Branch, and Grenville Armitage. "Detecting BGP instability using recurrence quantification analysis (RQA)". In: *2015 IEEE 34th International Performance Computing and Communications Conference (IPCCC)*. IEEE. 2015, pp. 1–8.
- [7] Nabil Al-Rousan, Soroush Haeri, and Ljiljana Trajković. "Feature selection for classification of BGP anomalies using Bayesian models". In: *2012 International Conference on Machine Learning and Cybernetics*. Vol. 1. IEEE. 2012, pp. 140–147.
- [8] Nabil Mashhour Al-Rousan. "Comparison of machine learning models for classification of BGP anomalies". In: (2012).
- [9] Jinwon An and Sungzoon Cho. "Variational autoencoder based anomaly detection using reconstruction probability". In: *Special lecture on IE 2.1* (2015), pp. 1–18.
- [10] Tatsuya Arai, Kotaro Nakano, and Basabi Chakraborty. "Selection of effective features for BGP anomaly detection". In: *2019 IEEE 10th International Conference on Awareness Science and Technology (iCAST)*. IEEE. 2019, pp. 1–6.
- [11] Julien Audibert et al. "Usad: Unsupervised anomaly detection on multivariate time series". In: *Proceedings of the 26th ACM SIGKDD*

- international conference on knowledge discovery & data mining*. 2020, pp. 3395–3404.
- [12] Monowar H Bhuyan, Dhruva Kumar Bhattacharyya, and Jugal K Kalita. “Network anomaly detection: methods, systems and tools”. In: *Ieee communications surveys & tutorials* 16.1 (2013), pp. 303–336.
 - [13] Rishi Bommasani et al. “On the opportunities and risks of foundation models”. In: *arXiv preprint arXiv:2108.07258* (2021).
 - [14] Hedde HWJ Bosman et al. “Spatial anomaly detection in sensor networks using neighborhood information”. In: *Information Fusion* 33 (2017), pp. 41–56.
 - [15] George EP Box and George C Tiao. “A Bayesian approach to some outlier problems”. In: *Biometrika* 55.1 (1968), pp. 119–129.
 - [16] Daniela Brauckhoff, Kave Salamatian, and Martin May. “Applying PCA for traffic anomaly detection: Problems and solutions”. In: *IEEE INFOCOM 2009*. IEEE. 2009, pp. 2866–2870.
 - [17] Markus M Breunig et al. “LOF: identifying density-based local outliers”. In: *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 2000, pp. 93–104.
 - [18] Sabbas Burdakis and Antonios Deligiannakis. “Detecting outliers in sensor networks using the geometric approach”. In: *2012 IEEE 28th International Conference on Data Engineering*. IEEE. 2012, pp. 1108–1119.
 - [19] Kevin Butler et al. “A survey of BGP security issues and solutions”. In: *Proceedings of the IEEE* 98.1 (2009), pp. 100–122.
 - [20] Jie Cai et al. “Feature selection in machine learning: A new perspective”. In: *Neurocomputing* 300 (2018), pp. 70–79.
 - [21] Mikel Canizo et al. “Multi-head CNN-RNN for multi-time series anomaly detection: An industrial case study”. In: *Neurocomputing* 363 (2019), pp. 246–260.
 - [22] Defu Cao et al. “Spectral temporal graph neural network for multi-variate time-series forecasting”. In: *Advances in neural information processing systems* 33 (2020), pp. 17766–17778.
 - [23] Kevin M Carter and William W Streilein. “Probabilistic reasoning for streaming anomaly detection”. In: *2012 IEEE Statistical Signal Processing Workshop (SSP)*. IEEE. 2012, pp. 377–380.
 - [24] Varun Chandola, Arindam Banerjee, and Vipin Kumar. “Anomaly detection: A survey”. In: *ACM computing surveys (CSUR)* 41.3 (2009), pp. 1–58.
 - [25] Yihao Chen et al. “Learning with Semantics: Towards a {Semantics-Aware} Routing Anomaly Detection System”. In: *33rd USENIX Security Symposium (USENIX Security 24)*. 2024, pp. 5143–5160.
 - [26] Min Cheng et al. “MS-LSTM: A multi-scale LSTM model for BGP anomaly detection”. In: *2016 IEEE 24th international conference on network protocols (ICNP)*. IEEE. 2016, pp. 1–6.
 - [27] Marijana Cosovic, Slobodan Obradovic, and Emina Junuz. “Deep learning for detection of BGP anomalies”. In: *Time Series Analysis and Forecasting: Selected Contributions from ITISE 2017*. Springer. 2018, pp. 95–113.

- [28] Xianbo Dai, Na Wang, and Wenjuan Wang. "Application of machine learning in BGP anomaly detection". In: *Journal of Physics: Conference Series*. Vol. 1176. 3. IOP Publishing. 2019, p. 032015.
- [29] Abhimanyu Das et al. *A Decoder-only Foundation Model for Time-series Forecasting*. 2024. arXiv: [2310.10688 \[cs.CL\]](#).
- [30] Abhimanyu Das et al. "A decoder-only foundation model for time-series forecasting". In: *Forty-first International Conference on Machine Learning*. 2024.
- [31] José Edson De Albuquerque Filho et al. "A review of neural networks for anomaly detection". In: *IEEE Access* 10 (2022), pp. 112342–112367.
- [32] Ailin Deng and Bryan Hooi. "Graph neural network-based anomaly detection in multivariate time series". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 35. 5. 2021, pp. 4027–4035.
- [33] Dorothy E Denning. "An intrusion-detection model". In: *IEEE Transactions on software engineering* 2 (1987), pp. 222–232.
- [34] Jacob Devlin et al. "Bert: Pre-training of deep bidirectional transformers for language understanding". In: *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019, pp. 4171–4186.
- [35] Chaoyue Ding, Shiliang Sun, and Jing Zhao. "MST-GAT: A multimodal spatial-temporal graph attention network for time series anomaly detection". In: *Information Fusion* 89 (2023), pp. 527–536.
- [36] Kaize Ding et al. "Deep anomaly detection on attributed networks". In: *Proceedings of the 2019 SIAM international conference on data mining*. SIAM. 2019, pp. 594–602.
- [37] Meimei Ding and Hui Tian. "PCA-based network traffic anomaly detection". In: *Tsinghua Science and Technology* 21.5 (2016), pp. 500–509.
- [38] WJ Dixon. "Processing data for outliers". In: *Biometrics* 9.1 (1953), pp. 74–89.
- [39] Yutao Dong et al. "ISP self-operated BGP anomaly detection based on weakly supervised learning". In: *2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE. 2021, pp. 1–11.
- [40] Alexey Dosovitskiy et al. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *arXiv preprint arXiv:2010.11929* (2020).
- [41] Yingdong Dou et al. "Enhancing graph neural network-based fraud detectors against camouflaged fraudsters". In: *Proceedings of the 29th ACM international conference on information & knowledge management*. 2020, pp. 315–324.
- [42] Alessandro D'Alconzo et al. "A Survey on Big Data for Network Traffic Monitoring and Analysis". In: *IEEE Transactions on Network and Service Management* 16.3 (2019), pp. 800–813. DOI: [10.1109/TNSM.2019.2933358](#).
- [43] Francis Ysidro Edgeworth. "Xli. on discordant observations". In: *The london, edinburgh, and dublin philosophical magazine and journal of science* 23.143 (1887), pp. 364–375.

- [44] Martin Ester et al. "A density-based algorithm for discovering clusters in large spatial databases with noise". In: *kdd*. Vol. 96. 34. 1996, pp. 226–231.
- [45] Wenqi Fan et al. "Graph neural networks for social recommendation". In: *The world wide web conference*. 2019, pp. 417–426.
- [46] Nick Feamster and Hari Balakrishnan. "Detecting BGP configuration faults with static analysis". In: *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*. 2005, pp. 43–56.
- [47] Gilberto Fernandes et al. "A comprehensive survey on network anomaly detection". In: *Telecommunication Systems* 70 (2019), pp. 447–489.
- [48] Stephanie Forrest et al. "A sense of self for unix processes". In: *Proceedings 1996 IEEE symposium on security and privacy*. IEEE. 1996, pp. 120–128.
- [49] Gastón García González et al. *TELCO*. 2023. DOI: [10 . 21227 / skpg - 0539](https://doi.org/10.21227/skpg-0539). URL: <https://dx.doi.org/10.21227/skpg-0539>.
- [50] Azul Garza and Max Mergenthaler-Canseco. *TimeGPT-1*. 2023. arXiv: [2310.03589](https://arxiv.org/abs/2310.03589) [cs.LG].
- [51] Xiaohu Ge. "Ultra-reliable low-latency communications in autonomous vehicular networks". In: *IEEE Transactions on Vehicular Technology* 68.5 (2019), pp. 5005–5016.
- [52] Yossi Gilad et al. "Are we there yet? On RPKI's deployment and security". In: *Cryptology ePrint Archive* (2016).
- [53] Justin Gilmer et al. "Neural message passing for quantum chemistry". In: *International conference on machine learning*. PMLR. 2017, pp. 1263–1272.
- [54] Jonathan Goh et al. "A dataset to support research in the design of secure water treatment systems". In: *Critical Information Infrastructures Security: 11th International Conference, CRITIS 2016, Paris, France, October 10–12, 2016, Revised Selected Papers 11*. Springer. 2017, pp. 88–99.
- [55] Gastón García González et al. "On the Quest for Foundation Generative-AI Models for Anomaly Detection in Time-Series Data". In: *2024 IEEE European Symposium on Security and Privacy Workshops (EuroSPW)*. 2024, pp. 252–260. DOI: [10 . 1109 / EuroSPW61312 . 2024 . 00034](https://doi.org/10.1109/EuroSPW61312.2024.00034).
- [56] Jake Grigsby et al. "Long-range transformers for dynamic spatiotemporal forecasting". In: *arXiv preprint arXiv:2109.12218* (2021).
- [57] Frank E Grubbs. "Procedures for detecting outlying observations in samples". In: *Technometrics* 11.1 (1969), pp. 1–21.
- [58] Nate Gruver et al. *Large Language Models Are Zero-Shot Time Series Forecasters*. 2023. arXiv: [2310.07820](https://arxiv.org/abs/2310.07820) [cs.LG].
- [59] Shengnan Guo et al. "Attention based spatial-temporal graph convolutional networks for traffic flow forecasting". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 33. 01. 2019, pp. 922–929.

- [60] Arpit Gupta et al. "Sonata: Query-driven streaming network telemetry". In: *Proceedings of the 2018 conference of the ACM special interest group on data communication*. 2018, pp. 357–371.
- [61] Umang Gupta, Vandana Bhattacharjee, and Partha Sarathi Bishnu. "Outlier detection in wireless sensor networks based on neighbourhood". In: *Wireless Personal Communications* 116 (2021), pp. 443–454.
- [62] Andreas Haeberlen et al. "NetReview: Detecting When Interdomain Routing Goes Wrong." In: *NSDI*. Vol. 2009. 2009, pp. 437–452.
- [63] Will Hamilton, Zhitao Ying, and Jure Leskovec. "Inductive representation learning on large graphs". In: *Advances in neural information processing systems* 30 (2017).
- [64] Frank R Hampel. "The influence curve and its role in robust estimation". In: *Journal of the american statistical association* 69.346 (1974), pp. 383–393.
- [65] Nicholas J. Hana. *Energy Consumption, Generation, Prices and Weather Dataset*. Accessed: 2025-03-28. 2021. URL: <https://www.kaggle.com/datasets/nicholasjhana/energy-consumption-generation-prices-and-weather>.
- [66] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. "Distilling the knowledge in a neural network". In: *arXiv preprint arXiv:1503.02531* (2015).
- [67] Kevin Hoarau, Tahiry Razafindralambo, and Pierre Ugo Tournoux. "Unsupervised representation learning for BGP anomaly detection using graph auto-encoders". In: *ITU Journal on Future and Evolving Technologies* 5.1 (2024), pp. 120–133.
- [68] Kevin Hoarau, Pierre Ugo Tournoux, and Tahiry Razafindralambo. "Bgnn: Detection of bgp anomalies using graph neural networks". In: *2022 IEEE Symposium on Computers and Communications (ISCC)*. IEEE. 2022, pp. 1–6.
- [69] Victoria Hodge and Jim Austin. "A survey of outlier detection methodologies". In: *Artificial intelligence review* 22 (2004), pp. 85–126.
- [70] Xin Hu and Z Morley Mao. "Accurate real-time identification of IP prefix hijacking". In: *2007 IEEE Symposium on Security and Privacy (SP'07)*. IEEE. 2007, pp. 3–17.
- [71] Xin Hu and Z. Morley Mao. "Accurate Real-time Identification of IP Prefix Hijacking". In: *2007 IEEE Symposium on Security and Privacy (SP '07)*. 2007, pp. 3–17. DOI: [10.1109/SP.2007.7](https://doi.org/10.1109/SP.2007.7).
- [72] Hong Huang, Hussein Al-Azzawi, and Hajar Brani. "Network traffic anomaly detection". In: *arXiv preprint arXiv:1402.0856* (2014).
- [73] Qiang Huang et al. "Graphlime: Local interpretable model explanations for graph neural networks". In: *IEEE Transactions on Knowledge and Data Engineering* (2022).
- [74] Sijiang Huang et al. "xnet: Modeling network performance with graph neural networks". In: *IEEE/ACM Transactions on Networking* (2023).
- [75] Tianjin Huang et al. "Hop-count based self-supervised anomaly detection on attributed networks". In: *Joint European conference on*

- machine learning and knowledge discovery in databases*. Springer. 2022, pp. 225–241.
- [76] Yiyi Huang et al. “Diagnosing Network Disruptions with Network-Wide Analysis”. In: *Proceedings of the 2007 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*. SIGMETRICS '07. San Diego, California, USA: Association for Computing Machinery, 2007, 61–72. ISBN: 9781595936394. DOI: 10.1145/1254882.1254890. URL: <https://doi.org/10.1145/1254882.1254890>.
 - [77] Peter J Huber and Elvezio M Ronchetti. *Robust statistics*. John Wiley & Sons, 2011.
 - [78] Alexis Huet, Jose Manuel Navarro, and Dario Rossi. “Local evaluation of time series anomaly detection algorithms”. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2022, pp. 635–645.
 - [79] Geoff Huston. *IETF 102: An update on securing BGP*. <https://www.ietfjournal.org/ietf-102-an-update-on-securing-bgp/>. Accessed: 2024-01-17. 2018.
 - [80] Anand Padmanabha Iyer, Li Erran Li, and Ion Stoica. “Automating diagnosis of cellular radio access network problems”. In: *Proceedings of the 23rd Annual International Conference on Mobile Computing and Networking*. 2017, pp. 79–87.
 - [81] Ehsan Jolous Jamshidi et al. “Detecting outliers in a univariate time series dataset using unsupervised combined statistical methods: A case study on surface water temperature”. In: *Ecological Informatics* 69 (2022), p. 101672.
 - [82] Ming Jin et al. “Time-LLM: Time series forecasting by reprogramming large language models”. In: *International Conference on Learning Representations (ICLR)*. 2024.
 - [83] Mohsen Karimi et al. “Border gateway protocol anomaly detection using neural network”. In: *2019 IEEE International Conference on Big Data (Big Data)*. IEEE. 2019, pp. 6092–6094.
 - [84] Josh Karlin, Stephanie Forrest, and Jennifer Rexford. “Pretty good BGP: improving BGP by cautiously adopting routes”. In: *Proceedings of the 2006 IEEE International Conference on Network Protocols*. IEEE. 2006, pp. 290–299.
 - [85] Ömer Kasim. “An efficient and robust deep learning based network anomaly detection against distributed denial of service attacks”. In: *Computer Networks* 180 (2020), p. 107390.
 - [86] Samina Khalid, Tehmina Khalil, and Shamila Nasreen. “A survey of feature selection and feature extraction techniques in machine learning”. In: *2014 science and information conference*. IEEE. 2014, pp. 372–378.
 - [87] Thomas N Kipf and Max Welling. “Semi-supervised classification with graph convolutional networks”. In: *arXiv preprint arXiv:1609.02907* (2016).

- [88] C. Labovitz, G.R. Malan, and F. Jahanian. "Internet routing instability". In: *IEEE/ACM Transactions on Networking* 6.5 (1998), pp. 515–528. DOI: [10.1109/90.731185](https://doi.org/10.1109/90.731185).
- [89] Mohit Lad et al. "PHAS: A Prefix Hijack Alert System." In: *USENIX Security symposium*. Vol. 1. 2. 2006, p. 3.
- [90] Guokun Lai. *Exchange Rate Multivariate Time Series Dataset*. Accessed: 2025-03-28. 2017. URL: https://github.com/laiguokun/multivariate-time-series-data/blob/master/exchange_rate/exchange_rate.txt.gz.
- [91] Anukool Lakhina, Mark Crovella, and Christophe Diot. "Detecting distributed attacks using network-wide flow traffic". In: *Proceedings of FloCon 2005 Analysis Workshop*. 2005.
- [92] Anukool Lakhina, Mark Crovella, and Christophe Diot. "Diagnosing network-wide traffic anomalies". In: *ACM SIGCOMM computer communication review* 34.4 (2004), pp. 219–230.
- [93] Aleksandar Lazarevic et al. "A comparative study of anomaly detection schemes in network intrusion detection". In: *Proceedings of the 2003 SIAM international conference on data mining*. SIAM. 2003, pp. 25–36.
- [94] Matt Lepinski and Stephen Kent. *An Infrastructure to Support Secure Internet Routing*. RFC 6480. Feb. 2012. DOI: [10.17487/RFC6480](https://doi.org/10.17487/RFC6480). URL: <https://www.rfc-editor.org/info/rfc6480>.
- [95] Matt Lepinski and Kotikalapudi Sriram. *BGPsec Protocol Specification*. RFC 8205. Sept. 2017. DOI: [10.17487/RFC8205](https://doi.org/10.17487/RFC8205). URL: <https://www.rfc-editor.org/info/rfc8205>.
- [96] Gen Li and Jason J Jung. "Deep learning for anomaly detection in multivariate time series: Approaches, applications, and challenges". In: *Information Fusion* 91 (2023), pp. 93–102.
- [97] Jun Li et al. "An Internet routing forensics framework for discovering rules of abnormal BGP events". In: *ACM SIGCOMM Computer Communication Review* 35.5 (2005), pp. 55–66.
- [98] Xiao Li et al. "A survey of graph neural network based recommendation in social networks". In: *Neurocomputing* 549 (2023), p. 126441.
- [99] Yan Li et al. "Classification of BGP anomalies using decision trees and fuzzy rough sets". In: *2014 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE. 2014, pp. 1312–1317.
- [100] Zhida Li, Ana Laura Gonzalez Rios, and Ljiljana Trajković. "Detecting internet worms, ransomware, and blackouts using recurrent neural networks". In: *2020 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. IEEE. 2020, pp. 2165–2172.
- [101] Yuxuan Liang et al. "Foundation models for time series analysis: A tutorial and survey". In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2024, pp. 6555–6565.
- [102] Bryan Lim and Stefan Zohren. "Time-series forecasting with deep learning: a survey". In: *Philosophical Transactions of the Royal Society A* 379.2194 (2021), p. 20200209.

- [103] Shuyu Lin et al. "Anomaly detection for time series using vae-lstm hybrid model". In: *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Ieee. 2020, pp. 4322–4326.
- [104] Jiawei Liu et al. "Graph Foundation Models: Concepts, Opportunities and Challenges". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [105] Yixin Liu et al. "Anomaly detection on attributed networks via contrastive self-supervised learning". In: *IEEE transactions on neural networks and learning systems* 33.6 (2021), pp. 2378–2392.
- [106] Wai Weng Lo et al. "E-graphsage: A graph neural network based intrusion detection system for iot". In: *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*. IEEE. 2022, pp. 1–9.
- [107] Ratul Mahajan, David Wetherall, and Tom Anderson. "Understanding BGP misconfiguration". In: *ACM SIGCOMM Computer Communication Review* 32.4 (2002), pp. 3–16.
- [108] Pankaj Malhotra et al. "LSTM-based encoder-decoder for multi-sensor anomaly detection". In: *arXiv preprint arXiv:1607.00148* (2016).
- [109] Augusto Mathurin. *A Regional Look into BGP Incidents in 2020*. <https://manrs.org/2021/03/a-regional-look-into-bgp-incidents-in-2020/>. Accessed: 2024-04-16.
- [110] Shengjie Min et al. "Stgsn—a spatial-temporal graph neural network framework for time-evolving social networks". In: *Knowledge-Based Systems* 214 (2021), p. 106746.
- [111] Asya Mitseva, Andriy Panchenko, and Thomas Engel. "The state of affairs in BGP security: A survey of attacks and defenses". In: *Computer Communications* 124 (2018), pp. 45–60.
- [112] Gerhard Münz, Sa Li, and Georg Carle. "Traffic anomaly detection using k-means clustering". In: *Gi/itg workshop mmbnet*. Vol. 7. 9. 2007.
- [113] Ali Bou Nassif et al. "Machine learning for anomaly detection: A systematic review". In: *Ieee Access* 9 (2021), pp. 78658–78700.
- [114] Edward Appau Nketiah et al. "Recurrent neural network modeling of multivariate time series and its application in temperature forecasting". In: *Plos one* 18.5 (2023), e0285713.
- [115] Sarah Parisot et al. "Disease prediction using graph convolutional networks: application to autism spectrum disorder and Alzheimer's disease". In: *Medical image analysis* 48 (2018), pp. 117–130.
- [116] R Pascanu. "On the difficulty of training recurrent neural networks". In: *arXiv preprint arXiv:1211.5063* (2013).
- [117] Songtao Peng et al. "A Multi-View Framework for BGP Anomaly Detection via Graph Attention Network". In: *arXiv preprint arXiv:2112.12793* (2021).
- [118] Jian Qiu et al. "Detecting bogus BGP route information: Going beyond prefix hijacking". In: *2007 Third International Conference on Security and Privacy in Communications Networks and the Workshops-SecureComm 2007*. IEEE. 2007, pp. 381–390.

- [119] Sridhar Ramaswamy, Rajeev Rastogi, and Kyuseok Shim. "Efficient algorithms for mining outliers from large data sets". In: *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*. 2000, pp. 427–438.
- [120] Kashif Rasul et al. *Lag-Llama: Towards Foundation Models for Probabilistic Time Series Forecasting*. 2024. arXiv: 2310.08278 [cs.LG].
- [121] Patrick Reiser et al. "Graph neural networks for materials science and chemistry". In: *Communications Materials* 3.1 (2022), p. 93.
- [122] RIPE Routing Information Service. <https://www.ripe.net/analyse/internet-measurements/routing-information-service-ris>. Accessed: 2024-02-22.
- [123] Peter J Rousseeuw and Mia Hubert. "Anomaly detection by robust statistics". In: *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* 8.2 (2018), e1236.
- [124] Amit Roy et al. "Gad-nr: Graph anomaly detection via neighborhood reconstruction". In: *Proceedings of the 17th ACM international conference on web search and data mining*. 2024, pp. 576–585.
- [125] Krzysztof Rusek et al. "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN". In: *IEEE Journal on Selected Areas in Communications* 38.10 (2020), pp. 2260–2270.
- [126] Mahmoud Said Elsayed et al. "Network anomaly detection using LSTM based autoencoder". In: *Proceedings of the 16th ACM Symposium on QoS and Security for Wireless and Mobile Networks*. 2020, pp. 37–45.
- [127] Mayu Sakurada and Takehisa Yairi. "Anomaly detection using autoencoders with nonlinear dimensionality reduction". In: *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*. 2014, pp. 4–11.
- [128] Stan Salvador and Philip Chan. "Toward accurate dynamic time warping in linear time and space". In: *Intelligent Data Analysis* 11.5 (2007), pp. 561–580.
- [129] Odnan Ref Sanchez et al. "Comparing machine learning algorithms for BGP anomaly detection using graph features". In: *Proceedings of the 3rd ACM CoNEXT Workshop on Big Data, Machine Learning and Artificial Intelligence for Data Communication Networks*. 2019, pp. 35–41.
- [130] Franco Scarselli et al. "The graph neural network model". In: *IEEE transactions on neural networks* 20.1 (2008), pp. 61–80.
- [131] Bernhard Schölkopf et al. "Support vector method for novelty detection". In: *Advances in neural information processing systems* 12 (1999).
- [132] Ben A Scott et al. "BGP anomaly detection as a group dynamics problem". In: *Computer Networks* 257 (2025), p. 110926.
- [133] Pavlos Sermpezis and Xenofontas Dimitropoulos. "Can SDN accelerate BGP convergence?" In: *arXiv preprint arXiv:1702.00188* (2017).
- [134] Tal Shapira and Yuval Shavitt. "SASA: Source-aware self-attention for ip hijack detection". In: *IEEE/ACM Transactions on Networking* 30.1 (2021), pp. 437–449.
- [135] Kamran Shaukat et al. "A review of time-series anomaly detection techniques: A step to future perspectives". In: *Advances in Information*

- and Communication: Proceedings of the 2021 Future of Information and Communication Conference (FICC), Volume 1. Springer. 2021, pp. 865–877.
- [136] Lifeng Shen, Zhuocong Li, and James Kwok. “Timeseries anomaly detection using temporal hierarchical one-class network”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 13016–13026.
 - [137] Yifei Shen et al. “Graph neural networks for wireless communications: From theory to practice”. In: *IEEE Transactions on Wireless Communications* 22.5 (2022), pp. 3554–3569.
 - [138] Weijing Shi and Raj Rajkumar. “Point-gnn: Graph neural network for 3d object detection in a point cloud”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2020, pp. 1711–1719.
 - [139] Xingang Shi et al. “Detecting prefix hijackings in the internet with argus”. In: *Proceedings of the 2012 Internet Measurement Conference*. 2012, pp. 15–28.
 - [140] National Institute of Standards and Technology. *NIST RPKI Monitor Global Analysis*. <https://rpki-monitor.antd.nist.gov/ROV>. Accessed: 2024-01-17. 2022.
 - [141] Andriy Stetsko, Lukas Folkman, and Vashek Matyáš. “Neighbor-based intrusion detection for wireless sensor networks”. In: *2010 6th International Conference on Wireless and Mobile Communications*. IEEE. 2010, pp. 420–425.
 - [142] Qingxiong Tan et al. “Data-gru: Dual-attention time-aware gated recurrent unit for irregular multivariate time series”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 34. 01. 2020, pp. 930–937.
 - [143] Kiran K Thekumparampil et al. “Attention-based graph neural network for semi-supervised learning”. In: *arXiv preprint arXiv:1803.03735* (2018).
 - [144] Georgios Theodoridis, Orestis Tsigkas, and Dimitrios Tzovaras. “A novel unsupervised method for securing BGP against routing hijacks”. In: *Computer and Information Sciences III*. Springer, 2013, pp. 21–29.
 - [145] Andree Toonk. *Massive route leak causes Internet slowdown*. <https://www.bgpmon.net/massive-route-leak-cause-internet-slowdown/>. Accessed: 2024-04-23.
 - [146] Pang-Wei Tsai et al. “Network monitoring in software-defined networking: A review”. In: *IEEE Systems Journal* 12.4 (2018), pp. 3958–3969.
 - [147] Iñigo Ortiz de Urbina Cazenave, Erkan Köşlük, and Murat Can Ganiz. “An anomaly detection framework for BGP”. In: *2011 International Symposium on Innovations in Intelligent Systems and Applications*. IEEE. 2011, pp. 107–111.

- [148] Iñigo Ortiz de Urbina Cazenave, Erkan Köşlük, and Murat Can Ganiz. "An anomaly detection framework for BGP". In: *2011 International Symposium on Innovations in Intelligent Systems and Applications*. 2011, pp. 107–111. DOI: [10.1109/INISTA.2011.5946083](https://doi.org/10.1109/INISTA.2011.5946083).
- [149] Boris Van Breugel and Mihaela Van Der Schaar. "Why tabular foundation models should be a research priority". In: *arXiv preprint arXiv:2405.01147* (2024).
- [150] Petar Veličković et al. "Graph attention networks". In: *arXiv preprint arXiv:1710.10903* (2017).
- [151] Huazhang Wang et al. "Ecggan: A framework for effective and interpretable electrocardiogram anomaly detection". In: *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2023, pp. 5071–5081.
- [152] Song Wang et al. "Machine learning in network anomaly detection: A survey". In: *IEEE Access* 9 (2021), pp. 152379–152396.
- [153] Xiao Wang et al. "Heterogeneous graph attention network". In: *The world wide web conference*. 2019, pp. 2022–2032.
- [154] Xing Wang et al. "A survey on deep learning for cellular traffic prediction". In: *Intelligent Computing* 3 (2024), p. 0054.
- [155] Yulei Wu, Hong-Ning Dai, and Haina Tang. "Graph neural networks for anomaly detection in industrial Internet of Things". In: *IEEE Internet of Things Journal* 9.12 (2021), pp. 9214–9231.
- [156] Zonghan Wu et al. "A comprehensive survey on graph neural networks". In: *IEEE transactions on neural networks and learning systems* 32.1 (2020), pp. 4–24.
- [157] Zonghan Wu et al. "Connecting the dots: Multivariate time series forecasting with graph neural networks". In: *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2020, pp. 753–763.
- [158] Zonghan Wu et al. "Graph wavenet for deep spatial-temporal graph modeling". In: *arXiv preprint arXiv:1906.00121* (2019).
- [159] Da Xu et al. "Inductive representation learning on temporal graphs". In: *arXiv preprint arXiv:2002.07962* (2020).
- [160] Haowen Xu et al. "Unsupervised anomaly detection via variational auto-encoder for seasonal kpis in web applications". In: *Proceedings of the 2018 world wide web conference*. 2018, pp. 187–196.
- [161] Jiehui Xu et al. "Anomaly transformer: Time series anomaly detection with association discrepancy". In: *arXiv preprint arXiv:2110.02642* (2021).
- [162] Keyulu Xu et al. "How powerful are graph neural networks?" In: *arXiv preprint arXiv:1810.00826* (2018).
- [163] Young Xu. NANOG 68: Decoding Performance Data from Large-Scale Internet Outages. <https://www.thousandeyes.com/blog/nanog-68-decoding-performance-data-internet-outages>. Accessed: 2024-04-23.
- [164] Hao Xue and Flora D. Salim. "PromptCast: A New Prompt-Based Learning Paradigm for Time Series Forecasting". In: *IEEE Transactions*

- on *Knowledge and Data Engineering* (2023), pp. 1–14. DOI: [10 . 1109 / TKDE. 2023. 3342137](https://doi.org/10.1109/TKDE.2023.3342137).
- [165] Nong Ye, Sean Vilbert, and Qiang Chen. “Computer intrusion detection through EWMA for autocorrelated and uncorrelated data”. In: *IEEE transactions on reliability* 52.1 (2003), pp. 75–82.
 - [166] Kun Yi et al. “FourierGNN: Rethinking multivariate time series forecasting from a pure graph perspective”. In: *Advances in neural information processing systems* 36 (2023), pp. 69638–69660.
 - [167] Kun Yi et al. “FourierGNN: Rethinking multivariate time series forecasting from a pure graph perspective”. In: *Advances in Neural Information Processing Systems* 36 (2024).
 - [168] Kun Yi et al. “Frequency-domain MLPs are more effective learners in time series forecasting”. In: *Advances in Neural Information Processing Systems* 36 (2024).
 - [169] Rex Ying et al. “Graph convolutional neural networks for web-scale recommender systems”. In: *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 2018, pp. 974–983.
 - [170] Bing Yu, Haoteng Yin, and Zhanxing Zhu. “Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting”. In: *arXiv preprint arXiv:1709.04875* (2017).
 - [171] Xiang Yu et al. “An adaptive method based on contextual anomaly detection in internet of things through wireless sensor networks”. In: *International Journal of Distributed Sensor Networks* 16.5 (2020), p. 1550147720920478.
 - [172] Jian Zhang, Jennifer Rexford, and Joan Feigenbaum. “Learning-based anomaly detection in BGP updates”. In: *Proceedings of the 2005 ACM SIGCOMM workshop on Mining network data*. 2005, pp. 219–220.
 - [173] Yin Zhang et al. “Network anomography”. In: *Proceedings of the 5th ACM SIGCOMM conference on Internet Measurement*. 2005, pp. 30–30.
 - [174] Ying Zhang. *Abilene Traffic Matrix Dataset*. Accessed: 2025-03-28. 2004. URL: <http://cs.utexas.edu/~yzhang/research/AbileneTM/>.
 - [175] Zheng Zhang et al. “iSPY: Detecting IP prefix hijacking on my own”. In: *Proceedings of the ACM SIGCOMM 2008 conference on Data Communication*. 2008, pp. 327–338.
 - [176] Haoyi Zhou et al. “Informer: Beyond efficient transformer for long sequence time-series forecasting”. In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 35. 12. 2021, pp. 11106–11115.
 - [177] Jiawei Zhou et al. “Automating botnet detection with graph neural networks”. In: *arXiv preprint arXiv:2003.06344* (2020).
 - [178] Jie Zhou et al. “Graph neural networks: A review of methods and applications”. In: *AI open* 1 (2020), pp. 57–81.